

作者：C3程序猿

C语言全面学习

内容包括：(共370课时 总50+小时)

0、198 页精美 PDF 课件

- 1、vs2022 安装使用教程 (15 课时, 1.5 小时)
- 2、零基础入门 (158 课时, 23 小时, 精学)
- 3、深入存储原理以及常见问题(83 课时, 提高篇, 13 小时)
- 4、29 个标准库详细介绍(84 课时, 拓展篇, 15 小时, 粗略学, 重印象)
- 5、新标准(C99 C11 C17 C23)(31 课时, 新标准, 4 小时, 快乐学)

1、编译器：Visual Studio 2022

大家有什么用什么，会用什么用什么，区别不是很大

2、参考资料：咱们的PDF 比市面上的书内容全面

我以前用的C Primer Plus 差不多8年了，1/3没看完

3、咱们晚上7:00 ~ 10:00左右，在B站给大家直播，等待大家来问问题

大家想听什么内容没讲到的，可以随时补充上去

4、课程特点：全面、细致

不敢说最全面的C语言知识点吧，但是也差不多

每个知识点都给大家白话细致介绍，不留问题，不留疑问

课程目录如下：

vs2022 安装使用教程 (15 课时，1.5 小时)

- 1、.net 安装.mp4
- 2、vs2022 下载.mp4
- 3、vs2022 安装.mp4
- 4、vs2022 无法下载.mp4
- 5、选择语言环境.mp4
- 6、账户登录不上.mp4
- 7、创建项目.mp4
- 8、文件筛选器与虚拟目录.mp4
- 9、添加源文件.mp4
- 10、各类窗口显示与关闭.mp4
- 11、窗口布局.mp4
- 12、生成与重新生成.mp4
- 13、代码的调试.mp4
- 14、设置字体、行号、背景色.mp4
- 15、同一个工程管理多个项目.mp4

基础入门(158 课时，23 小时，精学)

- | | |
|-----------------------------|------------------------------|
| 1、C 语言知识结构(11 分 37).mp4 | 38、逻辑运算符(7 分 25).mp4 |
| 2、主函数 6 种形式(18 分 23).mp4 | 39、范围写法(6 分 39).mp4 |
| 3、注释(11 分 53).mp4 | 40、逗号运算符(9 分 38).mp4 |
| 4、printf 函数与卡主(12 分 16).mp4 | 41、if 结构(7 分 28).mp4 |
| 5、换行与内建函数(7 分 35).mp4 | 42、if 结构注意点(11 分 22).mp4 |
| 6、整型常量与前缀(10 分 52).mp4 | 43、if-else 结构(10 分 49).mp4 |
| 7、整型常量的输出(8 分 41).mp4 | 44、条件表达式(9 分 38).mp4 |
| 8、内存单位简介(4 分 21).mp4 | 45、多选一结构(7 分 23).mp4 |
| 9、整型类型的分类(13 分 17).mp4 | 46、switch 结构(12 分 13).mp4 |
| 10、整型常量后缀(11 分 00).mp4 | 47、switch 结构表示范围(8 分 21).mp4 |
| 11、变量的声明与定义(13 分 49).mp4 | 48、while 循环(12 分 51).mp4 |
| 12 连续定义(5 分 31).mp4 | 49、循环三要素(7 分 35).mp4 |
| 13、变量的赋值(10 分 11).mp4 | 50、for 循环(8 分 04).mp4 |
| 14、连续赋值(3 分 56).mp4 | 51、for 循环的灵活性(10 分 10).mp4 |
| 15、其他整型格式说明符(9 分 47).mp4 | 52、do-while 循环(9 分 01).mp4 |
| 16、变量的地址(9 分 05).mp4 | 53、break(7 分 11).mp4 |
| 17、scanf 输入整型(14 分 00).mp4 | 54、continue(9 分 09).mp4 |
| 18、scanf 的分隔符(10 分 07).mp4 | 55、循环嵌套(9 分 55).mp4 |
| 19、sizeof(5 分 05).mp4 | 56、跳出多层循环(6 分 29).mp4 |
| 20、signed(4 分 16).mp4 | 57、一维数组的声明与初始化(11 分 22).mp4 |
| 21、浮点型常量(6 分 18).mp4 | 58、数组定义注意点(6 分 44).mp4 |
| 22、浮点型的属性(14 分 13).mp4 | 59、数组元素访问(10 分 53).mp4 |
| 23、控制小数位数(5 分 19).mp4 | 60、循环遍历数组(2 分 07).mp4 |
| 24、浮点型的输入(5 分 16).mp4 | 61、数组的大小(4 分 19).mp4 |
| 25、浮点型转换成整形(3 分 35).mp4 | 62、一维数组简介(3 分 57).mp4 |
| 26、字符常量(13 分 20).mp4 | 63、一维数组初始化(7 分 48).mp4 |
| 27、字符的输出(7 分 31).mp4 | 64、二维数组的下标(5 分 51).mp4 |
| 28 字符变量的输入(5 分 10).mp4 | 65、二维数组的遍历(5 分 41).mp4 |
| 29、字符输入前清空缓冲区(11 分 58).mp4 | 66、二维数组元素操作(5 分 00).mp4 |
| 30、算数运算符(17 分 17).mp4 | 67、二维数组的大小(2 分 28).mp4 |
| 31、小括号运算符(3 分 31).mp4 | 68、指针的简介(6 分 37).mp4 |
| 32、优先级与结合性(9 分 28).mp4 | 69、指针变量的声明(4 分 42).mp4 |
| 33、复合赋值运算符(3 分 58).mp4 | 70、指针变量的初始化(8 分 03).mp4 |
| 34、前置++与后置++(10 分 39).mp4 | 71、NULL 简介(5 分 26).mp4 |
| 35、前置--与后置--(4 分 52).mp4 | 72、指针的赋值(2 分 53).mp4 |
| 36、自加自减运算符的注意点(5 分 42).mp4 | 73、地址操作符(7 分 44).mp4 |
| 37、关系运算符(8 分 13).mp4 | 74、指针操作空间(4 分 26).mp4 |

- 75、不同类型的指针指向(8分55).mp4
- 76、二级指针(7分42).mp4
- 77、指针数组(7分05).mp4
- 78、指针的偏移计算(7分54).mp4
- 79、指针操作一维数组(9分53).mp4
- 80、数组名是首元素的首地址(3分06).mp4
- 81、下标运算(5分04).mp4
- 82、一维数组指针(13分08).mp4
- 83、一维数组指针的操作(8分43).mp4
- 84、二维数组指针(4分45).mp4
- 85、一维指针遍历二维数组(11分49).mp4
- 86、元素类型访问二维数组(4分57).mp4
- 87、指针的大小(7分15).mp4
- 88、通用类型指针(8分06).mp4
- 89、函数功能简介(5分14).mp4
- 90、无参无返函数定义(6分47).mp4
- 91、函数调用(11分02).mp4
- 92、函数声明(10分15).mp4
- 93、函数返回值(11分22).mp4
- 94、return的作用(4分38).mp4
- 95、不是所有路径都有返回值(7分46).mp4
- 96、谨慎返回局部变量地址(14分56).mp4
- 97、函数的参数(11分26).mp4
- 98、传递一维数组(11分17).mp4
- 99、一维数组做参数(4分29).mp4
- 100、传递二维数组(11分33).mp4
- 101、二维数组做参数(3分44).mp4
- 102、传址调用(9分58).mp4
- 103、传2级指针(6分34).mp4
- 104、递归的初步理解(6分40).mp4
- 105、循环角度理解递归(6分18).mp4
- 106、展开理解递归(9分40).mp4
- 107、通向公式递归方法(9分48).mp4
- 108、函数指针(14分36).mp4
- 109、堆区空间简单介绍(10分01).mp4
- 110、malloc函数的使用(16分25).mp4
- 111、空间使用(9分20).mp4
- 112、free释放空间(9分14).mp4
- 113、msize函数(4分59).mp4
- 114、realloc与calloc(10分10).mp4
- 115、字符数组(4分00).mp4
- 116、字符串结尾(6分44).mp4
- 117、字符串简介(3分56).mp4
- 118、常量字符串(10分01).mp4
- 119、常量字符串初始化(10分22).mp4
- 120、字符指针(8分04).mp4
- 121、字符串输出(16分12).mp4
- 122、字符串输入(9分08).mp4
- 123、字符串操作函数(17分36).mp4
- 124、结构体类型介绍(8分46).mp4
- 125、结构体变量定义(4分54).mp4
- 126、成员使用(8分51).mp4
- 127、无名结构体(2分26).mp4
- 128、特殊结构体成员(8分42).mp4
- 129、联合(10分40).mp4
- 130、枚举(9分55).mp4
- 131、枚举设置指定值(3分03).mp4
- 132、枚举的应用(8分02).mp4
- 133、文件操作简单介绍(8分21).mp4
- 134、fopen与fopen_s(11分33).mp4
- 135、相对路径与绝对路径(7分19).mp4
- 136、文本模式与二进制模式(12分54).mp4
- 137、fputc与fgetc(12分29).mp4
- 138、循环读文件(6分43).mp4
- 139、fputs与fgets(13分33).mp4
- 140、fscanf与fprintf(9分07).mp4
- 141、fread与fwrite(16分57).mp4
- 142、ftell rewind fseek(20分32).mp4
- 143、进制转换(9分49).mp4
- 144、位运算符(13分07).mp4
- 145、位左移(9分04).mp4
- 146、位右移(9分03).mp4
- 147、多文件介绍(15分26).mp4
- 148、尖括号与双引号的区别(15分41).mp4
- 149、防止头文件重复包含(14分54).mp4
- 150、宏(4分27).mp4
- 151、宏不计算(3分44).mp4
- 152、参数宏(6分52).mp4
- 153typedef(15分49).mp4
- 154、全局变量extern(16分48).mp4
- 155、static(13分47).mp4
- 156、auto局部变量(7分52).mp4
- 157、register(4分25).mp4
- 158、const(6分54).mp4

深入存储原理以及常见问题(83课时，提高篇，13小时，着急可跳)

- 1、课程介绍.mp4
- 2、运算符优先级与结合性本意.mp4
- 3、for循环的通用写法.mp4
- 4、vs下a++++a的运算结果.mp4
- 5、gcc下a++++a的运算结果.mp4
- 6、真短路或.mp4
- 7、假短路与.mp4
- 8、与或混合短路(1).mp4
- 9、与或混合短路(2).mp4
- 10、短路原理与代码优化.mp4
- 11、scanf报错问题.mp4
- 13、strcpy更安全?.mp4
- 14、缓冲区的基本理解.mp4
- 15、缓冲区的主要作用.mp4
- 16、输入缓冲区的大小.mp4
- 17、scanf输入变量的具体过程.mp4
- 18、输入字符总失败.mp4
- 19、循环清空缓冲区.mp4
- 20、库函数清空缓冲区.mp4
- 21、字节对齐的好处.mp4
- 22、结构体大小计算.mp4
- 23、设置字节对齐数.mp4
- 24、补码计算.mp4

- 12、scanf 更安全?. mp4
- 25、验证补码对错. mp4
- 26、大小端存储. mp4
- 27、测试大小端存储. mp4
- 28、大小端转换. mp4
- 29、unsigned char c=-12. mp4
- 30、越界存储处理. mp4
- 31、整型类型的边界. mp4
- 32、浮点型转二进制. mp4
- 33、浮点型符号位存储(8分53). mp4
- 34、指数设定1, 全为0(18分42). mp4
- 35、指数设定2, 全为1, 尾数全为0, 表示无穷(9分37). mp4
- 36、指数设定3, 全为1, 位数不全为0, nan(8分24). mp4
- 37、尾数位的正常值计算(11分57). mp4
- 38、尾数位的存储(3分40). mp4
- 39、float 类型范围(10分05). mp4
- 40、double 类型的范围(3分40). mp4
- 41、浮点型的精度位计算(18分28). mp4
- 42、浮点型实现相等比较(11分11). mp4
- 43、判断浮点型正常与否(3分46). mp4
- 44、浮点型16进制形式p记法(10分17). mp4
- 45、浮点型16进制2进制转换(3分22). mp4
- 46、8421码与421码(4分31). mp4
- 47、赋值转换(8分52). mp4
- 48、函数参数提升转换(19分10). mp4
- 49、表达式类型提升(18分07). mp4
- 50、指针的隐式转换(15分12). mp4
- 51、强制类型转换(5分51). mp4
- 52、指针的强制转换(12分19). mp4
- 53、a[2]与2[a(13分12). mp4
- 54、复杂指针的解析(12分27). mp4
- 55、复杂指针的理解(2)(12分59). mp4
- 56、复杂指针的理解(3)(7分26). mp4
- 57、内存分区总结(20分31). mp4
- 58、命令行参数介绍(7分44). mp4
- 59、命令行参数主函数(11分04). mp4
- 60、调试传递命令行参数(5分54). mp4
- 61、scanf与printf返回值(9分13). mp4
- 62、浮点型输出格式控制(8分07). mp4
- 63、整型输出格式控制(3分59). mp4
- 64、输入格式控制(11分29). mp4
- 65、volatile(7分19). mp4
- 66、restrict(12分25). mp4
- 67、字母的转义字符(10分58). mp4
- 68、特殊转义(7分03). mp4
- 69、数字的转义字符(7分19). mp4
- 70、单引号装多个字符(6分34). mp4
- 71、位域、位字段简介(12分46). mp4
- 72、多个位的字段(5分03). mp4
- 73、位字段对齐(13分08). mp4
- 74、位字段控制位(8分52). mp4
- 75、#与##(6分43). mp4
- 76、VA_ARGS与#undef(4分34). mp4
- 77、条件编译(8分36). mp4
- 78、预定义宏名(9分00). mp4
- 79、#line(4分44). mp4
- 80、#error(2分39). mp4
- 81、#pragma 常用(12分33). mp4
- 82、#pragma 不常用(5分01). mp4
- 83、宏拼接(4分55). mp4

29个标准库详细介绍(84课时, 拓展篇, 15小时, 粗略学, 重印象)

- 1、标准头文件简介(8分04). mp4
- 2、stdarg(1)可变参数(10分27). mp4
- 3、stdarg(2)可变参数的解读(15分20). mp4
- 4、stdbool(1)布尔类型(11分22). mp4
- 5、time(1)时间库time简介(5分02). mp4
- 6、time(2)时间操作time,ctime(17分08). mp4
- 7、time(3)计算时间差difftime(2分11). mp4
- 8、time(4)cpu时间clock(5分00). mp4
- 9、time(5)获取对应基底时间(8分04). mp4
- 10、time(6)localtime(8分06). mp4
- 11、time(7)asctime(4分28). mp4
- 12、time(8)gmtime(5分16). mp4
- 13、time(9)strftime与mktime(5分30). mp4
- 14、assert(1)断言(11分42). mp4
- 15、error(1)错误获取errno(16分11). mp4
- 16、limits(1)整型范围(6分29). mp4
- 17、stdint(1)整型属性(8分31). mp4
- 18、inttypes(1)整型格式说明符(3分56). mp4
- 19、inttypes(2)整型转换函数(11分37). mp4
- 20、inttypes(3)其他转换函数(4分12). mp4
- 21、float(1)浮点型属性(23分38). mp4
- 22、float(2)浮点型操作函数(7分09). mp4
- 23、float(3)浮点型的状态(10分47). mp4
- 24、fenv(1)浮点型的舍入方向(13分19). mp4
- 25、fenv(2)浮点型的状态(4分37). mp4
- 26、fenv(3)浮点型状态设置获取函数(12分01). mp4
- 27、iso646(1)运算符的宏名(2分41). mp4
- 28、locale(1)语言环境与字符集(8分26). mp4
- 29、locale(2)设置余元环境setlocale(7分52). mp4
- 30、locale(3)localeconv(18分01). mp4
- 31、locale(4)createlocale(13分35). mp4
- 32、locale(5)边角料函数(9分41). mp4
- 33、ctype(1)字符识别函数(20分49). mp4
- 34、ctype(2)转大写toupper tolower(3分19). mp4
- 35、ctype(3)isascii toascii(3分12). mp4
- 36、ctype(4)iscsymf iscsmy(2分09). mp4
- 37、wchar(1)多字节与宽字符介绍(10分41). mp4
- 38、wchar(2)宽字符wchar_t详解(20分01). mp4
- 39、wchar(3)btowc mbrlen mbstowc(11分15). mp4
- 40、wchar(4)多字节与宽字符的互相转换函数(14分28). mp4
- 41、wchar(5)宽字符串操作函数(10分24). mp4
- 42、wctype(1)宽字符识别函数(19分21). mp4

- 43、uchar(1)转换 24 字节的字符(8 分 03).mp4
- 44、string(1)头文件(15 分 08).mp4
- 45、string(2)(15 分 43).mp4
- 46、string(3)(13 分 23).mp4
- 47、string(4)mem 系函数(17 分 11).mp4
- 48、stdio(1)printf 系函数(12 分 35).mp4
- 49、stdio(2)vprintf 系列函数(8 分 15).mp4
- 50、stdio(3)scanf 系列函数(10 分 42).mp4
- 51、stdio(4)字符输入系列函数(5 分 45).mp4
- 52、stdio(5)字符输出系列函数(3 分 42).mp4
- 53、stdio(6)文件操作函数(1)(16 分 09).mp4
- 54、stdio(7)文件操作好玩儿函数 u(2)(23 分 56).mp4
- 55、stdlib(1)内存管理(11 分 12).mp4
- 56、stdlib(2)终止进程函数(17 分 22).mp4
- 57、stdlib(3)终止进程(16 分 35).mp4
- 58、stdlib(4)环境变量设置(19 分 58).mp4
- 59、stdlib(5)路径生成器(10 分 15).mp4
- 60、stdlib(6)整数字符串转整数(10 分 54).mp4
- 61、stdlib(7)小数字符串转小数(2 分 52).mp4
- 62、stdlib(8)数值转字符串(12 分 58).mp4
- 63、stdlib(9)多字节与宽字节转换(10 分 01).mp4
- 63、stdlib(9)数的旋转(7 分 51).mp4
- 64、stdlib(10)随机数(13 分 37).mp4
- 65、stdlib(11)随机数 3 种应用算法(12 分 39).mp4
- 66、stdlib(12)排序 qsort 与 qsort s(15 分 57).mp4
- 67、stdlib(13)查找元素算法(9 分 14).mp4
- 68、stdlib(14)数学函数(7 分 03).mp4
- 69、stddef 常用宏(10 分 42).mp4
- 70、stdnoreturn 便利宏(5 分 23).mp4
- 71、stdalign 便利宏 Alignas Alignof(8 分 32).mp4
- 72、math(1)浮点型基础运算(27 分 12).mp4
- 73、math(2)指数对数幂运算(2 分 44).mp4
- 74、math(3)三角函数双曲函数高斯伽玛函数(3 分 31).mp4
- 75、math(4)浮点型的终极运算(19 分 30).mp4
- 76、complex 复数运算(7 分 18).mp4
- 77、tqmath 泛型数学库(1 分 06).mp4
- 78、setjmp(1)跳跃 goto(12 分 52).mp4
- 79、setjmp(2)非局部跳跃(17 分 18).mp4
- 80、signal(1)注册信号(12 分 01).mp4
- 81、signal(2)raise 函数(9 分 09).mp4
- 82、signal(3)其他的信号介绍(6 分 28).mp4
- 83、threads 线程操作库(12 分 25).mp4
- 84、stdatomic(1)原子操作库(12 分 04).mp4

新标准(C99 C11 C17 C23)(31 课时，新标准，4 小时，快乐学)

- 1、标准简介(12 分 26).mp4
- 2、C99(1)移除隐式 int(4 分 12).mp4
- 3、C99(2)gets 函数移除(2 分 34).mp4
- 4、C99(3)标识符字符拓展(6 分 57).mp4
- 5、C99(4)注释(1 分 25).mp4
- 6、C99(5)新增数据类型(11 分 34).mp4
- 7、C99(6)柔性数组(1)(15 分 38).mp4
- 8、C99(6)柔性数组(2)(7 分 08).mp4
- 9、C99(7)变长数组 VLA(12 分 09).mp4
- 10、C99(8)初始化指定元素(8 分 54).mp4
- 11、C99(9)复合文字(14 分 41).mp4
- 12、C99(10)restrict(5 分 48).mp4
- 13、C99(11 12 13 14)讲过的(7 分 22).mp4
- 14、C99(15 16)变量随时定义(8 分 07).mp4
- 15、C99(17)inline 内联函数(13 分 04).mp4
- 16、C99(18)函数参数新特性(12 分 27).mp4
- 17、C99(19 20)可变参数宏与预定义宏(14 分 55).mp4
- 18、C99(21) Pragma(9 分 47).mp4
- 19、C11(1)原子操作(1 分 48).mp4
- 20、C11(2)存储类说明符 Thread local(9 分 47).mp4
- 21、C11(3) Alignas 与 Alignof(3 分 12).mp4
- 22、C11(4)字符与字符串的前缀(3 分 40).mp4
- 23、C11(5)泛型编程 Generic(1)(12 分 04).mp4
- 24、C11(5)泛型编程 Generic(2)(14 分 41).mp4
- 25、C11(67)匿名结构体与联合(8 分 36).mp4
- 26、C11(8 9 11)静态断言(5 分 27).mp4
- 27、C11(10)文件操作模式 ux(9 分 16).mp4
- 28、C17 与 C11 一样(2 分 01).mp4
- 29、C23(1 2)10 进制浮点型与 2 进制显示(3 分 58).mp4
- 30、C23(3)设置代码属性(16 分 53).mp4
- 31、C23(4 5 6 7)其他简单特性(8 分 23).mp4

C 语言知识结构：

编程的核心目的：计算数据

0、存储数据：数据类型

基本数据类型：整型(整数)，浮点型(小数)，字符型(ASCII 码)

构造数据类型：数组，指针，结构体，联合，枚举

由其他类型自定义结合而成的类型。

1、输入输出：数据显示与获取

控制台输入输出函数

文件输入输出函数：文件操作，操作硬盘存储

2、计算数据：运算符，看表

3、逻辑控制：流程结构

顺序结构，分支跳转结构，循环结构

4、模块化编程：函数

5、其他：存储类，多文件，预处理指令

运算符表

优先级			代码	结合性
一	++	后置++	b++	从左到右
	--	后置--	b--	
	()	改运算顺序	(4+2)*6	
	()	函数参数列表	fun(1,4)	
	[]	下标运算	arr[5]	
	{ }	数组初始化	int a[3] = {1,2,3}	
	.	变量取成员	stu.name	
二	->	指针取成员	pStu->name	从右到左
	++	前置++	++b	
	--	前置--	--b	
	+	正号	+20	
	-	负号	-20	
	~	按位取反	~b	
	!	取反	!1	
	sizeof	算字节数	sizeof(int)	
	*	内存操作符	*pp	
	&	取地址符	&b	
第一组	(type)	强制类型转换	(float)b	从左到右
	(type name)	复合文字	(int[5]){1,2,3,4,5}	
三	*	乘法	12*13	从左到右
	/	整数取整 小数除法	3/2=1 3.0/2.0=1.5	
	%	整数取余	22%4	
四	+	加法	12+13	从左向右
	-	减法	12-13	

五	>>	位右移	13>>1	从左向右
	<<	位左移	5<<12	
六	>	大于	12>11	从左向右
	<	小于	14<51	
	>=	大于等于	22>=12	
	<=	小于等于	14<=44	
七	==	等于	12==12	从左到右
	!=	不等于	12!=11	
八	&	按位与	12&16	从左到右
九	^	按位异或	12^16	从左到右
十		按位或	12 16	从左到右
十一	&&	逻辑与	真&&假	从左到右
十二		逻辑或	假 真	从左到右
十三	? :	条件运算符	1>2?1:0	从右向左
十四	=	赋值运算符	b = 14	从右向左
	+=	加后赋值	b+=14	
	-=	减后赋值	b-=14	
	=	乘后赋值	b=14	
	/=	除后赋值	b/=14	
	%=	取余后赋值	b%=14	
	>>=	位右移后赋值	b>>=14	
	<<=	位左移赋值	b<<=14	从右到左
	&=	按位与后赋值	b&=14	
	^=	按位异或后赋值	b^=14	
	=	按位或后赋值	b =14	
十五	,	逗号运算符	(2+3,4,5-3)	从左到右

ASCII 码表

ASCII 值	控制字符	解释	ASCII 值	控制字符	解释
0x00 0 000	NUL	\0	0x20 32 040	(space)	空格
0x01 1 001	SOH	标题开始	0x21 33 041	!	感叹号
0x02 2 002	STX	正文开始	0x22 34 042	"	引号 (双引号)
0x03 3 003	ETX	正文结束	0x23 35 043	#	数字符号
0x04 4 004	EOT	传输结束	0x24 36 044	\$	美元符
0x05 5 005	END	询问字符	0x25 37 045	%	百分号
0x06 6 006	ACK	认可	0x26 38 046	&	和号
0x07 7 007	BEL	\a	0x27 39 047	'	单引号
0x08 8 010	BS	\b	0x28 40 050	(	空格
0x09 9 011	HT	\t	0x29 41 051	)	感叹号
0x0A 10 012	LF	\n	0x2A 42 052	*	引号 (双引号)

0x0B 11 013	VT	\v	0x2B 43 053	+	
0x0C 12 014	FF	\f	0x2C 44 054	,	
0x0D 13 015	CR	\r	0x2D 45 055	-	
0x0E 14 016	SO	移出	0x2E 46 056	.	
0x0F 15 017	SI	移入	0x2F 47 057	/	正斜杠
0x10 16 020	DLE	认可	0x30 48 060	0	
0x11 17 021	DC1	设备控制 1	0x31 49 061	1	
0x12 18 022	DC2	设备控制 2	0x32 50 062	2	
0x13 19 023	DC3	设备控制 3	0x33 51 063	3	
0x14 20 024	DC4	设备控制 4	0x34 52 064	4	
0x15 21 025	NAK	否定接受	0x35 53 065	5	
0x16 22 026	SYN	同步闲置符	0x36 54 066	6	
0x17 23 027	ETB	传输块结束	0x37 55 067	7	
0x18 24 030	CAN	取消	0x38 56 070	8	
0x19 25 031	EM	媒体结束	0x39 57 071	9	
0x1A 26 032	SUB	替换	0x3A 58 072	:	
0x1B 27 033	ESC	换码符	0x3B 59 073	;	
0x1C 28 034	FS	文件分隔符	0x3C 60 074	<	
0x1D 29 035	GS	组分分隔符	0x3D 61 075	=	
0x1E 30 036	RS	记录分隔符	0x3E 62 076	>	
0x1F 31 037	US	单位分隔符	0x3F 63 077	?	
0x40 64 100	@		0x60 96 140	`	
0x41 65 101	A		0x61 97 141	a	
0x42 66 102	B		0x62 98 142	b	
0x43 67 103	C		0x63 99 143	c	
0x44 68 104	D		0x64 100 144	d	
0x45 69 105	E		0x65 101 145	e	
0x46 70 106	F		0x66 102 146	f	
0x47 71 107	G		0x67 103 147	g	
0x48 72 110	H		0x68 104 150	h	
0x49 73 111	I		0x69 105 151	i	
0x4A 74 112	J		0x6A 106 152	j	
0x4B 75 113	K		0x6B 107 153	k	
0x4C 76 114	L		0x6C 108 154	l	

0x4D 77 115	M		0x6D 109 155	m	
0x4E 78 116	N		0x6E 110 156	n	
0x4F 79 117	O		0x6F 111 157	o	
0x50 80 120	P		0x70 112 160	p	
0x51 81 121	Q		0x71 113 161	q	
0x52 82 122	R		0x72 114 162	r	
0x53 83 123	X		0x73 115 163	s	
0x54 84 124	T		0x74 116 164	t	
0x55 85 125	U		0x75 117 165	u	
0x56 86 126	V		0x76 118 166	v	
0x57 87 127	W		0x77 119 167	w	
0x58 88 130	X		0x78 120 170	x	
0x59 89 131	Y		0x79 121 171	y	
0x5A 90 132	Z		0x7A 122 172	z	
0x5B 91 133	[		0x7B 123 173	{	
0x5C 92 134	\	反斜杠	0x7C 124 174		
0x5D 93 135	]		0x7D 125 175	}	
0x5E 94 136	^		0x7E 126 176	~	
0x5F 95 137	—		0x7F 127 177	DEL	删除

main 函数

main 函数叫主函数，是应用程序的入口，即一堆代码的执行起点。

形式如下：

<pre>main() { //C 语言最初的主函数形态。 }</pre>	<pre>int main(void) { //c99 标准主函数的形态 return 0; //可以不写 }</pre>
<pre>int main(int argc, char**argv) //char*argv[] { //最通用的命令行参数的主函数形态 return 0; }</pre>	<pre>int main(int argc, char**argv, char*env[]) { //命令行参数的主函数形态 return 0; }</pre>
<pre>int main() { //C++的主函数形态，C 也支持。 return 0; }</pre>	<pre>void main() { //非标准主函数形态 //有的编译器不支持 }</pre>

选择：根据学校老师教的哪个就用哪个。自己学习，就用标准点儿的。

错误注意：

- 1、一个工程中只能有 1 个主函数。
- 2、符号都是英文的，中文报错。
- 3、main 别写错，MAIN、Main、mian 等都是错误写法。
- 4、main()后别加分号。

```
main();
{
    //C 语言最初的主函数形态。
}
```

- 5、大括号风格随意。

<pre>main() { //C 语言最初的主函数形态。 }</pre>	<pre>main() { //C 语言最初的主函数形态。 }</pre>
---	---

注释：增加代码的可读性。对代码进行说明，方便他人对代码理解，方便后期自己认识。

如下：一看便知整个代码的基本逻辑，如果没有注释，那就不容易看了

```
void AddToHead(int iData)
{
    //创建节点，申请节点空间
    struct Node* pTemp = (struct Node*)malloc(sizeof(struct Node));
    //如果节点申请失败，结束函数
    if (NULL == pTemp)
        return;
    //节点成员赋值
    pTemp->iData = iData;
    pTemp->pNext = NULL;
    pTemp->pPre = NULL;
    //链接到链表上，链表头添加
    pTemp->pNext = pHead;
    pHead->pPre = pTemp;
}
```

注释语法：

单行注释： //

多行注释： /*

*/

多行注释不能嵌套，直接报错：

```
/*    1
    /*    3
    */    4
*/    2
```

逻辑上，1 匹配 2，但是实际 1 匹配 4，导致 2 只有半个，错误语法

vs 快捷注释按钮：(视图->工具栏->文本编辑器)

特点：

注释的内容不会称为最终编译的软件的一部分，所以写多少都对最终程序没有一点儿影响。
也可以用于将旧代码注释，而不是删除，保留学习痕迹。

输出一段：hello C3 程序猿

包含头文件： #include <stdio.h>

标准输入输出头文件 standard input output head

头文件是相应库函数的声明

输出函数： printf("hello C3 程序猿\n")

功能是将双引号中的内容显示在控制台窗口，没有它就不能显示。

\n：换行作用，可以加多个，演示效果

有分号结尾：分号作为一条语句的结束的标记。

有的编译器中，不加头文件也能使用 printf，原因是该编译器将 printf 设计成了内置函数，名为内建函数，但是该行为不是所有编译器都支持，所以建议加上头文件

```
#include <stdio.h>
#include <stdlib.h> //卡主头文件
//项目属性->连接器->系统->子系统->未设置
取消vs三行提示
int main(void)
{
    printf("hello C3程序猿\n");

    system("pause>0"); //卡主
    return 0;
}
```

数据类型本质就是对各种数据进行分类，类型名就是对应分类的名字。不同类型的数据长相不一样，计算机处理的方式也不同，所以分类处理有利于计算机的计算速度。

基本数据类型：整型(整数)，浮点型(小数)，字符型(ASCII 码)

整型与浮点型也叫数值类型，字符型就是字符型

构造数据类型：数组，指针，结构体，联合，枚举

由其他类型自定义结合而成的类型。

整数(正整数，负整数，0)的类型就叫整型数据类型

主要内容：

整型常量

整型变量**声明/定义/使用/赋值**

整型数据的输入/输出

整型各类型区别

常量就是常数，恒定不可被改变的：-3, 0, 5

整型常量形式有四：

125 默认是 10 进制

0125 0 前缀的 8 进制，对应10进制85

0x125 0x/0X 前缀是 16 进制，对应10进制293

0b1010 0b/0B 前缀是 2 进制 1010，c23 标准新增，老版编译器不支持

整型常量的输出：格式说明符

16 进制形式：%x %X 没有负数形式

10 进制形式：%d 正负都有

8 进制形式：%o 没有负数形式

没有 2 进制的输出格式说明符。

代码演示：数据会替换格式说明符的位置

```
printf("value16:%x value10:%d value8:%o\n", 26, 26, 26);
```

```
printf("value16:%x value10:%d value8:%o\n", 0253, 0253, 0253);
```

```
printf("value16:%x value10:%d value8:%o\n", 0xa4, 0xa4, 0xa4);
```

输出：

```
value16:1a value10:26 value8:32
```

```
value16:ab value10:171 value8:253
```

```
value16:a4 value10:164 value8:244
```

1a==26==032 三者长相不一样，但是数值大小一模一样，计算机也是以 2 进制：11010 对待他们三个。

字节:

内存大小的描述单位, 身高 1.8m 180cm, 米和厘米都是数量的单位。不可观, 大概是一堆与非门构成。

计算机常用的单位是字节, 数据的读写都是按字节读写的, 最小读写 1 字节

计算机最小的存储单位是一个二进制位, 简称位, 1 位存 1 或者 0。

位 bit
字节 byte 1byte==8bit
千字节 KB 1kb == 1024byte
兆字节 MB 1mb == 1024kb
吉字节 GB 1GB == 1024MB 千兆字节
太字节 TB 1TB == 1024GB 万亿字节, 一般个人电脑硬盘
帕字节 PB 1PB == 1024TB 千万亿字节
艾字节 EB 1EB == 1024PB
ZB YB BB NB DB CB XB

整型根据数值的大小范围再次进行细分, 如下:

目的: 更高效的利用有限的存储空间

代码类型名	名称	字节数	可表示数据范围
short	短整形	2 字节	-32768 ~ 32767
unsigned short	无符号短整形	2 字节	0 ~ 65535
int	整型	2 字节	-32768 ~ 32767
		4 字节	-2147483648 ~ 2147483647
unsigned int	无符号整形	2 字节	0 ~ 65535
		4 字节	0 ~ 4294967295
long	长整型	4 字节	-2147483648 ~ 2147483647
		8 字节	-9223372036854775808 9223372036854775807
unsigned long	无符号长整形	4 字节	0 ~ 4294967295
		8 字节	0 ~ 18446744073709551615
long long(c99)	超长整型	8 字节	-9223372036854775808 9223372036854775807
unsigned long long(c99)	无符号超长整形	8 字节	0 ~ 18446744073709551615

选择:

如果精细选择, 根据数据大小选择即可。

比如数据是 4 位数, 这些类型选哪个都行, 看你的心情。

类型名	标准字节数	实际字节数	可表示数据范围
short	不小于 2	2 字节	$-2^{15} \sim 2^{15} - 1$
unsigned short	不小于 2	2 字节	$0 \sim 2^{16} - 1$
int	不小于 2	2 字节	$-2^{15} \sim 2^{15} - 1$
		4 字节	$-2^{31} \sim 2^{31} - 1$
unsigned int	不小于 2	2 字节	$0 \sim 2^{16} - 1$
		4 字节	$0 \sim 2^{32} - 1$
long	不小于 4	4 字节	$-2^{31} \sim 2^{31} - 1$
		8 字节	$-2^{63} \sim 2^{63} - 1$
unsigned long	不小于 4	4 字节	$0 \sim 2^{32} - 1$
		8 字节	$0 \sim 2^{64} - 1$
long long	不小于 8	8 字节	$-2^{63} \sim 2^{63} - 1$
unsigned long long	不小于 8	8 字节	$0 \sim 2^{64} - 1$

这个 2 的次方的形式跟位数有关，记不记的都行。

整型常量后缀：

给定一个常数 12，这个 12 到底是哪个整型呢？由后缀决定！

整型常量后缀：一般用小写的，大小写均不影响

short	无后缀	-1,0,34
unsigned short	无后缀	0,34
int	无后缀	-1,0,34
unsigned int	U/u	23U,456u,0U
long	L/l	-12L,0L,45l
unsigned long	LU/lu/ul/UL/Lu/Lu/UI/UI	0UL,23Lu
long long (C99)	LL/lL/LI/IL	-12LL,0ll,45lL
unsigned long long (C99)	LLU/llu/ull/ULL/LLu/llu/Ull/uLL...	0ULL,23llu

short 类型无专门常量后缀，跟 int 共用。

每个类型对应格式说明符（必须小写）：

类型	10 进制	16 进制	8 进制
short	%hd	负无，正%hx	负无，正%ho
unsigned short	%hu	%hx	%ho
int	%i 或 %d	负无，正%x	负无，正%o
unsigned int	%u	%x	%o
long	%ld	负无，正%lx	负无，正%lo
unsigned long	%lu	%lx	%lo
long long (C99)	%lld	负无，正%llx	负无，正%llo
unsigned long long (C99)	%llu	%llx	%llo

使用 printf 输出。

unsigned称为无符号，即没有正负号，全是正数。

常量是恒定不变的，变量就是可以变化的，变量相当于仓库，装 12 就是 12，装 18 就是 18，新装的会把原来的覆盖掉。

声明、定义整形变量：

声明形式： 类型名 变量名;

定义形式： 类型名 变量名 = 初始值;

代码如下：

```
int a;           //声明变量 a
unsigned int b = 23u; //定义变量 b，也叫初始化
```

注意点：

- 1、类型名别写错，全都小写
- 2、类型名与变量之间空格隔开，空格个数不限制
- 3、结尾用英文分号
- 4、=叫赋值运算符，作用就是装，不是数学中的等于，数学中的等于在 C 语言里是==
- 5、23u 是数据，也可以用 23 对 b 变量赋值，但是由于类型不同，编译器第一先把 23 转成 23u，然后再将 23u 赋值给 b 变量，所以数据与变量类型一致，可以增加程序效率。
- 6、变量名规则：
 - 1、由大小写字母、下划线、数字组成，现代编译器也支持中文名了，但是可能不通用
 - 2、首字符不能是数字
- 7、也可以用另外一个变量初始化，如下：

```
unsigned int c = b;
```
- 8、变量没有初始化，默认未知其值。
- 9、变量名不能重复，报错重定义

连续定义：

```
int a, b = 3, d = b;
```

注意点：

- 1、中间用逗号隔开
- 2、定义先后顺序从左到右
- 3、相当于：两种喜欢哪个用哪个

```
int a ;
int b = 3;
int d = b;
```
- 4、可以分行写，如下

```
int a,
    b = 3,    //用逗号
    d = b;    //最后用分号
```
- 5、不同类型的，不能一起定义，只能分开单独定义。

变量的赋值

形式：变量名 = 数据 ;

如下：

```
a = 23;
```

注意点：

- 1、将右侧的赋值给左侧，左侧叫左操作数，右侧叫右操作数

2、赋值时，直接使用变量了，不用再加 int，名字只定义一次

3、=叫赋值运算符，可以变量给变量赋值，

4、只能给变量赋值，其他都不能被赋值，直接报错

```
a = 12;    //正确
```

```
34 = 5;    //直接报错
```

```
(b+a)=45; //直接报错，只能给变量赋值，(b+a)不管是啥，肯定不是变量
```

5、数据与变量类型不同时，系统先将变量的类型转换成跟变量一样的类型，然后再赋值

```
a = 56llu; //先将 56llu 转成 56，然后再将 56 赋值给 a
```

变量数据的输出：以 int 类型举例子

```
int a = 5, b = 3;
```

```
printf("%d, %i\n", a, b); //格式说明符的位置直接依次替换成数据参数。
```

int 类型的格式说明符是%i i 是 int 的首字符 也可用%d，一般习惯用%d

连续赋值：

```
a = b = 34;
```

执行过程：从右到左，相当于

```
b = 34;
```

```
a = b;
```

左侧操作数必须是变量，最右侧 1 个，可以是变量，可以是常量

其他整型类型变量定义：

```
short s = 34;
```

```
unsigned short us = 56;
```

```
int i = 34;
```

```
unsigned int ui = 45u;
```

```
long l = 34l;
```

```
unsigned long ul = 56lu;
```

```
long long ll = 34ll;
```

```
unsigned long long ull = 56llu;
```

所有输出：

```
printf("%hd, %hu\n", s, us);
```

```
printf("%d, %u\n", i, ui);
```

```
printf("%ld, %lu\n", l, ul);
```

```
printf("%lld, %llu\n", ll, ull);
```

变量的地址：

定义 1 个变量，系统就给该变量分配指定大小的空间，对应字节数。

比如：

int a = 12; ,系统为 a 分配 4 字节空间，如下图示



这连续的 4 字节存储 12 的 2 进制值。a 就是这段空间的名字。

0x10 0x11 0x12 0x13 分别是四个字节的地址值，地址就是内存的编号，按字节编号的，从 0 开始，一直到最后 1 个字节，地址与空间是一一对应的。0x10 永远对应固定的那字节空间。亘古不变。

获取地址形式：&a &叫取地址运算符，a 是变量名字。

地址输出格式说明符：%p 输出的是 16 进制形式。也可以用%d 转换成 10 进制输出

变量的输入

输入函数：scanf()

形式：

```
int a = 2;
int b = 3;
printf("a=%d, b=%d", a, b);
scanf_s("%d%d", &a, &b);    //一定要变量的地址
printf("a=%d, b=%d", a, b);
```

键盘输入：

12<空格>4<回车> 中间的空格可以用<回车><tab>分隔两个数据，不可以用其他

scanf_s 是新标准函数，c99 标准推荐使用，c11 之后的标准直接删掉了老版，要求使用新版。

分隔符：

如果想用逗号分隔，双引号内也必须用逗号分隔

```
scanf_s("%d,%d", &a, &b);
```

输入时：12,4<回车> 必须用逗号

同理，使用其他的字符分隔，输入格式必须跟双引号中一样

```
scanf_s("a=%d,b=%d", &a, &b);
```

输入时：a=12,b=4<回车> a= b=也必须输入

也别加\n，跟 printf 有本质的使用区别

```
scanf_s("%d%d\n", &a, &b);
```

输入时：12<空格>4<空格>56<回车> 这时必须要多输入一个数，否则 scanf_s 不走

变量的大小：sizeof

类型定义变量，变量占用固定字节数，该字节数前面表中有总结了。

代码类型名	名称	字节数
short	短整形	2 字节
unsigned short	无符号短整形	2 字节
int	整型	2 字节/4 字节
unsigned int	无符号整形	2 字节/4 字节
long	长整型	4 字节/8 字节
unsigned long	无符号长整形	4 字节/8 字节
long long(c99)	超长整型	8 字节
unsigned long long(c99)	无符号超长整形	8 字节

使用方式

```
unsigned int a = sizeof(double);    //a 装 8
```

```
unsigned int b = sizeof(a);          //b 装 4
```

```
long long d;
unsigned int c = sizeof d;    //当对变量名求 sizeof, 可以不加小括号
平时都加上小括号
```

输出：

```
printf("%u, %u", sizeof(double), sizeof(c));
```

signed ：有符号的关键字，即有正负数，可省略，跟 unsigned 对应

short -> signed short

int: 也是类型名的一部分，也可省略

short -> short int

全称: short -> signed short int

简写: short ，这 4 个名均可用

表如下：

名称	日常使用简写	其他写法
短整形	short	short int signed short signed short int
无符号短整形	unsigned short	unsigned short int
整型	int	signed int
无符号整型	unsigned int	unsigned int
长整型	long	long int signed long signed long int
无符号长整形	unsigned long	unsigned long int
超长整型	long long	long long int signed long long signed long long int
无符号超长整形	unsigned long long	unsigned long long int

浮点型：数学中的小数，也叫实数

浮点型常量：

基本形式：-386.54 0.0 0.000053

科学计数法：-3.8654e2 5.3E-5

e 可大写可小写，e2 就是 10 的 2 次方，e-5 就是 10 的-5 次方

次方必须为整数

特殊写法：

.054 .5e4 即 0.054, 0.5e4, 前面的 0 可以省略

5. 65.e-3 即 5.0, 65.0, 后面的 0 可以省略

浮点型类型属性：

类型名	字节数	后缀	格式说明符	科学记数法	范围	精度(有效数位)
float	4	f / F	%f	%e	-3.40e+38F 3.40e+38F	8 (6)
double	8	无	%lf		-1.79e+308 1.79e+308	17 (10)
long double	8	l / L	%Lf		-1.79e+308 1.79e+308	17 (10)
	10				-1.18e+4932 1.18e+4932	22 (10)
	16				-1.18e+4932 1.18e+4932	38 (10)

1、long 类型常量也是后缀 L，区别：4l 4.5l 一眼就看出来

2、输出

```
printf("%lf", 45.6);
```

显示：45.600000 默认输出 6 位小数

3、控制小数位数

```
printf("%.1lf", 45.6);
```

显示：45.6 .1 表示输出 1 位小数

```
printf("%.2lf", 45.6789);
```

显示：45.68 .2 表示输出 2 位小数，第 3 位四舍五入

4、精度

float 叫单精度浮点型，double 叫双精度浮点型，平时能用到的就是这两了。

精度指的是有效数位，即左侧第一个非 0 开始数，数够指定的。不是小数位数

浮点型变量输入

```
float a = 2.3f;
```

```
double b = 4.5;
```

```
printf("a:%f,b:%lf\n", a, b);
```

```
scanf_s("%f%lf", &a, &b);
```

```
printf("a:%f,b:%lf\n",a,b);
```

小数赋值给整形变量

```
int a = 34.56; //直接舍弃小数部分, 没有四舍五入, a==34
int b = (int)56.67; //强制类型转换, 结果一样
```

字符类型: char

字符常量:

```
'a' 'W' ';' '' '4'
```

字符单引号内, 一个里只能装 1 个字符, 字符都在 ASCII 码表上。

ASCII 码表是国际应用最广泛的编码表, 即编程用的各种符号字母都是这个表上的, 他不是 C 语言专属, 其他语言也用。

类型名	字节数	格式说明符	范围
char	1	%c %hhd	-128, 127

char 的本质就是 1 字节整形, 做字符使用时, 其格式说明符是%c, 做 1 字节整形使用时, 其格式说明符是%hhd

ASCII 码编号是 0~127

类型名	字节数	格式说明符	范围
unsigned char	1	%c %hhu	0, 255

%c 输入字符, %hhu 输入数字

字符变量声明/定义

```
char c = 'A', d = 65;
printf("%c,%d\n", c, c);
printf("%c,%d\n", d, d);
```

字符就是整数, 以%c 输出就是字符形式, 以%d 输出就是 10 进制整数形式

```
putchar(c);
putchar(d);
```

专门以字符形式输出, 字符专用。printf 更灵活

字符输入输出

```
char c = 0;
scanf_s("%c", &c, 1); //_s 与老版不一样
printf("%c\n", c);

c = getchar();
putchar(c);
```


清空缓冲区

键盘上按的按键都以其字符形式存在输入缓冲区，也就是一块内存空间里，输入函数就是从缓冲区往外一个一个拿。对于字符输入来说，缓冲区里的每个都是字符，他都会读出来。对于数值类型，只认数字(正负号，小数点)，别的(空格 回车 字符等)都不认。

所以，缓冲区的残留都会对字符输入有影响，如下：

```
char c = 'A', d = 'B';
scanf_s("%c%c", &c, 1, &d, 1);
输入：C<空格>D<回车>    c == C    d == <空格>
```

```
char c = 'A';
int a = 12;
scanf_s("%d", &a);
c = getchar();
输入：12<回车>    a == 12    c == <回车>
```

即缓冲区的残留会被字符输入无差别读取，造成意想不到的错误输入。

解决办法：

在输入字符前加上：`rewind(stdin);`
清空缓冲区。

运算符：用来运算操作数据，前面学过：=, &, sizeof、函数调用()

算术运算符：加+ 减- 乘* 除/ 取余%

加法：+

常数+常数：3+4+5+7

变量+变量：a+b+d

变量+常量：a+4+5+b

```
int a = 3 + 4 + 5 + 7;
printf("%d", a);
printf("%d", a + 5 + 6);
```

减法：-

常数-常数：3-4-5-7

变量-变量：a-b-d

变量-常量：a-4-5-b

```
int a = 3 - 4 - 5 - 7;
printf("%d", a);
printf("%d", a - 5 - 6);
```

乘法：* 数学中乘号可以省略，编程中一定不能省略

常数-常数：3*4*5*7

变量-变量：a*b*d

变量-常量：a*4*5*b

```
int a = 3 * 4 * 5 * 7;
printf("%d", a);
printf("%d", a * 5 * 6);
```

除法：/ 整数取整，小数除法

5/2 为 2 余 1，结果是 2

7/2 为 3 与 1，结果是 3

5.0/2.0 为 2.5

7.0/2.0 为 3.5

5.0/2 先将 2 转成 2.0 然后 5.0/2.0 结果是 2.5

取余：% 只能用于整数

5/2 为 2 余 1，结果是 1

7/2 为 3 与 1，结果是 1

改变运算顺序：()

```
int a = 4+5*3-2; //先算乘法
int b = (4+5)*(3-2); //先算小括号的
```

正负号：正号可以省略

+5 -5 +4.5 -5.6

int a = -3; -a 结果是负负得正，为 3

优先级与结合性

$a = 4 + 5 * 3 - 2$ 在编程中叫表达式，由运算符与操作数构成，表达式必有一个结果

其中 $5 * 3$ $4 + 5 * 3$ $4 + 5 * 3 - 2$ 叫子表达式，是整个表达式的一部分

优先级与结合性可简单理解为先算谁后算谁，如此可以简单快捷得到答案，如上例子

但是编程中的运行过程不是先算谁后算谁，可理解为加小括号，然后从左到右计算，如下

$3 + 4 - 5 * 6$

*优先级高于+ -

$3 + 4 - (5 * 6)$

+ -结合性从左到右

$(3 + 4) - (5 * 6)$

$((3 + 4) - (5 * 6))$ 然后从左到右计算

先算

$3 + 4 == 12$ $(12 - (5 * 6))$

然后算

$5 * 6 == 30$

最后算

$12 - 30 == -18$

当然平时口算就直接先后就行了，不用这么考虑具体执行过程。

复合赋值运算符: $+=$ $-=$ $*=$ $/=$ $\%=$

`int a = 5;`

`a += 3;` 即 `a = a + 3;` a 变 8 左操作数必须是变量，因为要赋值

`a -= 3;` 即 `a = a - 3;` a 变 2 左操作数必须是变量，因为要赋值

`a *= 3;` 即 `a = a * 3;` a 变 15 左操作数必须是变量，因为要赋值

`a /= 3;` 即 `a = a / 3;` a 变 1 左操作数必须是变量，因为要赋值

`a %= 3;` 即 `a = a % 3;` a 变 2 左操作数必须是变量，因为要赋值

自加 1 运算符: $++$

$++$ 运算符分前置 $++$ 与后置 $++$ ，作用是使得变量自加 1，形式如下：

`int a = 2;`

`a++;` //后置 $++$ 3

`++a;` //前置 $++$ 4

只能作用于变量。

区别:

在参与运算时，后置 $++$ 用自加前的值参与运算，比如

`int a = 2, b = 0;`

`b = a++ + 4;` 2+4

`printf("%d %d", b, a);` 6 3

可理解为

`int a = 2, b = 0;`

`b = a + 4;`

`a += 1;` //在后

```
printf("%d %d", b, a); 6 3
```

在参与运算时，前置++用自加后的值参与运算，比如

```
int a = 2, b = 0;
b = ++a + 4;      3+4
printf("%d %d", b, a); 7 3
```

可理解为

```
int a = 2, b = 0;
a += 1;    //在前
b = a + 4;
printf("%d %d", b, a); 6 3
```

自减 1 运算符：-- 跟++一样的，只不过是减 1

--运算符前置--与后置--，作用是使得变量自减 1，形式如下：

```
int a = 2;
a--; //后置++ 1
--a; //前置++ 0
只能作用于变量。
```

区别：

在参与运算时，后置--用自减前的值参与运算，比如

```
int a = 2, b = 0;
b = a-- + 4;      2+4
printf("%d %d", b, a); 6 1
```

可理解为

```
int a = 2, b = 0;
b = a + 4;
a -= 1;    //在后
printf("%d %d", b, a); 6 1
```

在参与运算时，前置--用自减后的值参与运算，比如

```
int a = 2, b = 0;
b = --a + 4;      1+4
printf("%d %d", b, a); 5 1
```

可理解为

```
int a = 2, b = 0;
a -= 1;    //在前
b = a + 4;
printf("%d %d", b, a); 5 1
```

注意：

同一条语句中，出现了1个变量a\a++\++a\--a\a--，就不能再出现a++ a-- --a ++a等表达式，不同的编译器运行结果不一样。只能出现单一。

关系运算符

关系运算符用来表示两个数据的关系

包括：>、<、>=、<=、==、!=

关系表达式的结果为 1 或者 0，即真或者假，如下：

10 > 20	关系错误，结果是 0，即假
10 < 20	关系正确，结果是 1，即真
10 >= 20	关系错误，结果是 0，即假
10 >= 10	关系正确，结果是 1，即真
20 >= 10	关系正确，结果是 1，即真
10 == 10	关系正确，结果是 1，即真
20 == 10	关系错误，结果是 0，即假
20 != 10	关系正确，结果是 1，即真
10 != 10	关系错误，结果是 0，即假

用法如下：

```
int a = 10 < 20;           //10 < 20 为 1，将 1 赋值给 a
int c = (10 > 20) - a;      //10 > 20 为 0，0+a 赋值给 b
printf("%d", 10 != 20);    //直接输出 1
```

它仅仅是个表达式，跟 1+1 是一样的。

逻辑运算符

C 语言规定，非 0 即真，0 即假

真：50.3, a+b-4, -3, 'w' 等等，均为真

假：0

4 > 3 结果是真，用 1 作为真的结果，因为真太多了，所以作为结果就规定用 1 代表

逻辑运算符：与(&&)、或(||)、非(!)

运算规则：

真&&真为 1，真&&假为 0，假&&真为 0，假&&假为 0

真 || 真为 1，真 || 假为 1，假 || 真为 1，假 || 假为 0

! 真 为 0，! 假 为真

其中真假为任意合法表达式

比如：

```
int a = 5-9*2 && 5-5;      //-13 && 0 为 0,0 赋值给 a
int b = !0;                //b 的值是 1
int c = 20 || 0;           //c 的值是 1
int d = !(5-5) + 3 && 0     //1+3 && 0    4&&0 为 0
```

判断范围的写法：

目标逻辑：如果 a 在该范围内，表达式结果是真即 1，不在这个范围，表达式结果是假即 0。

写法 1：20 < a < 40

执行过程为：

a=30 时，20 < a 为真，结果是 1，然后 1<40 为真，结果是 1，所以最终结果是 1

a=50 时，20 < a 为真，结果是 1，然后 1<40 为真，结果是 1，所以最终结果是 1，不合逻辑

a=10 时，20 < a 为假，结果是 0，然后 0<40 为真，结果是 1，所以最终结果是 1，不合逻辑

写法 2: $20 < a \ \&\& \ a < 40$

执行过程为:

a=30 时, $20 < a$ 为真, 结果是 1, 然后 $30 < 40$ 为真, 结果是 1, 所以最终结果是 1

a=50 时, $20 < a$ 为真, 结果是 1, 然后 $50 < 40$ 为假, 结果是 0, 所以最终结果是 0

a=10 时, $20 < a$ 为假, 结果是 0, $0 \ \&\& \$ 任何都是假, 结果是 0, 所以最终结果是 0

逗号运算符: 最后一个

形式: 用逗号分隔多个表达式, 如下

```
int a = 1, b=2, c=3;
```

```
a+=1, 2+c, b+=1, a+b+c+2    //逗号表达式
```

运算规则:

1、从左到右依次执行各表达式

2、最右侧的子表达式是逗号表达式的最终结果

第一: $a+=1$ a 变为 2

第二: $2+c$ 为 4

第三: $b+=1$ b 变为 3

第四: $a+b+c+2$ 为 10, 整个逗号表达式的结果是 10

输出:

由于逗号优先级最低, 所以参与运算一定要加上小括号包起来, 如下

```
int a = 1, b = 2, c = 3, d;
```

```
d = (a += 1, 2 + c, b += 1, a + b + c + 2);    //存入变量, 必须加上小括号
```

```
printf("%d", (a += 1, 2 + c, b += 1, a + b + c + 2)); //直接输出
```

流程结构：代码执行的流程。

顺序结构：按照既定的顺序执行，比如代码一行一行从上到下执行，表达式按优先级结合性执行

选择分支：

如果 条件真
 执行代码块

循环结构：

循环 条件真
 重复执行代码块

选择分支：

if 结构

```
if (条件)
{
    code;    //条件为真，则执行 code 代码块，如果条件为假，则不执行
}
```

举例：

```
int score = 70;
if (score >= 60)
{
    printf("及格\n");
}
if (score < 60)
{
    printf("不及格\n");
}
```

注意点：

- 1、条件是合法的表达式即可，结果是真就执行，结果是假就不执行
- 2、当结构只有一条语句时，可以不加大括号。即 if 结构将其下第一条语句作为自己的代码块。

```
int score = 70;
if (score >= 60)
    printf("及格\n");
if (score < 60)
    printf("不及格\n");
```

- 3、缩进对代码执行没有任何影响，它仅仅是让程序员看着舒服，清晰

```
int score = 70;
if (score >= 60)    printf("及格\n");
if (score < 60)    printf("不及格\n");
```

- 4、if 后别加分号，一个分号就是 1 个空语句，相当于 3

二选一结构: if else

二选一就是有两个分支，必执行其一，如下

```
if (条件)
{
    code1;    //条件为真，执行次代码分支
}
else
{
    code2;    //条件为假，执行此分支
}
```

举例:

```
int age = 20;
if (age >= 18)
{
    printf("成年人\n");
}
else
{
    printf("未成年人\n");
}
```

注意点:

- 1、if else 关键字后不要加分号
- 2、else 没有条件
- 3、if else 结构紧挨着，不要在中间加别的代码
- 4、else 分支不能独立存在
- 5、当分支只有一条语句时，可以不加大括号，平时都加上

条件运算符: ? :

条件运算符跟 if-else 结构执行逻辑一样，如下:

条件 ? 表达式 1 : 表达式 2;

条件为真时，执行表达式 1，整个条件表达式的结果就是表达式 1 的值

条件为假时，执行表达式 2，整个条件表达式的结果就是表达式 2 的值

举例:

```
age >= 18 ? printf("成年人\n") : printf("未成年人\n");
```

同 if-else 结构区别

- 1、if-else 是结构，专业名词是复合语句，其分支内可以写任何合法的语句。

条件运算符是运算符，其三个部分必须都是表达式，不能是语句。有返回值的函数也是表达式。printf 函数本身也是表达式，讲函数时候专门介绍返回值

- 2、条件表达式是有结果的，可以赋值，可以输出

```
int a = 3==3 ? 5 : 6;
a 的值装 5
```


多选一结构: if else if else if else

举例如下: 给成绩打评级

```
int score = 70;
if (score >= 0 && score < 60)
{
    printf("不及格\n");
}
else if (score >= 60 && score < 70)
{
    printf("及格\n");
}
else if (score >= 70 && score < 90)
{
    printf("良好\n");
}
else if (score >= 90 && score <= 100)
{
    printf("优秀\n");
}
else
{
    printf("成绩错误\n");
}
```

特点:

- 1、别加分号
- 2、多选一, 从第一个条件判断, 执行第一个条件为真的分支, 执行完直接结束结构, 即使下面的条件为真, 也不看了。
- 3、else 分支一般用来处理默认情况, 即上面条件没有涉及的部分
- 4、else 分支可以不写, 但是建议写上。保证结构的逻辑完整性
- 5、对比多个 if 并列, 此属于多选多

多选一: switch 结构

结构如下:

```
switch (匹配标签)
{
    case 标签 1:
        code1;
        break;
    case 标签 2:
        code2;
        break;
    case 标签 3:
        code3;
        break;
    default:
}
```

执行过程:

- 1、用匹配标签与标签进行依次比较, 相等的, 就执行相应分支
- 2、匹配标签是整型表达式
- 3、标签是整型常量表达式
- 4、break 的作用是跳出结构

举例如下:

```
int a = 3;
switch (a)
{
case 1:
    printf("case 1\n");
    break;
case 2:
    printf("case 2\n");
    break;
case 3:
    printf("case 3\n");
    break;
default:
    printf("default\n");
}
```

特点:

- 1、标签就是分支编号，不能重复
- 2、编号不一定连续，是个整数即可
- 3、default 相当于 else，可以不用写
- 4、break 的作用是跳出结构，最后一个分支不用写 break
- 5、标签只能是整数，小数直接报错
- 6、标签内定义变量需要加大括号

不写 break

break 的作用是中断跳出 switch 结构，没有 break，对应的分支执行完不结束 switch，继续执行下一个分支，直到遇见 break 或者 switch 结构结束。

```
int a = 2;
switch (a)
{
case 1:
    printf("case 1\n");
    break;
case 2:
    printf("case 2\n");    //无 break, 执行完 2 继续执行 3
case 3:
    printf("case 3\n");
    break;                //此处结束 switch 结构
default:
    printf("default\n");
}
```

switch 结构实现范围的思路，依赖上面的思路，判断变量 a 是否在 2~8 之间，a 必须是整型

```
int a = 2;
switch (a)
```

```
{  
  
case 2:  
case 3:  
case 4:  
case 5:  
case 6:  
case 7:  
case 8:  
    printf("在范围\n");  
    break;  
default:  
    printf("不在范围\n");  
}
```

对比 if 结构

- 1、switch 实现范围过于繁琐，如果范围加大，那么就要写更多 case，而 if 可以写范围判断
- 2、switch 不能涉及小数，if 可以

循环就是重复的执行一段代码

循环结构有三：while 循环，for 循环，do while 循环

while 循环：

形式：

```
while(条件)
{
    code;    //条件为真就重复执行
}
```

举例：输出 1,2,3,4,5

```
int i = 1;
while (i <= 5)
{
    printf("%d ", i);
    i++;
}
```

执行过程：

- 1、i==1,i<=5 为真，执行循环，输出 i 即 1，然后 i++,i 变为 2
- 2、i==2,i<=5 为真，执行循环，输出 i 即 2，然后 i++,i 变为 3
- 3、i==3,i<=5 为真，执行循环，输出 i 即 3，然后 i++,i 变为 4
- 4、i==4,i<=5 为真，执行循环，输出 i 即 4，然后 i++,i 变为 5
- 5、i==5,i<=5 为真，执行循环，输出 i 即 5，然后 i++,i 变为 6
- 6、i==6,i<=5 为假，结束循环

注意点：

- 1、不要加分号，分号是空语句，会自动作为 while 的代码块。
- 2、也可以写成 printf("%d ", i++);
- 3、i 的自变是任意的，位置也是根据需要在指定位置
- 4、只有一条语句，可以不加大括号

可控循环三要素，可循环指定次数

要素 1、循环控制变量有初始值，即 i=1，初始值不一定是 1，根据需要来

要素 2、循环控制变量参与条件，即 i<=5

要素 3、循环控制变量规律变化，比如 i++,i+=3,i--等等，按照规律变化，乱变不行

死循环：条件恒为真

for 循环：

循环形式：

```
for (1; 2; 3)
{
    code;
}
```

小括号内被两个分号分成三部分，分别是要素 1，要素 2，要素 3
即 for 循环就是把循环三要素集中在头部了

举例：输出 1,2,3,4,5

```
int i;
for (i = 1; i <= 5; i++)
{
    printf("%d ", i);
}
```

执行过程：

- 1、i=1, i==1, 部分 1 只执行一次
- 2、i==1, i<=5 为真, 执行循环, 输出 i 即 1
- 3、i++, i 变为 2, i<=5 为真, 执行循环, 输出 i 即 2
- 4、i++, i 变为 3, i<=5 为真, 执行循环, 输出 i 即 3
- 5、i++, i 变为 4, i<=5 为真, 执行循环, 输出 i 即 4
- 6、i++, i 变为 5, i<=5 为真, 执行循环, 输出 i 即 5
- 7、i++, i 变为 6, i<=5 为假, 结束循环

for 循环灵活性： for 循环三个部分都可以灵活多变

部分 1：可以多个变量，可以什么都不写，可以定义变量，最通用写法

```
int i;
for (i = 1, printf("hello"); i <= 5; i++) //用逗号分开
{
    printf("%d ", i);
}
```

部分 2：可以不写，为恒为真。最好只写条件

部分 3：可以不写，把条件变化写在 for 循环内部，也可写多个，用逗号隔开

do while 循环：形式如下

```
int i = 1;
do
{
    printf("%d ", i);
    i++;
}while(i <= 5); //有分号
```

执行过程：先执行，后判断条件

- 1、i==1, 输出 i 即 1, 然后 i++, i 变为 2, 条件 i<=5 为真, 继续回到 do
- 2、i==2, 输出 i 即 2, 然后 i++, i 变为 3, 条件 i<=5 为真, 继续回到 do
- 3、i==3, 输出 i 即 3, 然后 i++, i 变为 4, 条件 i<=5 为真, 继续回到 do
- 4、i==4, 输出 i 即 4, 然后 i++, i 变为 5, 条件 i<=5 为真, 继续回到 do
- 5、i==5, 输出 i 即 5, 然后 i++, i 变为 6, 条件 i<=5 为假, 结束循环

对比：

while for 叫入口条件循环，二者用谁都行，习惯哪个用哪个，没有好坏。

do while 叫出口条件循环，该循环至少执行一次

初始循环条件为真时，三者用谁都行。初始循环条件为假时，for,while 一次都不执行，do while 执行 1 次。

break: 中断循环并直接跳出

前面学过跳出 switch, 此时跳出循环

举例如下:

```
int i = 1;
while (i <= 5)
{
    if (3 == i) //i==3 时中断跳出循环
        break;
    printf("%d ", i);
    i++;
}
```

执行过程:

- 1、i==1,i<=5 为真, 执行循环, 3 == i 为假, 输出 i 即 1, 然后 i++,i 变为 2
- 2、i==2,i<=5 为真, 执行循环, 3 == i 为假, 输出 i 即 2, 然后 i++,i 变为 3
- 3、i==3,i<=5 为真, 执行循环, 3 == i 为真, break 跳出循环

循环 switch 结构嵌套怎么跳: break 跳所在的结构。

continue: 结束本次循环, 跳到循环头, 进行下一轮循环

如下:

```
int i;
for (i = 1; i <= 5; i++)
{
    if (i == 2 || i == 5)
        continue;
    printf("%d ", i);
}
```

执行过程:

0、i=1, i==1, 部分 1 只执行一次

- 1、i==1, i<=5 为真, 执行循环, if 条件为假, 继续执行, 输出 i 即 1
- 2、i++, i 变为 2, i<=5 为真, 执行循环, if 条件为真, 跳到循环头
- 3、i++, i 变为 3, i<=5 为真, 执行循环, if 条件为假, 继续执行, 输出 i 即 3
- 4、i++, i 变为 4, i<=5 为真, 执行循环, if 条件为假, 继续执行, 输出 i 即 4
- 5、i++, i 变为 5, i<=5 为真, 执行循环, if 条件为真, 跳到循环头
- 6、i++, i 变为 6, i<=5 为假, 结束循环

循环嵌套：嵌套就是一层循环套一层循环

```
int i, j;
for (i = 1; i <= 3; i++)
{
    for (j = 1; j <= 2; j++)    //j = 1 的作用
    {
        printf("i=%d, j=%d\n", i, j);
    }
}
```

执行过程：

- 1、i=1, i<=3 为真，进入循环
 j=1, j<=2 为真，进入循环，输出 i=1j=1
 j++, j 为 2, j<=2 为真，进入循环，输出 i=1j=2
 j++, j 为 3, j<=2 为假，结束内层循环
- 2、i++, i 为 2, i<=3 为真，进入循环
 j=1, j<=2 为真，进入循环，输出 i=2j=1
 j++, j 为 2, j<=2 为真，进入循环，输出 i=2j=2
 j++, j 为 3, j<=2 为假，结束内层循环
- 3、i++, i 为 3, i<=3 为真，进入循环
 j=1, j<=2 为真，进入循环，输出 i=3j=1
 j++, j 为 2, j<=2 为真，进入循环，输出 i=3j=2
 j++, j 为 3, j<=2 为假，结束内层循环
- 4、i++, i 为 4, i<=3 为假，结束循环

执行过程：如果没有 j=1 int i, j=1;

- 1、i=1, i<=3 为真，进入循环
 j=1, j<=2 为真，进入循环，输出 i=1j=1
 j++, j 为 2, j<=2 为真，进入循环，输出 i=1j=2
 j++, j 为 3, j<=2 为假，结束内层循环
- 2、i++, i 为 2, i<=3 为真，进入循环
 j 为 3, j<=2 为假，结束内层循环
- 3、i++, i 为 3, i<=3 为真，进入循环
 j 为 3, j<=2 为假，结束内层循环
- 4、i++, i 为 4, i<=3 为假，结束循环

嵌套跳出：接力跳

```
int i, j, flag = 0;
for (i = 1; i <= 5; i++)
{
    for (j = 1; j <= 5; j++)
    {
        if (i + j == 5)
        {
            flag = 1;
            break;
        }
        printf("i=%d, j=%d\n", i, j);
    }
    if (1 == flag)
        break;
}
```

```
int i, j;
for (i = 1; i <= 3; i++)
{
    for (j = 1; j <= 2; j++)
    {
        printf("i=%d, j=%d\n", i, j);
        if (j == 2)
        {
            goto loop; //直接跳loop
        }
    }
}
loop:
```

循环 switch 结构嵌套怎么跳：break 跳所在的结构。

int a = 12; a 是一个变量，一次只能装 1 个数，有 100 个数想要存储，怎么办？可以定义 100 个变量装，但是太麻烦了。所以有了数组类型。

数组：用来装一组数的类型。声明形式如下：

```
int a[10]; //声明数组 a
```

int 表示该数组用来装 int 类型的元素，其他类型就写其他的，比如 double b[10];

a 是数组名字，是合法的 C 语言标识符即可

[] 是指示的作用，表示变量 a 是数组变量，没有 [] 即 int a，就是普通的整形变量

10 表示该数组最多装 10 个 int 类型数据，个数自定。

定义数组类型变量：即初始化，断点看数据

```
int a[10] = {6,4,7,3,8,3,2,8,1,0}; //初始化形式，最多初始化 10 个元素
```

```
int a[10] = {6,4,7,3,8}; //初始化部分元素，其他元素默认初始化成 0
```

```
int a[10] = {0}; //10 个元素全部初始化 0
```

```
int a[] = {6,4,7,3}; //初始化时可不写元素个数，系统根据元素多少自定
```

注意点：

1、数组大小要确定，个数要用常量，部分编译器支持变量，但是不通用。形式如下：

```
int n = 10;
```

```
int a[n] = {6,4,7,3,8,3,2,8,1,0};
```

2、元素个数不能是 0，不能是负数，不能是小数

3、默认最大申请不能超过 1m，多了会爆栈

4、所有元素都是相邻的，是一块连续的内存空间

下标：即各元素的编号，编号从 0 开始

```
int a[5] = {4,2,7,8,4};
```

5 个元素的标号依次是：0,1,2,3,4 最大下标是元素个数-1，即 5-1

元素访问：数组名[下标]

5 个元素依次是：a[0],a[1],a[2],a[3],a[4]

相当于 5 个变量，即变量名字，跟 int c = 9; 的 c 的用法一模一样

元素赋值：

```
a[0]=6, a[1]=5, a[2]=3, a[3]=9, a[4]=0;
```

输出：

```
int a[5] = { 4, 2, 7, 8, 4 };
```

```
printf("%d %d %d %d %d", a[0], a[1], a[2], a[3], a[4]);
```

取地址：

```
scanf("%d%d%d%d%d", &a[0], &a[1], &a[2], &a[3], &a[4]);
```

参与计算：

```
int b = a[3] * 2 + 1;
```

[]的区别:

int a[5]; 定义变量时, 是个指示符, 表示 a 是数组变量, 需是常量

a[3] = 3; 元素访问时, 是下标运算, 访问指定的元素, 可为变量

比如: int n=3; a[n]就是 a[3];

循环遍历数组

```
int a[5] = { 4, 2, 7, 8, 4 };
```

```
int i;
```

```
for (i = 0; i < 5; i++)
```

```
    printf("%d ", a[i]);
```

数组的大小: sizeof

数组类型: int [5] 去掉变量名, 就是变量的类型

这就是 5 个 int 类型元素的数组类型

double [20] float [4] 等等, 都是不同类型的数组

数组的大小就是所有元素大小之和。

口算: int a[5]; 就是 sizeof(int)*5 为 20 字节

计算: printf("%u %u", sizeof(a), sizeof(int[5]));

二维数组

上面学习的叫一维数组, 此时就是二维数组, 对比定义如下:

```
int a[5]; //一维数组
```

```
int c[3][4]; //二维数组
```

一维数组: 元素是数据类型的数组

二维数组: 元素是一维数组的数组, 本质还是一维数组

c 是 3 个元素的一维数组, 每个元素是 4 元素的一维数组。



二维数组一般理解为行列, 对初学者比较友好, int a[行][列], 即 3 行 4 列, 可抽象如下



二维数组初始化:

```
int a[3][2] = {{3,2},{6,5},{8,7}}; //内部大括号对应每个小一维数组
```

3 2 6 5 8 7

```
int a[3][2] = {{3},{9},{8,7}}; //初始化部分元素, 其他默认 0
```

3 0 9 0 8 7

```
int a[3][2] = {3,9,8}; //没有内部大括号, 就依次初始化各元素, 其他为 0
```

3 9 8 0 0 0

```
int a[ ][2] = {3,9,8}; //初始化时, 可不写行, 系统根据数据个数计算行, 2 行
```

3 9 8 0

下标: 行下标与列下标, 都是从 0 开始

```
int a[3][4] = {{1,2,3,4},{5,6,7,8},{9,10,11,12}};
```

行下标: 0 1 2

列下标: 0 1 2 3

元素:

a[0][0] a[0][1] a[0][2] a[0][3]

a[1][0] a[1][1] a[1][2] a[1][3]

a[2][0] a[2][1] a[2][2] a[2][3]

另一个角度

a[0]是第 1 个小数组的数组名字

a[1]是第 2 个小数组的数组名字

a[2]是第 3 个小数组的数组名字

第一个小数组的第一个元素就是 a[0][0]

	0	1	2	3
a[0]	1	2	3	4
a[1]	5	6	7	8
a[2]	9	10	11	12

二维数组元素遍历:

```
int i, j, a[3][4] = { {1,2,3,4}, {5,6,7,8}, {9,10,11,12} };
```

```
for (i = 0; i < 3; i++)
```

```
{
```

```
    for (j = 0; j < 4; j++)
```

```
    {
```

```
        printf("%d ", a[i][j]);
```

```
    }
```

```
    putchar('\n');
```

```
}
```

赋值：

```
a[0][0]=11  a[0][1]=12  a[0][2]=13  a[0][3]=14  
a[1][0]=15  a[1][1]=16  a[1][2]=17  a[1][3]=18  
a[2][0]=10  a[2][1]=11  a[2][2]=12  a[2][3]=13
```

取地址： `scanf_s`

各元素名字相当于 1 个普通变量名字，`&a[1][2]`就是该元素的地址

```
scanf_s("%d", &a[2][2]);
```

数组的大小： `sizeof`

```
int a[3][4] ;
```

数组类型：`int [3][4]` 去掉变量名，就是变量的类型

这就是3行4列12 个 `int` 类型元素的二维数组类型

数组的大小就是所有元素大小之和。

口算：就是 `sizeof(int)*3*4` 为 48 字节

```
计算：printf("%zd %zd", sizeof(a), sizeof(int[3][4]));
```

其他类型的数组同理

此外还有三维数组，四维数组等等多维数组，

```
int a[2][3][4];    三维数组
```

```
int c[2][3][4][5];  四维数组
```

原理跟二维数组一样，平时基本用不到。

整型数据用整型变量来装，比如 1,2,3，用 int a;来装

浮点型数据用浮点型变量来装，比如 1.2,2.3,3.4，用 double a;来装

字符型数据用char型变量来装，比如'a',';','W'，用 char a;来装

每个变量都有自己的地址，地址也是数据，就用地址类型的变量来装，地址类型就叫指针类型，即指针类型定义的变量就是专门装地址的。

地址值是专门的地址类型，跟整型是完全不一样的，虽然宏观上看到的是一个整数。就像 1 辆自行车与 1 斤大米饭，两个都是 1，但是这个 1 的类型不一样，即指代的物体不一样，所以 1 的操作完全不一样。

基本数据类型指针：

基本数据类型变量的地址，用对应的基本数据类型指针变量来装

声明指针变量：

```
short *ps;
```

```
char *pc;
```

注意点：

- 1、short* char*叫指针类型，ps pc 叫指针变量，很多资料简称指针
- 2、* 表示 ps pc 是指针变量，类型 int a[10]的[]，在定义变量时是指示作用
- 3、空格放在哪儿都行，short *ps, short * ps, short* ps 等等，没有区别。看心情。
- 4、没有初始化的情况下，数据就是未知的，同 int a;意义一样。

定义(初始化)指针变量：

```
int a = 12;
```

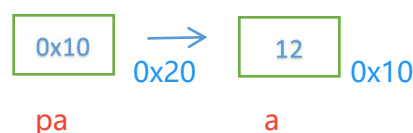
```
int* pa = &a;
```

```
float b = 2.3f;
```

```
float* pb = &b;
```

注意点：

- 1、各种类型变量的地址的类型不同，a b 变量的地址类型就是变量对应的类型，所以装地址的指针变量的类型，是相应的类型。
- 2、指针的赋值操作又叫指向，即 pa 指向变量 a 的地址，名字而已
- 3、要指向合法的地址空间，非法的空间会异常中断。
- 4、内存模型



NULL:

```
double* pd = NULL; //NULL 就是 0，用来初始化指针，
```

```
int a = 0; //意义一样，
```

赋值：重新指向

```
double d = 4.5;
```

```
pd = &d;
```

注意点：

- 1、指针变量也是变量，可以装别的地址，但是要是同类型的
- 2、重新赋值也叫重新指向

对于一块空间 `int a=12`; `a` 是这个块空间的名字，同名名字可以操作这块空间。`&a` 是空间的地址，通过地址也能操作该空间。

如何通过地址操作空间？使用内存操作运算符：*

计算规则：*+空间的地址 就是 该空间名字

*+变量的地址 就是 该变量本身

即：`*&a == a` 这就是*的计算规则，大家一定要记住，很多人不会用指针，就是这个计算规则完全记忆模糊。

既然`*&a == a`，所以`*&a`与`a`的用法完全一模一样。

读：`printf *&a`，输出 12

写：`*&a = 34`，原 12 就被覆盖了，`a` 也是 34 了

取地址：`&*&a == &a` 得到该空间的地址

我们可以通过指针变量记录地址值，如下：

```
int a = 12;
```

```
int* p = &a; //p == &a
```

所以上面的`&a` 直接换成 `p` 就行了。`*&a == a` 即 `*p == a`

所以`*p`与`a`的用法完全一模一样。

读：`printf *p`，输出 12

写：`*p = 34`，原 12 就被覆盖了，`a` 也是 34 了

取地址：`&*p == &a` 得到该空间的地址

不同指针指向会咋样？

指针的类型决定了指针的操作规则，`short*`指针一次操作 2 字节空间，`char*`指针一次操作 1 字节空间，`int*`指针一次操作 4 字节空间，`float*`指针一次操作 4 字节空间，`double*`指针一次操作 8 字节空间.....并且浮点型存储跟整型存储是不一样的，`int*`与`float*`也是完全不一样，就像同一个集装箱，装自行车与装大米，本质就不一样。

比如：

```
int a = 12;
```

```
char *pc = &a;      *p=2, 只操作其 1 字节首字节
```

```
long long *pll = &a; *pll=34, 操作 8 字节, 4 字节越界
```

```
float *pf = &a;      读出的*pf 跟 12 完全看不出没关系
```

二级指针:

```
int a = 12;    //a 变量有自己的空间, 假设地址是 0x10, 里边装 12
int *p = &a;   //p 变量有自己的空间, 假设地址是 0x20, 里边装 0x10
int **pp = &p; //pp 也有自己的空间, 假设地址是 0x30, 里边装 0x20
```

注意点:

1、pp 就是二级指针变量, 用来装 1 级指针变量的地址

int**就是二级指针类型, int*是一级指针类型

同理: int***是三级指针, 装二级指针的地址, &pp

int****是四级指针, 装三级指针的地址

平时顶多用到 2 级, 本质都是一样的

二级指针使用:

计算规则: *+空间的地址 就是 该空间本身

*+变量的地址 就是 该变量本身

所以:

```
p == &a
*p == *&a == a
pp == &p
*pp == *&p == p
**pp == *p == a
```

所以: **pp 跟 a 是一模一样的。

读: printf **pp, 输出 12

写: **pp = 34, 原 12 就被覆盖了, a 也是 34 了

取地址: &**pp == &a 得到该空间的地址

指针数组

int a[10]; //每个元素都是 int 类型数据

int* c[10]; //每个元素都是 int*类型的, 即地址

这就是指针数组, 或者叫地址数组

int d=1, e=2, f=3, g=4;

int* c[5] = {&d,&e,&f,&g};

使用:

c[1]就是&e

*c[1] == *&e == e

所以*c[1]与 e 的用法完全一模一样。

读: printf *c[1], 输出 2

写: *c[1] = 34, 原 2 就被覆盖了, e 也是 34 了

取地址: &*c[1] == &e 得到该空间的地址

数组与指针：

前面说指针的类型决定了指针的操作规则。`int*p1` 指针*p1 一次操作 4 字节空间，`float*p2` 指针*p2 一次操作 4 字节空间，`double*p3` 指针*p3 一次操作 8 字节空间。

指针可以进行加减运算，且只能进行加减法，名字叫指针偏移。加减 n，实为加减 n 个类型的大小。如下：

<code>p1+1</code>	1 为 <code>sizeof(int)</code>	4
<code>p2-2</code>	2 为 <code>sizeof(float)*2</code>	8
<code>p3+3</code>	3 为 <code>sizeof(double)*3</code>	24

一维数组与指针

一维数组 `int a[5] = {8,7,6,5,4};` 每个元素类型都是一样的 `int`，并且空间是连续的

所以：`&a[0] + 1 == &a[1]`

`&a[0] + 2 == &a[2]`

`&a[4] - 1 == &a[3]`

每个元素都是 `int` 类型，可以用指针装地址

`int *p = &a[0];`

那么

所以

<code>p+0 == &a[0]</code>	<code>a[0] == *&a[0] == *(p+0) == *p</code>
<code>p+1 == &a[1]</code>	<code>a[1] == *&a[1] == *(p+1)</code>
<code>p+2 == &a[2]</code>	<code>a[2] == *&a[2] == *(p+2)</code>
<code>p+3 == &a[3]</code>	<code>a[3] == *&a[3] == *(p+3)</code>
<code>p+4 == &a[4]</code>	<code>a[4] == *&a[4] == *(p+4)</code>

注意：`*(p+1)`一定要加小括号，因为加法的优先级低于*，

`*(p+1)`先执行 `p+1` 为 `&a[1]`，再取*，为 `a[1]`，为 7

`*p+1` 先执行 `*p`，为 8，再+1 为 9

数组属性：数组名是首元素的首地址：`&a[0] == a`

`int *p = &a[0];`

`int a[5] = {8,7,6,5,4};`

`int *p = a;`

二者一模一样，平时都是用第二种，因为写起来方便

下标运算符：`[]`

`*(p+4) == p[4] == 4[p]`

一维数组指针

数组是一个变量，它里边有很多的小元素，对元素取地址，得到的是元素地址，对数组名取地址，得到的就是数组的地址，又称数组指针


```
int a[5] = {7,6,5,4,3};
```

```
int (*p)[5] = &a; //p 就是一维数组指针变量
```

类型思路: int [5]是数组类型, 指针指向它, int[5] *p, 但是 C 语言不让这么写, 必须写中间 int (*p)[5], 必须加上小括号, 如果不加 int *p[5]这是指针数组, 因为[]的优先级高于*, 先跟*结合, p 就是指针, 先跟[]结合, p 就是数组

使用:

```
p == &a;
```

所以:

```
*p == *&a == a
```

*p 就跟 a 一样的使用了。

&a 与 &a[0] 数值一样类型不一样 + 1
&a是数组的地址, +1加整个数组的大小,
&a[0]是元素地址, +1加一个元素大小

读:

写

取地址

a[0] == (*p)[0]	(*p)[0] = 2;	&a[0] == &(*p)[0]
a[1] == (*p)[1]	(*p)[1] = 5;	&a[1] == &(*p)[1]
a[2] == (*p)[2]	(*p)[2] = 3;	&a[2] == &(*p)[2]
a[3] == (*p)[3]	(*p)[3] = 7;	&a[3] == &(*p)[3]
a[4] == (*p)[4]	(*p)[4] = 8;	&a[4] == &(*p)[4]

二维数组与指针

```
int a[2][3] = {{3,6,2},{1,9,7}};
```

二维数组类型指针:

```
int (*p)[2][3] = &a; //&a 是整个二维数组的地址
```

int (*)[2][3]为二维数组类型指针

p == &a 所以 *p == a, (*p)的用法跟 a 一样, a[1][1] == (*p)[1][1]

a[0] a[1]分别是两个小数组的名字, 所以&a[0] &a[1]是一维数组类型 int(*)[3]

所以:

```
int(*p1)[3] = &a[0];
```

所以: *p1 = a[0] 6 -> a[0][1] == (*p1)[1]

```
int(*p2)[3] = &a[1];
```

所以: *p2 = a[1] 9 -> a[1][1] == (*p2)[1]

数组名是首元素首地址

所以

```
a == &a[0];
```

即:

```
int(*p)[3] = a; //最常用的
```

p==a, 所以 p 跟 a 的用法一模一样

```
a[0][0] == p[0][0]   a[0][1] == p[0][1]   a[0][2] == p[0][2]
a[1][0] == p[1][0]   a[1][1] == p[1][1]   a[1][2] == p[1][2]
```

元素地址类型是 `int*`，并且每个元素类型一样，连续
所以

```
int *p = &a[0][0];
```

因为 `a[0] == &a[0][0]`，所以 `int *p = a[0];`

```
a[0][0] == *(p+0) == p[0]
```

```
a[0][1] == *(p+1) == p[1]
```

```
a[0][2] == *(p+2) == p[2]
```

```
a[1][0] == *(p+3) == p[3]
```

```
a[1][1] == *(p+4) == p[4]
```

```
a[1][2] == *(p+5) == p[5]
```

指针的大小：一切指针均如此

32 位编译环境下是 4 字节

64 位编译环境下是 8 字节 演示

`&a` 与 `&a[0]` 数值一样类型不一样 + 1
`&a`是数组的地址，+1加整个数组的大小，
`&a[0]`是元素地址，+1加一个元素大小

通用类型指针: void*

通用类型就是指没有具体类型, 什么指针都能装:

```
int a;      void *p = &a;
double b;   void* p = &b;
float c[5]; void* p = &c;
```

.....

但是, 可装不可用, 只能作为中转站, 因为指针的使用是由类型决定的, void*无类型, 所以不能使用, 比如:

```
*p = 34; //错误
p + 2;   //错误
```

需要将其转换成指定类型, 才能使用, 比如:

```
*(int*)p = 34; //将地址转成 int*, 相当于&a了, 然后*, 就是 a
```

前面测试, 普通的数组变量默认最大不能申请 1M 的空间, 甚至远远小于 1M 的空间, 毕竟程序中其他的变量也要使用空间。总共就 1M, 得省着用。

想要使用更大的, 没有限制的空间, C 语言给我们提供了方式: 如下

malloc //申请空间 理论上物理内存多大, 他就能申请多大, 当然并不能, 毕竟系统运行其他软件都要使用空间。

兄弟函数: calloc realloc

free //释放空间, 申请的空间必须我们自己手动释放

使用:

头文件: malloc.h 一定要加上, 老版本可以不加, 新版本更加规范了

malloc 函数原型:

```
void* malloc(size_t _Size);
```

malloc 作用: 申请一段空间, 并返回该空间的首地址

void*: 首地址返回 void*, 我们可以将其转换成任意类型去使用

参数: 要申请的字节数

```
int* p = (int*)malloc(sizeof(int) * 10); //也可以直接写 40
```

做 10 元素的 int 类型数组使用

p[0], p[1], p[2], p[3], p[4]... p[9] 共 10 个元素

用法直接类比之前学的:

```
int a[10];
```

```
int* p = a; //对于 p 而言, 二者没有区别, 都是 40 字节的空间
```

```
int(*p1)[4] = (int(*)[4])malloc(sizeof(int) * 3 * 4); //也可以直接写 48
```

做 12 元素的 3 行 4 列的 int 类型数组使用

p[0][0] p[2][3] 12 个元素

用法直接类比之前学的:

```
int a[3][4];
```

```
int(*p)[4] = a; //对于 p 而言, 二者没有区别, 都是 48 字节的空间
```

```
int(*p1)[3][4] = (int(*)[3][4])malloc(sizeof(int) * 3 * 4); //也可以直接写 40
    做 12 元素的 3 行 4 列的 int 类型数组使用
(*p)[0][0] ..... (*p)[2][3]    12 个元素
用法直接类比之前学的:
int a[3][4];
int(*p)[3][4] = &a;    //对于 p 而言，二者没有区别，都是 48 字节的空间
```

free 函数原型:

```
void free(void* _Block);    不能重复释放，不能释放非malloc的空间

free(p); //直接放首地址即可
```

得到合法空间大小

```
size_t _msize(void* _Block );
```

calloc

```
void* calloc( size_t _Count, size_t _Size);
```

作用跟 malloc 一样，参数拆开了

```
int* p = (int*)malloc(sizeof(int) * 10);
int* p1 = (int*)calloc(sizeof(int), 10); //乘积是总字节数

free(p);
free(p1);
```

平时用哪个都行

realloc

```
void* realloc(void* _Block, size_t _Size );
```

重新申请更大的空间

```
int* p = (int*)malloc(sizeof(int) * 10);
int* p1 = (int*)realloc(p , sizeof(int) * 20); //申请更大的 80 字节
```

过程:

- 1、申请更大的 80 字节，将首地址返回，装进 p1
- 2、将原空间的数据，依次复制进新空间
- 3、将原 p 指向的 40 字节释放, p1 装着新空间的

```
free(p1)
```

函数：C 语言模块编程的核心语法

作用：将代码按照功能划分成一段一段独立的代码，有利于代码的管理，调试，维护。

无参无返的函数

基本形式：

```
void fun(void) //函数头, 无分号
{
    //函数体
    printf("hello C3~");
}
```

函数头：

第一个 void：函数返回值，void 表示无返回值

fun：函数名字，合法的 C 语言标识符即可

()：叫参数列表，传递参数的，无参数就写个 void

函数体：

函数封装的内容。大括号不能省略

我们自己定义的函数叫自定义函数，个数随意，1 万个都行

调用：

```
int main(void)
{
    fun(); //函数调用
}
```

注意点：

- 1、函数名加上参数列表，有参数填参数，没参数什么都不要写，void 也不要写
- 2、函数只有在主函数逻辑中调用了，才能被执行，否则没有一点儿作用
光在别的自定义函数内调用了，没有在主函数逻辑内调用是没用的。
- 3、函数调用执行过程
调用时跳到函数内部执行，执行完跳回到函数调用处，再继续往下执行
- 4、函数调用比直接写代码执行慢，但是它的好处远远大于这点儿慢，所以忽略

声明：

函数可以理解为一个复杂的变量。C 语言里，标识符的使用前要定义或者声明，不然编译器不认识它。

函数声明形式：函数头加上分号

```
void fun(void);
```

位置：放在调用前的全局位置，全局就是最外层，跟主函数同逻辑层，演示

```
void fun1(void); //全局位置
int main(void)
{
    fun1();
}
```

```

        return 0;
    }

    放在同一局部区域之前，某组大括号内，
    int main(void)
    {
        void fun1(void); //局部位置，好用
        fun1();
        return 0;
    }

```

不在一起不管用，演示

```

    void fun2(void)
    {
        void fun1(void); //局部位置，也行
        fun1();          //好使
    }

    int main(void)
    {
        fun1();          //好使
        return 0;
    }

```

数量：声明可以放一万个。演示

定义只能有一个。演示

声明必须要有本体，即函数定义，没有定义没法用。

返回值：函数的值，就像 `double b = 34.5;` `b` 的值就是 34.5

形式如下：

```

    int fun1(void);          //函数声明
    int main(void)
    {
        int a = fun1();      //函数调用，将函数的返回值 12 赋值给 a
    }

    int fun1(void)           //函数定义
    {
        printf("hello C3~");
        return 12;          //返回值 12，跟返回值 int 匹配
    }

```

注意点：

- 1、可用可不用，虽然写了返回值，但是可以不用它。直接调用：fun1();
- 2、不影响其他代码的执行，fun1 函数内其他代码依然正常执行，该是什么就是什么

```

    int a = fun1();

    printf("%d %d\n", a, fun1()); //讲解该小例子

```

函数调用只要写了，就一定会执行函数的代码，调用几次，执行几遍。

3、return 的作用：

在有返回值的函数：结束函数，返回一个值

在没有返回值的函数：结束函数

结束这个功能就像循环中的 break，直接中断跳出。

函数里可以写无数个 return。

4、警告：不是每个路径都有返回值

```
int fun1(void)
{
    if (3 > 4)
        return 12;
} //当条件为假时，函数就没有返回值了
int fun1(void)
{
    if (3 > 4)
        return 12;
    return 45; //保证函数以 return 结束，才能掌控
}
```

5、谨慎返回局部变量的地址，如下：

```
int* fun(void);
int main(void)
{
    int* a = fun();
    //此时不能通过*a 进行操作，因为空间已经被释放
}
int* fun(void)
{
    int a[5] = { 1, 2, 3, 4, 5 };
    return a; //不可以
}
```

函数参数：

函数参数是函数内外链接的接口，可以互通数据，内传外，外传内

举例：

```
int Sum(int a, int b); //声明二选一，声明的参数，叫形式参数，简称形参
int Sum(int , int ); //声明可以不写变量名，形参
int main(void)
{
    int a = Sum(5, 4); //调用传参数，叫实际参数，简称实参
    printf("%d \n", a );
}
int Sum(int a, int b) //形式参数，简称形参
```

```

{
    return a+b; //求和并返回
}

```

注意点:

- 1、调用时传递实参，可以常量，可以变量，可以是任何表达式
- 2、形参是函数的局部变量，只能在函数内使用
- 3、传递参数的本质是实参给形参初始化

```

int Sum(int a, int b);

Sum(5, 4);

```

本质就是：int a = 5; int b = 4; 所以就是值传递，传递的值

4、其他类型同理

传一维数组:

函数调用时，实参是给形参初始化，所以，实参传递什么类型的数据，形参就以什么类型去接住，比如一维数组，如下：

```

void fun1(int* p, int len);
void fun2(int(*p)[5], int len);
int main(void)
{
    int a[5] = { 5, 7, 3, 8, 2 };
    fun1(a, 5); //fun1 传递 a，数组名是首元素首地址，所以用 int*p 形参
    fun2(&a, 5); //fun2 传递&a, 是一维数组地址，所以用 int(*p)[5]形参
}

void fun2(int(*p)[5], int len)
{
    int i = 0;
    for (i = 0; i < len; i++)
        printf("%d ", (*p)[i]); //不同类型的指针使用方式不一样，指针部分讲了
}

void fun1(int* p, int len)
{
    int i = 0;
    for (i = 0; i < len; i++)
        printf("%d ", p[i]);
}

```

传递数组时，也会传递数组的元素个数，这样可以在函数内直接作为循环的条件，非常方便，灵活。可以对任意长度的数组都适用了。当然不是必须的，这是经验。
所以函数想怎么传，完全取决写代码的人，没有必须。

规则：数组形式做参数，紧挨着变量名的方括号会被解析成 *

```
void fun1(int p[5], int len); //int p[5]被编译器解释成 int* p
```

```
void fun1(int* p, int len); //二者一模一样
```

```
void fun1(int p[ ], int len); //[ ]就是*, 注意只有 1 个，二维数组先不要惯性套用，再讲
```

三种写法一模一样，用谁都行，没有好坏

传二维数组：

我们之前学习，二维数组可以用三种类型的地址进行遍历，那么将二维数组传递进函数，跟我们传递的实参类型，决定形参类型，如下：

```
void fun1(int(*p)[2][3], int hang, int lie);
```

```
void fun2(int(*p)[3], int hang, int lie);
```

```
void fun3(int *p, int hang, int lie);
```

```
int main(void)
```

```
{
```

```
    int a[2][3] = {5, 7, 3, 8, 2, 6};
```

```
    fun1(&a, 2, 3);
```

```
    fun2(a, 2, 3); //
```

```
    fun2(&a[0][0], 2, 3);
```

```
}
```

```
void fun1(int(*p)[2][3], int hang, int lie)
```

```
{
```

```
    int i, j;
```

```
    for (i = 0; i < hang; i++)
```

```
        for (j = 0; j < lie; j++)
```

```
            printf("%d ", (*p)[i][j]);
```

```
}
```

```
void fun2(int(*p)[3], int hang, int lie)
```

```
{
```

```
    int i, j;
```

```
    for (i = 0; i < hang; i++)
```

```
        for (j = 0; j < lie; j++)
```

```
            printf("%d ", p[i][j]);
```

```
}
```

```
void fun3(int* p, int hang, int lie)
```

```
{
```

```
    int i;
```

```
    for (i = 0; i < hang*lie; i++)
```

```
        printf("%d ", p[i]);
```

```
}
```

规则：数组形式做参数，紧挨着变量名的方括号会被解析成 *

```
void fun1(int p[2][5], int hang, int lie); //[2]的方括号被编译器解释成*
```

```
void fun1(int p[ ][5], int hang, int lie); //二者一模一样
```

```
void fun1(int (*p)[5], int hang, int lie); //[ ]就是*, 注意只有 1 个
```

三种写法一模一样，用谁都行，没有好坏

传地址：

如何通过函数参数修改外部变量的空间？

```
void fun(int a);
int main(void)
{
    int a = 2;    //a 是 main 的局部变量，只在 main 有效，跟 fun 函数里完全不一样
    fun(a);
    printf("%d", a);
}
void fun(int a)    //a 是 fun 的局部变量，只在 fun 有效
{
    a = 3;
}
```

函数调用使用：int a = 2; 实参给形参初始化，传递的是 2 这个值

fun 函数内 a=3 是赋值的 fun 的 a，跟 main 的 a 完全不一样

空间的操作有两种：名字，地址，传名字不行，传地址一定行

```
void fun(int *p);
int main(void)
{
    int a = 2;
    fun(&a);
    printf("%d", a);
}
void fun(int* p)
{
    *p = 3;
}
```

如此，fun 内 p=&a,所以*p 就是外部的 a 了，可以修改

即，要修改谁，就传谁的地址，叫传址调用。

传二级指针

如果想要修改指针的指向，那就传指针的地址，如下：

```
void fun(int **p);
int main(void)
{
    int* p;
    fun(&p);    //修改一级指针的指向，传指针变量的地址
    printf("%p", p);
}
void fun(int** p)    //所以此处用 int** p
```

```
{
    *p = NULL;
}
```

误区：传地址就是传址调用，这是不对的

正确：要修改谁，就传谁的地址，叫传址调用

```
void fun(int *p);
int main(void)
{
    int a = 12;
    int* p = &a; // 想要让 p 重新指向
    fun(p);      // 没有用, 仅仅是把&a 传进去了, 无法修改 p 的指向
    printf("%p", p);
}
void fun(int* p)
{
    p = NULL;
}
```

递归函数：函数自己调用自己，就叫递归调用

```
void fun(void)
{
    fun();
}
```

递归的本质就是循环。循环可以完全代替递归，但是递归在某些情况下代码会简洁一些。

可控递归三要素：

```
void fun(int i)    // 循环控制变量
{
    if (i < 5)     // 循环的条件
    {
        printf("%d ", i);
        fun(i+1); // 循环控制变量变化, 不要用 i-- i++
    }
}
int main(void)
{
    fun(1);        // 循环控制变量初始值
}
```

输出：1 2 3 4

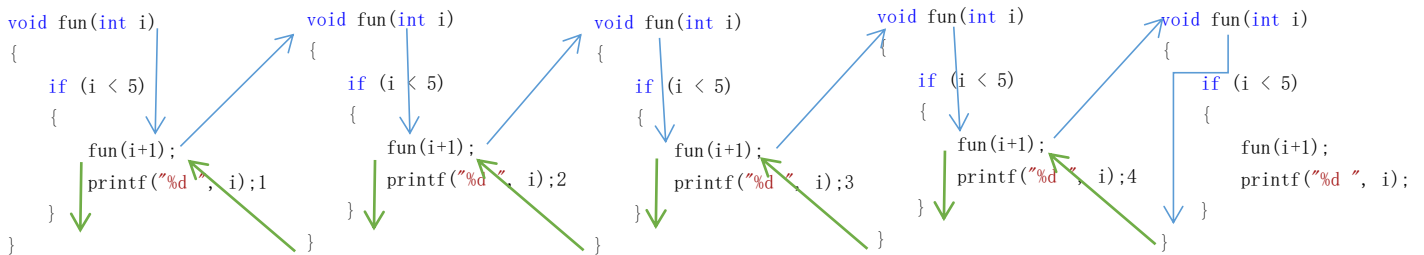
递归层数不能太多，会爆栈

递归展开理解

```
void fun(int i)
{
    if (i < 5)
    {
        //printf("%d ", i);    //在前输出 1234
        fun(i+1);
        printf("%d ", i);    //在后输出 4321
    }
}

int main(void)
{
    fun(1);
}
```

展开分析详细过程，自己调用自己



通项公式的递归写法:

知道数列的指定项，然后根据通项公式

1、斐波拉契数列:

第 1 项 $f(1) = 0$

第 2 项 $f(2) = 1$

第 n 项 $f(n) = f(n-1) + f(n-2)$

```
int fun(int n)
{
    if (n == 1)        //指定项
        return 0;
    else if (n == 2)    //指定项
        return 1;
    else
        return fun(n-1) + fun(n-2);    //通项公式
}
```

2、阶乘:

第 1 项 $f(1) = 1$

第 n 项 $f(n) = n * f(n-1)$

```
int fun(int n)
```

```

{
    if (n == 1)                //指定项
        return 1;
    else
        return n * fun(n-1);  //通项公式
}

```

函数指针

函数也有地址，叫函数地址，对函数名取地址：&fun

特点：函数名就是函数地址：fun == &fun

类型：去掉函数名，就是函数的类型，比如

int fun(int a, double b) 函数类型是 int (int a, double b)

函数指针就是：int (*p)(int a, double b) = fun; //&fun

定义规则跟数组指针定义方式一样

函数调用：

形式是：函数名+参数列表

本质是：函数地址+参数列表

fun(12, 4.5);

(&fun)(12, 4.5);

p(12, 4.5);

(*p)(12, 4.5);

均可，为方便，常采用 1 3

字符数组

```
char ch[5] = { 'A', 66, 'C', 'D', 69 };
char ch[5] = { 'A', 66, 'C' };           //初始化部分元素，其余默认初始化为 0
char ch[ ] = { 'A', 66, 'C' };          //初始化时，可不写个数
```

输出

```
int i = 0;
for (i = 0; i < 5; i++)
    printf("%c ", ch[i]);
```

字符串

定义：以 `\0` 结尾的字符数组，`\0` 就是数字 0，ASCII 表上第一个字符

'`\0`'：字符，数字 0 的字符形式，`\`叫转义字符，`\0` 共同构成 1 个字符，占 1 字节
0：数字 0，数值上'`\0`'，NULL，0 三者一样，作用在不同场景，以经验区分
'0'：字符 0，48 的字符形式

举例：

```
char ch1[5] = { 'A', 'B', 'C', 'D', '\0' }; //字符串
char ch2[5] = { 'A', 'B', 'C', 0 };         //字符串
char ch3[ ] = { 'A', 'B', 'C', NULL };      //字符串，但是一般不这么写
char ch3[5] = { 'A', 'B', 'C' };            //字符串，默认有 0
```

所以，字符串的本质还是字符数组，

常量字符串：双引号包着

"hello c3" "您好" "C3 程序猿"

1、常量字符串本质就是字符数组，该字符串就是数组的名字

"hello c3"[0]

"hello c3"[1]

"hello c3"[2] ... 访问元素，可以输出，但是不能赋值修改，因为是常量字符串

2、自带 `\0` 结尾，"hello c3"就是：'h','e','l','l','o',' ',' ','c','3','\0' 共 9 个字符。

可以使用 `sizeof("hello c3")`;字节数为 9

字符串初始化：用常量字符串

```
char ch5[10] = {"hello c3"};
char ch6[10] = "hello c3";           //常用，2
char ch7[ ] = "hello c3";            //初始化时，可不写个数
```

初始化过程：

数组 `ch5`,`ch6`,`ch7` 是两块空间，初始化的过程是将常量字符串复制进数组里，操作 `ch` 是操作的复制份，是不是常量字符串本身。

字符指针初始化:

```
char* str = "hello c3~"; //C 语言里可以, c++要用下面
const char* str = "hello c3~"; //最完美写法, 最精确的写法
```

const 就是常的意思, 常量指针, 指向常量字符串, 因为字符串就是自身的数组名字
相当于

```
char a[10];
char* str = a;
```

中间\0

"he\0llo c3~", 那么字符串就是: he 字符串只找结尾

输出: %s

```
char ch[10] = "hello c3";
printf("%s", ch); //%s 从首地址 ch 输出, 一直到\0 结束, 没有\0 就会越界
printf(ch); //printf 的第一个参数就是常量字符串, 所以可直接输出
puts(ch); //puts 专门用于输出字符串
```

这两种输出都是从首地址开始, 一直输出到\0 结束

```
char ch[10] = "hell\0o c3";
printf(ch+2); //输出 ll
puts(ch+1); //输出 ell
```

\0 是字符串操作结束标志, 当然从字符数组角度, 按元素操作, 就不分\0 还是别的字符了

输入: %s

```
char str[20] = { 0 };
scanf_s("%s", str, 20); //两个参数, 原版 1 个参数
puts(str);
```

空格会作为分隔符, 所以无法获取到空格

输入: hello world<回车> str 里装 hello

想输入空格怎么办? 使用 gets_s 函数

```
char str[20] = { 0 };
gets_s(str, 20); //原版 gets 没了
输入: hello world<回车> str 里装 hello world
```

字符串操作函数: string.h

字符串拷贝: 将字符串装进参数 1 的数组, 注意别越界

```
char str[20] = { 0 };           //必须有合法空间, char* str 不行, 除非 malloc
strcpy(str, "hello world");     //旧版, #define _CRT_SECURE_NO_WARNINGS
strcpy_s(str, 20, "hello world"); //新版
```

字符串拷贝 n 个: 将字符串前 n 个字符

```
char str[20] = { 0 };
strncpy(str, "hello world", 3); //旧版, 无\0
strncpy_s(str, 20, "hello world", 3); //新版, 自动加\0
```

字符串拼接: 将一个字符串拼接在另一个字符串尾部, 注意别越界

```
char str[20] = "hello ";
strcat(str, "world"); //旧版, #define _CRT_SECURE_NO_WARNINGS
strcat_s(str, 20, "world"); //新版
```

字符串拼接 n 个: 将一个字符串前 n 个拼接在另一个字符串尾部, 注意别越界

```
char str[20] = "hello ";
strncat(str, "world", 3); //旧版, #define _CRT_SECURE_NO_WARNINGS
strncat_s(str, 20, "world", 3); //新版
```

字符串比较: 从头依次比较, 直到第一个不同的字符, 此时谁大就谁大, 否则相等

```
int a = strcmp("abc", "abc"); //一样大, 返回 0
int a = strcmp("abr", "abcde"); //前大, 返回 1
int a = strcmp("abcde", "ar"); //后大, 返回 -1
```

字符串比较前 n 个: 从头依次比较, 直到第一个不同的字符, 此时谁大就谁大, 否则相等

```
int a = strncmp("ab", "abc", 1); //一样大, 返回 0
int a = strncmp("abr", "abcde", 2); //一样大, 返回 0
int a = strncmp("abcde", "ar", 2); //后大, 返回 -1
```

字符串长度: 到\0 终止, 不算\0

```
size_t a = strlen("abcd"); //长度是 4
size_t a = sizeof("abcd"); //5 个字节
size_t a = strlen("abc\0def"); //长度是 3
size_t a = sizeof("abc\0def"); //8 个字节
```


结构体:

前面学习的数据都是单一的数据, 如果某个数据节点包含很多个类型, 比如学生信息, 包含名字(字符串), 年龄(int), 成绩(double), 可以单独定义三个变量:

```
char name[20];  
int age;  
double score;
```

但是当操作某个学生信息时, 这三个变量都得操作, 如果信息更多, 那么有可能会造成误操作, 或者漏操作, 非常的不方便。

如何把这些数据组合在一起, 构成一个类型呢?那就是结构体, 即通过一个语法结构, 将这些数据类型包在一起, 这样操作时就一块操作了, 非常的方便。如下

```
struct Node           //struct 是关键字   Node 是名字, 合法的标识符即可  
{  
    char name[20];     //叫结构体成员  
    int age;  
    double score;  
};                     //结尾加上分号, 不加报错
```

整个这个结构是一个数据类型, 跟 int double 一样, 类型本身不占空间, 用这个类型定义变量时, 变量占空间。

定义变量:

```
struct Node no;           //声明结构体变量, 类型名固定  
struct Node nd = {"小明", 23, 98.5}; //依次初始化给成员  
struct Node ne = {"小明"}; //初始化部分元素, 其他为 0  
struct Node* np = &nd;    //指针对象指向合法空间
```

成员访问: 普通变量用 . 指针变量用-> 叫取成员运算符

```
printf("%s %d %lf\n", nd.name, nd.age, nd.score);  
printf("%s %d %lf\n", np->name, np->age, np->score);
```

成员赋值

```
strcpy_s(nd.name, 20, "大华"); //字符串必须使用循环或者 strcpy_s 函数, 不能=  
np->age = 45;  
nd.score = 56.7;
```

互相赋值: 结构体变量可以直接互相赋值

```
ne = nd;  
*np = ne;  
no = (struct Node) {"栗子", 19, 78.9}; //复合文字
```

```
printf("%s, %d, %lf\n", no.name, no.age, no.score);  
printf("%s, %d, %lf\n", (&no)->name, (&no)->age, (&no)->score);  
printf("%s, %d, %lf\n", pno->name, pno->age, pno->score);  
printf("%s, %d, %lf\n", (*pno).name, (*pno).age, (*pno).score);
```

无名结构体：结构体可以没有名字，如下：

```
struct          //没有名字，无法通过名字定义结构体变量，使用受限
{
    char name[20];
    int age;
    double score;
} nd = {"栗子", 19, 78.9}, *ne;    //只能在这里定义变量，有名字的也可以在这定义变量
```

指针成员

```
struct Node
{
    char* name;          //指针成员要指向合法空间，才能被使用
    struct Node* next;
};

struct Node nd;

struct Node no = {(char*)malloc(20), &nd};    //指针指向空间
free(no.name);          //释放， next 不用释放，因为是 nd 局部变量的地址
```

结构体成员

```
struct Node
{
    struct Node* next;    //可以是当前结构体类型指针
    struct Node ne;       //不可以是当前结构体类型变量
};
```

因为出现了悖论。

函数成员：结构体内不能放函数，但是可以放函数指针

```
int sum(int a, int b)
{
    return a + b;
}

struct Node
{
    int a;
    int (*p)(int a, int b);
} no = {12, sum};

int main(void)
{
    int a = no.p(3, 4);
}
```

结构体的大小，sizeof 计算，并不是单纯的所有元素大小之和。

```
sizeof(struct Node), sizeof(nd)
```

联合：所有成员共用一块空间，起始地址起一样

sizeof 得到最大成员的空间

```
union UN
```

```
{  
    int a;  
    short s;  
    char c;  
};
```

```
union UN un = { 666666 }; //只能初始化 1 个数据，即第一个数据
```

修改一个成员，其他的成员也会变化。

```
un.s = 456;  
un.c = 78;  
un.a = 1111;
```

数值在可表示范围内，正常表示，数值不在可表示范围内，存余数

char %256

short %65536

int %4294967296

枚举： enum，一组有名字的 int 类型数据类型的类型

```
enum color {yellow, red, black, white, pink, blue};
```

定义了一个整型类型，该整形类型有 6 个数据，默认 0 ~ 5，分别是各自的名字

int 整型类型的子集

char 类型可表示 256 个数据 -128 ~ 127

short 类型可表示 65536 个数据 -32768 ~ 32767

定义变量

```
enum color co = yellow;
```

//理论上 co 是一个 int 类型变量，只能赋值这几个枚举，标准用法，通用

//但是实际中，可以简单的做 int 使用，可以赋值 100，也可以 int a = yellow，不通用

设置指定值

默认从 0 开始，yellow 是 0 的名字，可以直接使用 yellow 代替 0，不是变量名，是单纯的名字。

```
yellow, red, black, white, pink, blue  
0      1      2      3      4      5
```

也可以指定值，形式如下：

```
enum color {yellow = 3, red, black, white = 20, pink, blue};
```

```
yellow, red, black, white, pink, blue  
3      4      5      20    21    22
```

指定了某个，后面的值依次递增

枚举应用：枚举在日常编程中应用很多

比如：贪吃蛇小游戏中方方向的判断，设定：{0,1,2,3}分别代表{东,西,南,北}四个方向

```
if (0 == dir) {}  
else if (1 == dir) {}  
else if (2 == dir) {}  
else if (3 == dir) {}
```

但是当使用 3 时，一时间不知道 3 代表哪个方向。

枚举来了

```
enum DIR{east, south, west, north} dir;  
if (east == dir) {}  
else if (south == dir) {}  
else if (west == dir) {}  
else if (north == dir) {}
```

如此，根本不需要知道 3 是几，只需要使用 west 这个名字即可

当然这种写法可以用变量代替

```
int east = 0, south = 1, west = 2, north = 3; //占空间
```

也可以用宏代替

```
#define east 0  
#define south 1  
#define west 2  
#define north 3
```

感觉哪个方便用哪个，宏跟枚举都常用。

文件操作：

什么是文件？

磁盘中的各种文件，图片文件(jpg,png,bmp,gif...), 文本(doc,txt,dot,rtf,doct,wps,wpt,pdf,c,cpp,html,css,py...), 其他(zip,7z,exe,msi,dll,lib,apk...)无数种文件类型，这些都是文件，文件操作就是操作这些文件。

不同的文件区别？

不同的文件存储的数据类型不同，文件内存储数据的格式不同。

打开文件：fopen fopen_s

读文件：fgetc fgets fscanf fread

写文件：fputc fputs fprintf fwrite

文件指针：fseek rewind ftell

关闭文件：fclose

操作流程：打开文件，读写操作，关闭文件

打开文件：

```
FILE* pFile = fopen("stu.txt", "r"); //旧版的函数
```

```
FILE* pFile = NULL;
errno_t et = fopen_s(&pFile, "stu.txt", "r"); //新版的函数
```

简介:

FILE: 文件类型, *, 就是文件指针, 打开文件的本质就是将文件内容存进文件缓冲区, FILE* 可以理解为文件缓冲区首地址。随着操作, 文件指针偏移, 指向哪儿就从哪儿开始操作。

返回值: 旧函数直接返回文件操作地址。新函数通过参数 1 的传址调用获得文件地址

新函数返回值表示错误码, 0 表示成功, 非 0 表示打开失败

参数 1: 新函数, 参数 1 的传址调用获得文件地址

参数 2: 文件路径:

相对路径: 默认相对于项目文件所在目录, 写个名字即可

绝对路径: 完整路径名

参数 3: 打开方式:

文本模式:

"r"/"rt" 只读, 只能调用读函数, 文件必须存在, 否则失败。文件指针指向头字节。
"r+" 可读可写读, 读写函数都能调用, 文件必须存在, 否则失败。文件指针指向头字节。
"w"/"wt" 擦除写, 只能调用写函数, 文件不存在时创建文件。文件指针指向头字节。
"w+" 可读可写, 读写函数都能调用, 文件不存在时创建文件。文件指针指向头字节。
"a"/"at" 附加写, 只能调用写函数, 文件不存在时创建文件。文件指针指向尾字节。
"a+" 可读可写, 读写函数都能调用, 文件不存在时创建文件。文件指针指向尾字节。

二进制模式:

"rb" 只读, 只能调用读函数, 文件必须存在, 否则失败。文件指针指向头字节。
"rb+" 可读可写读, 读写函数都能调用, 文件必须存在, 否则失败。文件指针指向头字节。
"wb" 擦除写, 只能调用写函数, 文件不存在时创建文件。文件指针指向头字节。
"wb+" 可读可写, 读写函数都能调用, 文件不存在时创建文件。文件指针指向头字节。
"ab" 附加写, 只能调用写函数, 文件不存在时创建文件。文件指针指向尾字节。
"ab+" 可读可写, 读写函数都能调用, 文件不存在时创建文件。文件指针指向尾字节。

二者区别:

对文件行结尾代码层面处理不同, 在 windows 系统中, 行结尾是\r\n, 文本模式读到的是\n, 二进制模式读到的是\r\n

在 linux 系统下都是\n, 没有区别。

默认情况下不能打开两次

关闭文件:

```
fclose(pFile);
```

使用前判断打开成功:

```
if (0 != et || NULL == pFile)
    return 0;
```

文件读写操作：一次读写 1 个字符

```
int a = fputc('a', pFile); //一次写入文件 1 个字符，返回值是字符的 ASCII 整数
int a = fgetc(pFile);      //一次读出文件 1 个字符，返回值是字符的 ASCII 整数
注意，读用 r，写用 w a，暂时先不用+，后面例子用
```

行结尾演示：r 读出\n

rb 读出\r\n

w a 写入\n

wb ab 写入\r\n

文件结尾演示：文件结尾标记 EOF 即 -1

feof(pFile) 当到结尾时，返回真，不是结尾返回假

循环读文件：

```
while (1)
{
    int a = fgetc(pFile);
    if (feof(pFile))
        break;
    putchar(a);
}

while (feof(pf) == 0) //!feof(pf)
{
    char c = fgetc(pf);
    //if (EOF == c)
    //if (feof(pf))
    //break;
    putchar(c);
}
```

文件读写操作：一次读写 n 个字符

```
int a = fputs("hello world\n", pFile); //写入字符串
char str[20] = { 0 };
char* p = fgets(str, 5, pFile); //读取 5-1 个字符，返回值是 str 的地址
```

文件读写操作：格式化读写

```
fprintf(pFile, "a:%d, b:%lf, s:%s", 12, 45.6, "hello c3"); //格式化字符串写入文件
int a = 0;
double b = 0.0;
char str[20] = { 0 };
fscanf_s(pFile, "a:%d, b:%lf, s:%s", &a, &b, str, 20); //成对使用，新版
fscanf(pFile, "a:%d, b:%lf, s:%s", &a, &b, str); //老版
```

前三组都是专门读写字符串的。

最后一组：fread fwrite 是以 2 进制数据读写。

比如 12 这个整数，前三组就是存入的字符 1 2 。而最后一组存 00001100 存入 1 字节。

所以数值类型的数据存储，可以直接使用 fread fwrite，比前三者效率高，因为前三者要先转换，然后存。

```

struct Node
{
    int a;
    char str[20];
    double b;
};

struct Node no = { 12, 34.5, "hello c3" };
int a = fwrite(&no, sizeof(no), 1, pFile);
返回值：实际写入的组数，即参数 3

```

```

struct Node nd;
int a = fread(&nd, sizeof(nd), 1, pFile); //一次读出一个结构体，前三个就不好办
返回值：实际写入的组数，即参数 3，读没了，返回 0

```

文件指针操作函数：

前面的 4 组函数，每个函数的操作，都会使得文件指针移动。

```

rewind(pFile); //将文件指针指向文件首
long l = ftell(pFile); //返回文件指针指向的文件中的字节下标
int a = fseek(pFile, 0, SEEK_SET); //设置文件指针指向哪个字节，成功返回 0
    参数 2：相对位置
    参数 3：SEEK_SET 文件首，配合参数 2，比如 3，就是首+3 的位置，即第四个字节
           SEEK_END 文件尾，配合参数 2，比如-3，就是结尾-3 的位置，即倒数第四个字节
           SEEK_CUR 当前位置，配合参数 2，负数左移，正数右移。

```

进制转换：日常编程用到了二进制，直接使用计算器等工具转换。不用非得口算

10 进制转其他进制，翻转取余法

359 / 2 = 179	余 1	359 / 16 = 22	余 7
179 / 2 = 89	余 1	22 / 16 = 1	余 6
89 / 2 = 44	余 1	1 / 16 = 0	余 1
44 / 2 = 22	余 0		167
22 / 2 = 11	余 0		
11 / 2 = 5	余 1		
5 / 2 = 2	余 1		
2 / 2 = 1	余 0		
1 / 2 = 0	余 1		
1 0110 0111			

其他进制转 10 进制，基数相加法

359

10 进制： $9 * 1 + 5 * 10 + 3 * 100$

2 进制： $1*1 + 1*2 + 1*4 + 0*8 + 0*16 + 1*32 + 1*64 + 0*128 + 1*256$

16 进制： $7*1 + 6*16 + 1*256$

位运算：就是使用其二进制位进行计算。

位运算符：

& 按位与
| 按位或
~ 按位取反
^ 按位亦或
<< 位左移
>> 位右移

复合赋值运算符：

a&=2 a = a&2
a|=2 a = a|2
a^=2 a = a^2
a<<=2 a = a<<2
a>>=2 a = a>>2

& 按位与

规则：1&1==1 1&0==0 0&1==0 0&0==0

```
unsigned char a = 12, b = 3;
```

```
12  0000 1100
3   0000 0011
&   0000 0000    0
```

| 按位或

规则：1|1==1 1|0==1 0|1==1 0|0==0

```
unsigned char a = 12, b = 3;
```

```
12  0000 1100
3   0000 0011
|   0000 1111    15
```

~ 按位取反

规则：1反0 0反1

```
unsigned char a = 12, b = 3;
```

```
12  0000 1100    ~a  1111 0011    243
3   0000 0011    ~b  1111 1100    252
```

^ 按位亦或

规则：1^1==0 0^0==0 1^0==1 0^1==1 相同为0，不同为1

```
unsigned char a = 9, b = 3;
```

```
9   0000 1001
3   0000 0011
^   0000 1010    10
```

<< 位左移

规则：所有位向左平移，左侧截断，右侧补0

```
unsigned char a = 12, b = 3;
```

```
a           0000 1100
```



```

a << 3  0000110 0000  96
b                0000 0011
b << 7  0000 001 1000 0000  128

```

>> 位右移

规则：所有位向右平移，右侧截断，负数左侧补 1，正数左侧补 0

```

unsigned char a = 181; char b = 3, c = -17;

a          1011 0101
a >> 3     0001 0110 101  22
b          0000 0011
b >> 3     0000 0000 011  0

c          11101111
c >> 3     11111101 111  -3

```

多文件：就是有多个源文件的编程，

函数的作用：将一大堆代码按照功能封装成多个函数，方便维护管理

文件的作用：将一大堆函数按照功能分别写在多个文件中，方便维护管理

多文件的本质跟单文件是一样的，比如想要使用使用一个函数，不管在不在同一个文件中，都需要声明，声明一下就能使用别的文件里内容了。简单演示：

如果要使用的函数比较多，那么每个使用的文件内的最前都要写上相同的声明，写很多次，非常不方便，所以就出来了头文件，头文件内写着所有需要的声明，那个文件需要，就直接包含该头文件即可，`#include <stdio.h>`，这样就省时省力了。

既然包含头文件可以代替直接写声明，所以包含头文件本身就是声明了其内所有声明。

推断出：`#include <stdio.h>`的作用就是单纯的复制黏贴。实际也是这样。

`#include <stdio.h>`

`#include "stdio.h"`

单引号与双引号的区别：

1、标准库头文件使用尖括号，自定义头文件使用双引号

2、查找起始路径不一样：

工程所在相对路径：`vcxproj`

包含默认查找路径：项目属性 -> `vc++目录` -> 包含目录

双引号先在工程文件所在路径查找，然后在包含的默认路径依次查找，直到找到它

尖括号直接在包含的默认路径依次查找，直到找到它

直接写头文件的绝对路径是编译效率最高的。

防止头文件重复包含

```
#pragma once
```

```
#ifndef _AH //条件编译
```

```
#define _AH
```

```
各种声明
```

```
#endif
```

宏 #define

常量宏：宏是单纯的替换

```
#define N 12
```

```
#define M printf
```

```
#define Y , a, b);
```

宏不进行计算，只是单纯的替换

```
#define N 3+2
```

```
#define M (3+2)
```

输出 $N*2$ $3+2*2$

$M*2$ $(3+2)*2$

参数宏：

```
#define N(x,y) x+y
```

```
#define M(x,y) (x+y)
```

```
#define X(x,y) N(x,y)*M(x,y)
```

输出 $N(4,5)*2$ $4+5*2$

$M(4,5)*2$ $(4+5)*2$

$2*X(4,5)$ $N(4,5)*M(4,5)*2$ $2*4+5*(4+5)$

存储类

typedef 类型重命名，给类型名起个新的名字

```
typedef int myint;                //myint a = 34;
```

```
typedef int* pint;                //pint p = &a;
```

```
typedef int** ppint;              //ppint pp = &p;
```

```
typedef int arr[5];                //arr b = {1, 2, 3, 4, 5};
```

```
typedef int(*parr)[5];            //parr pa = &a;
```

```
typedef int(*pfun)(int, double); //pfun pf = fun;
```

```
typedef struct Node node;        //node a = {"小明", 56.5};
```

```
typedef struct Node* pnode;       //pnode pn = &a;
```

```
typedef struct Node
```

```

{
    char str[20];
    double b;
}node;          //node a = {"小明", 56.5};
typedef struct  //无名
{
    char str[20];
    double b;
}NODE;          //NODE b = {"小明", 56.5};

```

全局变量：定义在全局位置的变量，作用域是整个工程，生命周期与程序共存亡
只能定义在源文件中，不能重复定义，所有文件都可用。

默认初始化成 0

在其他源文件使用，先声明：extern int a; //不能初始化

叫声明外部变量 extern

全局变量与局部变量重名，在局部区域内，局部变量起效。实际应用中不要重名。

static 静态全局变量：

在全局变量前加个 static，作用域是所在文件，生命周期与程序共存亡

如果想要这个变量只在某个源文件中使用，其他文件不可用，那么就声明成静态全局变量

static int a = 12; //默认初始化成 0

static 静态局部变量：默认初始化 0

在局部变量前加上 static，作用域是所在大括号，生命周期与程序共存亡

相当于全局静态变量，其作用域仅是所在大括号。

```

static void fun(void)
{
    static int a = 2;
    a++;
    printf("%d ", a);
}

```

fun(); //输出 3

fun(); //输出 4

fun(); //输出 5

而不是每次都重新定义变量 a，相当于如下：

static int a = 2; //a 只能在 fun 中使用，本质还是全局静态变量

```

static void fun(void)
{
    a++;
    printf("%d ", a);
}

```

局部变量：

在某个大括号内定义的变量，叫局部变量。作用域是所在大括号，生命周期是所在大括号

auto 关键字，局部变量前默认有 **auto 关键字**，可省略，auto 是自动的意思，局部变量也叫自动变量，**自动就是空间自动申请自动释放**，完全操作系统管理。

```
auto int a = 12;
```

在 vc++2010 中，auto 有自动识别类型的功能：

```
auto a = 12; //int
auto b = 23.4; //double
auto c = 4.5f; //float
```

register: 寄存器变量

寄存器不是物理内存，它是集成在 cpu 上的一块存储区域，所以不能取地址。

```
register int a = 23;
&a; //直接报错
```

实际上，寄存器空间非常珍贵有限，所以 a 是否在寄存器上，取决于操作系统的算法调度，一般都是不行。

const 常量修饰符

```
const int a = 23; //报错 a=45;
const int b[5] = {1,2}; //报错 b[2] = 4;
```

```
int a = 12, b=56 ;
const int* p = &a; //报错 *p = 23; p = &b;不报错
int* const p = &a; //不报错 *p = 23; p = &b;报错
const int* const p = &a; //报错 *p = 23; p = &b;报错
```

运算符优先级与结合性的本意是先后结合哪个子表达式，相当于先分组，分组完毕后，从左向右依次计算，如下：

```
int a = 1, b = 2, c = 3, d = 4, e = 5, f;  
f = a + b - c * d / e;
```

结合过程如下：

1、乘除优先级高，先结合，乘除同级从左到右结合，如下

```
f = a + b - ((c * d) / e);
```

即按优先级先结合乘法/除法，乘除结合性从左到右，所以先结合 $c*d$ ，然后 $/e$

2、再结合加减，同级从左到右结合

```
f = ((a + b) - X); // X 表示 ((c * d) / e)
```

即先结合 $a+b$ ，然后结合减法

3、最后结合赋值运算符

```
( f = (Y - X) ) // Y 表示 (a + b)
```

运算过程如下：从左到右运算

1、先执行子表达式 Y, $a+b==3$

2、再执行 X, $c*d==12$ ，然后执行 $12/e==2$

3、然后执行 $3-2==1$

4、最后执行 $f=1$

此表达式先算 $a+b$ ，而不是 $c*d$ ，所以优先级与结合性不是单纯的谁先算谁后算。我们正常脑算看待优先级可以先算后算来处理，不会影响结果。

最通用的写法：

```
int i; //即循环控制变量的定义位置  
for (i = 0; i < 5; i++)
```

在 vs2013 以上版本可以如下写：

```
for (int i = 0; i < 5; i++)
```

但是该写法很多编译器都不支持，所以建议使用上边那种，以增加代码移植性。

vc++6.0 中对第二种写法的逻辑跟第一种写法一样，而不是把 i 解释成 for 循环的局部变量。

++运算符分后置++与前置++，相同点，其作用都是使得变量的值自加 1,即 a++或者++a 执行后，a 的值都变大 1 了。区别，参与运算时，前置++用自加后的值参与运算，后置++用自加前的值参与运算。

后置++使得变量自加 1，参与运算时使用自加前的值：

```
int a = 1, b;  
b = 2 + a++ + a++;
```

vs:

后置++，先用 a 计算表达式，后自加 1，如下：

```
b = 2 + a + a; //4  
a+=1; //在后 a == 2  
a+=1; // a == 3
```

vs:

前置++，先自加 1，后用 a 参与运算，如下：

```
b = 2 + ++a + ++a;  
相当于：  
a+=1; //在前 a==2  
a+=1; // a == 3  
b = 2 + a + a; // 8
```

gcc :

```
int a = 1, b;  
b = 2 + a++ + a++;  
严格遵循优先级与结合性，先组合然后从左到右计算  
b = 2 + (a++) + (a++);  
先执行 2+(a++)，即 3，a==2  
然后 3+(a++)，即 5，a==3
```

```
b = 2 + ++a + ++a;  
严格遵循优先级与结合性，先组合然后从左到右计算  
b = 2 + (++a) + (++a);  
先执行 2 + (++a)，即 4，a==2  
再执行 4 + (++a)，即 7，a==3
```

--同理，只不过变减法了。

逻辑运算符：|| 读 或

运算规则：

真 || 真 为真 结果 1

真 || 假 为真 结果 1

假 || 真 为真 结果 1

假 || 假 为假 结果 0

真短路 || ：因为“真 || 表达式”结果一定是真，即“表达式”不管是什么都不改变结果，所以“表达式”就不执行了，执行也是浪费时间，举例如下：

```
int a = 1, b = 1;
```

```
5 || (a=2) || (b=2);
```

过程：（依据优先级结合性执行即可）

5 为真，子表达式 5 || (a=2) 中发生短路，所以 (a=2) 不执行，子表达式 5 || (a=2) 结果为真，即 1。

剩下 1 || (b=2)，1 为真，短路||，所以 (b=2) 不执行，表达式 1 || (b=2) 结果为真。

最终整个表达式结果为真，即 1，a、b 值保持 1 不变。

```
0 || (a=2) || (b=2);
```

过程：（依据优先级结合性执行即可）

0 为假，子表达式 0 || (a=2) 不发生短路，执行 a=2，a 变为 2，赋值表达式的结果为 2，子表达式 0 || (a=2) 为 0 || 2 为真，即 1。

剩下 1 || (b=2)，1 为真，子表达式 1 || (b=2) 发生短路，(b=2) 不被执行。表达式 1 || (b=2) 结果为真。

最终整个表达式结果为真，即 1，a 变为 2、b 保持 1 不变。

短路原理的目的：增加程序执行效率

逻辑运算符: && 读 与

运算规则:

真 && 真 为真 结果 1

真 && 假 为假 结果 0

假 && 真 为假 结果 0

假 && 假 为假 结果 0

假短路&&: 因为“假 && 表达式”结果一定是假, 即“表达式”不管是什么都不改变结果, 所以“表达式”就不执行了, 执行也是浪费时间, 举例如下:

```
int a = 2, b = 2;
```

```
0 && (a=0) && (b=0);
```

过程: (依据优先级结合性执行即可)

0 为假, 子表达式 0 && (a=0)发生短路, 不执行(a=0), 子表达式 0 && (a=0)为假, 即 0。

剩下 0 && (b=0), 0 为假, 子表达式 0 && (b=0)发生短路, 不执行(b=0), 子表达式 0 && (b=0)为假, 即 0。

最终整个表达式结果为假, 即 0, a、b 保持 2 不变

```
5&&(a=0)&&(b=0);
```

过程: (依据优先级结合性执行即可)

5 为真, 子表达式 5&& (a=0)不发生短路, 执行 a=0, a 变为 0, 赋值表达式的结果为 0, 子表达式 5&& (a=0)为 5&& 0 为假, 即 0。

剩下 0 && (b=0), 0 为假, 子表达式 0 && (b=0)发生短路, (b=0)不被执行。表达式 0 && (b=0)结果为假。

最终整个表达式结果为假, 即 0, a 变为 0、b 保持 2 不变。

短路原理的目的: 增加程序执行效率

与(&&)、或(||)混合短路:

执行原则: 按照优先级与结合性分组, 然后从左到右计算, 遇到短路就短路, &&优先级高于 || 。举例 1:

```
int a = 1, b = 1, c = 1, d = 1;  
(a=0) || (b=0) && (c=2) || (d=0);
```

执行过程:

先结合, 加上小括号, $(a=0) || ((b=0) \&\& (c=2)) || (d=0)$

子表达式 $a=0$ 的结果是 0, 不短路, 继续执行子表达式 $((b=0) \&\& (c=2))$

$b=0$ 的结果是 0, 0 短路 &&, 所以 $c=2$ 就不执行了, $((b=0) \&\& (c=2))$ 结果是 0, 前半部分 $0 || 0$ 为 0

剩下 $0 || (d=0)$, 执行 $d=0$, 最终 $a=0, b=0, c=1, d=0$

举例 2:

```
int a = 1, b = 1, c = 1, d = 1;  
(a=2) || (b=0) && (c=2) || (d=0);
```

执行过程:

先结合, 加上小括号, $(a=2) || ((b=0) \&\& (c=2)) || (d=0)$

子表达式 $a=2$ 的结果是 2, 为真, 发生短路, 不执行子表达式 $((b=0) \&\& (c=2))$, 子表达式 $(a=2) || ((b=0) \&\& (c=2))$ 结果是真, 即 1

剩下 $1 || (d=0)$, 发生短路, 不执行 $d=0$, 最终整个表达式结果是 1, 其中 $a=2, b=1, c=1, d=1$

举例 3:

```
int a = 1, b = 1, c = 1, d = 1;  
(a=0) || (b=2) && (c=2) || (d=0);
```

执行过程:

先结合, 加上小括号, $(a=0) || ((b=2) \&\& (c=2)) || (d=0)$ 从左到右计算

子表达式 $a=0$ 的结果是 0, 为假, 不发生短路, 执行子表达式 $((b=2) \&\& (c=2))$

子表达式 $((b=2) \&\& (c=2))$ 先执行 $b=2$ 结果是 2 为真, 不短路, 继续执行 $c=2$, 结果是 $2 \&\& 2$ 结果是真, 即 1, $(a=0) || ((b=2) \&\& (c=2))$ 为 $0 || 1$, 结果是真, 即 1

剩下 $1 || (d=0)$, 1 为真, 发生短路, $d=0$ 不执行了。整个表达式最终结果为 1。其中 $a=0, b=2, c=2, d=1$, d 没变

短路原理对代码优化的警示

真短路或：

(条件 1) || (条件 2)

为真概率大的条件放左边，可以减少两个条件同时执行的概率，从而增加程序执行效率。

数学角度分析：

条件 1 为真的概率是 70%，条件 2 为真的概率是 20%。

条件 1 在左，两个条件都执行判断的概率是 30%

条件 2 在左，两个条件都判断执行的概率是 80%

假短路与：

(条件 1) & & (条件 2)

为假概率大的条件放左边，可以减少两个条件同时执行的概率，从而增加程序执行效率。

数学角度分析：

条件 1 为假的概率是 70%，条件 2 为假的概率是 20%。

条件 1 在左，两个条件都执行判断的概率是 30%

条件 2 在左，两个条件都判断执行的概率是 80%

scanf_s 与 scanf

_s 版本的函数是 C11 标准的新函数，非_s 就是旧版 C 语言函数，新编译器建议大家使用新版本的_s 函数。

scanf 在高版本 vs 报错问题解决办法

- 1、声明宏：#define _CRT_SECURE_NO_WARNINGS
- 2、关掉 sdl：属性->c/c++->常规->sdl 检查
- 3、scanf_s

scanf_s 比 scanf 安全？

主要体现在越界问题中的处理措施

scanf 是在程序结束后报异常中断，不会立即中断

缺点 1，假设数万行代码，有数百个 scanf 输入，这时很难短时间内定位到越界点。

缺点 2，越界操作可能会侵害其他变量的空间，导致未知的莫名其妙的错误。

缺点 3，程序不结束，其问题可能会像蝴蝶效应，影响越来越离谱。

scanf_s 会执行失败，变量输入不成功，函数返回值为 0

优点 1，可能及时知道错误点，并及时做出相应处理

优点 2，不会导致越界问题，所以不会侵害其他变量空间

参数使用区别

```
strcpy_s(str, 20, "hello wrld"); //参数 2 表示参数 1 的字节数  
strcpy(str, "hello wrld");
```

strcpy_s 比 strcpy 安全?

主要体现在越界问题中的处理措施

strcpy 是在程序结束后报异常中断，不会立即中断，跟 scanf 越界情况 1 样

缺点 1，假设数万行代码，有数百个 strcpy 使用，这时很难短时间内定位到越界点。

缺点 2，越界操作可能会侵害其他变量的空间，导致未知的莫名其妙的错误。

缺点 3，程序不结束，其问题可能会像蝴蝶效应，影响越来越离谱。

strcpy_s 会立即中断，即函数调用位置中断，跟 scanf_s 处理方式不同

优点 1，立即知道错误点，并及时做出相应处理

优点 2，不会导致越界问题，因为直接停下了，不会往下运行了，所以不会侵害其他变量空间

1、缓冲区有很多：输入缓冲区，输出缓冲区，文件缓冲区，键盘缓冲区，输出双缓冲等等。有时候还叫 buf，爸福。都是一个东西。

2、缓冲区本质就是一段连续的空间，比如 `char a[40];`，a 就是一片连续的 40 字节的空间。

3、这个名字是根据它的功能来的，用来处理输入数据的，就叫输入缓冲区，用来处理输出数据的就叫输出缓冲区，宏观上取个名字，方便程序员分析使用。

4、缓冲具体是什么样的意义？

本身就是一种有效处理问题的算法，比如输入缓冲区的处理逻辑

1、键盘输入的各种字符，都会统一的一个不拉的存储在输入缓冲区中

2、输入完毕，scanf 等输入函数在缓冲区中拿数据，然后存入变量里

意义：

假设没有中间的缓冲区，咱们向一个 int 变量 a 中输入数据：scanf("%d", &a);

输入：123 这个数，依次摞的，然后如何存储这单个的数字？

1 先存入 a，a=1

然后存 2，此时 a 中应该是 12，所以存 2 时是加 11，a=12

然后存 3，此时 a 中应该是 123，所以存 3 时是加 111，a=123

这时发现输入错误，想输入 145，不是 123，那就得删除，还得减，惨不忍睹。

缓冲区来了，输入缓冲区是将所有的输入做字符串处理

输入 123，那就是输入的字符串 123，增删直接就是字符数组的增删了，回车表示输入结束，然后 scanf 根据格式说明符，将数据 123 转成整数，存一下就行了。

这种逻辑可以简单理解为：整合再分发，还有很多类似的逻辑应用，拓展理解应用

像是 windows 消息队列，将所有消息装进消息队列，然后从另一头往外一个个拿，分类处理。

像是利用堆栈遍历二叉树，这个堆栈可以理解为辅助缓冲区，作为二叉树节点的临时存储区，并辅以一定的出入逻辑。并不一定非得是连续的，如果逻辑结构是链式的也可以。

需要大家广开思路，学以致用。

缓冲区的主要作用：平衡输入端与计算端速度不一致

比如：

网络通信中的接收/发送缓冲区

发送一串数据到对端，计算执行发送也是一点儿一点儿的发送数据，代码中发出发送的指令，相应的文件或者数据就被存入发送缓冲区，然后计算机底层的硬件设备就开始从发送缓冲区拿数据一点儿一点儿的发送出去。而不是我们 send 函数直接就发了，send 函数的主要作用就是发出发送的指令。

接收消息缓冲区也是如此，对端发来若干数据，他是先进计算机底层的接收缓冲区，然后我们代码中 recv 函数，从接收缓冲区中拿到对端发来的数据。

所以如此，缓冲的这个意义就更加理解了，就像河流一样，有激流区，有缓流区。

输入缓冲区大小是：4096字节

输入缓冲区与写函数的逻辑关系

比如：

```
int a = 0;
double b = 0.0;
scanf("%d,%lf", &a, &b);
```

输入：12,43.5<回车>

缓冲区里装的就是：12,43.5\n

正常来说 windows 下摁回车产生的字符是\r\n，由于文本模式转换(文件操作)的原因，我们代码中得到的就是\n，所以就以\n 代表回车就行了。

取：

1、%d 是一取个整数，即一串连续的数字(比如 235rt7，就会取出 235)，取出 12，转成整数，存入变量 a，取了就没了

2、双引号里 “,” 分隔符，取出 1 个逗号

如果缓冲区里这个位置不是逗号，假设是#，那么这里就匹配不上，就不取了，scanf 执行结束，缓冲区里剩下后面的 #43.5\n。所以最终 a 输入成功，b 没有输入，还是初始值

3、取出一串连续的小数字符串，即 43.5，转成小数后存入变量 b

4、scanf 执行完毕，缓冲区还剩 1 个回车字符.\n，演示取出来

空格、回车作为 scanf 的默认分隔符，会被跳过，数量没有影响，其他字符不行

比如：

```
scanf(" %d \n\n %lf", &a, &b);
```

//输入时，正常用空格或者回车隔开数据即可，不能用别的字符，演示一下

键盘键入后，都是以字符形式存储在存在缓冲区，所以输入缓冲区里都是字符。在输入字符时，尤其注意此事，如下：

```
char a,c,e;  
a = getchar();  
c = getchar();  
e = getchar();
```

输入：q<回车>w<回车> 结果 a->q c->\n e->w 缓冲区内还剩下<回车>\n
即输入缓冲内字符的残留会影响字符的输入。

常见使用情景如下：

```
int a = 12;  
scanf("%d", &a);  
char c = 0;  
scanf("%c",&c);
```

输入：12<回车> 直接完事儿，a->12 c->\n 从而使得大家疑惑。

输入字符前清空输入缓冲区：

首先对于我们自己的输入情况，缓冲区里残留什么字符，我们自己心里要有数，根据我们实际输入的情况，选择适合的清空方式。

1、通过 getchar 函数将缓冲区搬空，无脑搬。

```
while ((c=getchar()) != '\n' && c != EOF); //getchar读取途中出错
```

即使缓冲区满了，最后也是\n，没读到 EOF，暂且写上。

有 bug，该写法遇到\n就停止，如果缓冲区残留 123\n\n456，则清完第一个\n就结束，残留\n456，该情况比较极端，一般正常输入完摞 1 个\n

2、清楚知道缓冲区残留 1 个\n，针对上面的常见使用情景

```
scanf(" %c", &c); // %c 前加个空格
```

或者

```
getchar(); //先拿出来
```

```
scanf("%c", &c);
```

可以完全使用 1 代替，只不过这样明确的情况，使用简单点儿，少写几个字母

```
while (1)  
{  
    char c = getchar();  
    if (c == '\n' || c == EOF)  
        break;  
}
```

3、跳过若干个字符

```
scanf("%*[^\\n]%c");
```

清除所有字符，遇到\n停下，然后%c清除\n，同 1

4、fflush(stdin) 刷新输入缓冲区，低版本编译器好使，高版本不好使了。所以不建议使用。

rewind(stdin); 清空stdin输入缓冲区

fseek(stdin,0,SEEK_SET); 可以

移动了输入缓冲区指针，缓冲区就没得了。

字节对齐，从形式上简单理解，比如：

char c ;

4 字节对齐，那么 c 就存储在 1 个 4 字节的单元中，占据首字节，其余 3 字节空闲。

2 字节对齐，那么 c 就存储在 1 个 2 字节的单元中，占据首字节，其余 1 字节空闲。

假设此时又有 int a；需要存储，那么 a 存储在 c 后第 4 字节处，不会占据 c 的 4 字节。

字节对齐数都是 2 的 n 次方。1,2,4,8,16 字节对齐

结构体大小计算方法：

1、整体对齐：以最大的类型的字节数为对齐字节数，成员按顺序填充

2、局部对齐：填充时，与前面已分配好的成员，最大字节对齐

3、结尾补齐：补齐最大字节数，最终为最大字节的整数呗

比如：

struct Node

{

char c;

int h;

double s;

void *p;

char str[10];

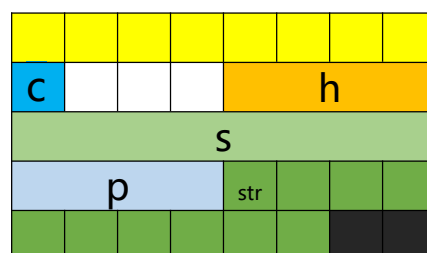
};

最大的类型是 double，8 字节，数组看元素类型

指针类型在 32 位编译环境下是 4 字节，在 64 位环境下是 8 字节

计算过程：

1、以 8 字节为模板，依次填充



8 字节模板

结尾补够 8 字节

强行设置字节对齐数：一般设置为 2 的 n 次方

`#pragma pack(1)`

1 就是 1 字节对齐，那么所有成员就是 1 个挨着 1 个

`#pragma pack( )`

什么都不写，系统默认的对齐字节，这根据成员最大的字节数决定，目的就是防止跨字节单元存储。

设置小了，有效果，比如最大字节数 8 字节，可以设置为 1,2,4，有效果

设置大了，没效果，比如最大字节数 8 字节，可以设置为 16,32，没效果

`#pragma pack()` 常见使用方式

一般将要被设置的结构体放在如下区间

`#pragma pack(1)`

`struct Node //此结构体按照 1 字节对齐`

```
{
    char c;
    int h;
    short t;
    double s;
```

```
};
```

`#pragma pack( )`

`struct Node2 //此结构体是默认字节对齐，此个是 8`

```
{
    char c;
    int h;
    short t;
    double s;
```

```
};
```

整数的存储：以整数的补码进行存储

正数的补码：是直接转的 2 进制，直接通过计算器转换

举例：

45 -> 10 1101

45 是 int 类型，补码补够 32 个位：0000 0000 0000 0000 0000 0000 0010 1101

45 是 short 类型，补码补够 16 个位：0000 0000 0010 1101

45 是 char 类型，补码补够 8 个位：0010 1101

负数的补码：

最高位是符号位，负数为 1，正数为 0

其余位叫数据位，按位取反，再加 1

举例

-45：假设-45 是 char 类型

最高位 1 表示负号

其余 7 位是数据位，45 的二进制是 10 1101，补够 7 位 010 1101

按位取反：010 1101 -> 101 0010

再加 1：101 0011

补上负号：1101 0011

假设-45 是 short 类型

最高位 1 表示负号

其余 15 位是数据位，45 的二进制是 10 1101，补够 15 位 000 0000 0010 1101

按位取反：000 0000 0010 1101 -> 111 1111 1101 0010

再加 1：111 1111 1101 0011

补上负号：1111 1111 1101 0011

32 位，64 位同理

验证 1：

```
char c = 0b11010011; //计算器没有补码
printf("%d", c);
```

验证 2：

程序猿计算器

验证 3：itoa 函数

```
char str[33] = { 0 };
_itoa_s(-45, str, 33, 2);
puts(str);
```

数据的高位叫高数据位

数据的低位叫低数据位

int a = 73170262; 0x045C7D56

0000 0100 0101 1100 0111 1101 0101 0110

高数据位

低数据位

高低是相对的

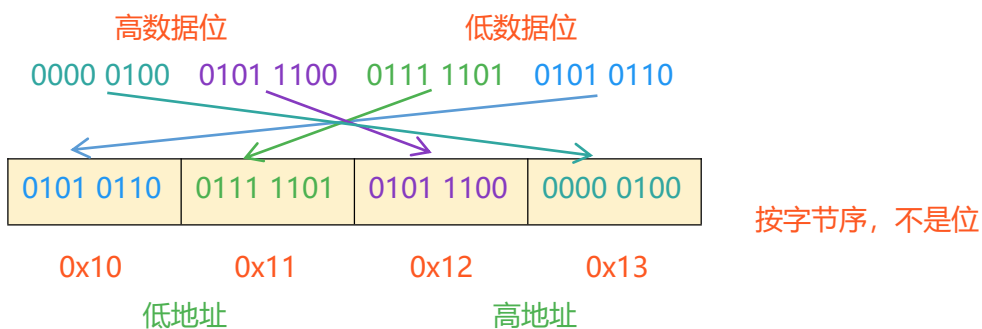
地址的大头叫高地址，高字节，高 8 位，高 16 位

地址的小头叫低地址，低字节，低 8 位，低 16 位

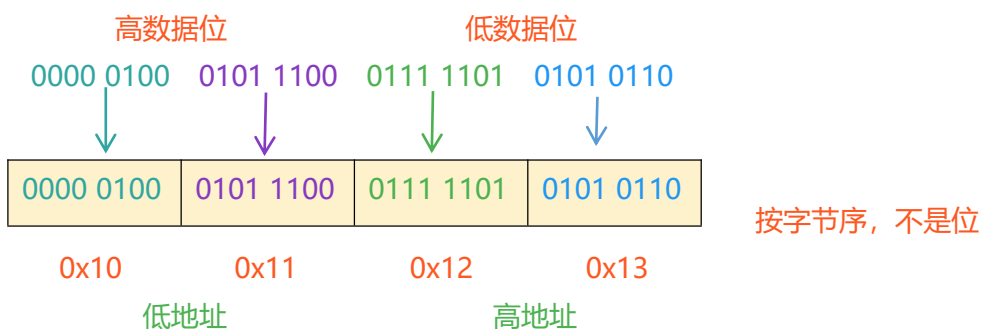
4 字节空间：



小端存储：高数据在高地址，低数据在低地址的字节序，高高低低



大端存储：高数据在低地址，低数据在高地址的字节序，高低低高



测试大小端存储 空间强转

```
int a = 0x045C7D56;
char* p = (char*)&a;
printf("%p, %x\n", p, p[0]);
printf("%p, %x\n", p+1, p[1]);
printf("%p, %x\n", p+2, p[2]);
printf("%p, %x\n", p+3, p[3]);
```

联合测大小端

```
union bm
{
    int a;
    char c[4];
};
union bm p = { 0x045C7D56 };
printf("%p, %x\n", &p.c[0], p.c[0]);
printf("%p, %x\n", &p.c[1], p.c[1]);
printf("%p, %x\n", &p.c[2], p.c[2]);
printf("%p, %x\n", &p.c[3], p.c[3]);
```

大小端转换，就是交换字节内容

```
int a = 0x045C7D56;
char* p = (char*)&a;

printf("%x\n", a);
char t = p[0];
p[0] = p[3];
p[3] = t;

t = p[1];
p[1] = p[2];
p[2] = t;

printf("%x\n", a);
```

unsigned char c = -12

-12 的二进制 1111 0100

负数的特点是 最高位是符号位，剩下的 7 位数是数据位

正数的特点是 8 个位都是数据位

1111 0100 以有符号输出就是-12，以无符号输出就是 244

-12 与 244 是该 1 字节 2 进制的两种表面形式而已，就像同一个数的 8 进制 10 进制 16 进制，仅仅是不同进制下的长相不同。

unsigned char c = 17707;

17707 0100 0101 0010 1011

c 仅仅 1 字节 8 个位，直接截取低 8 位数据 0010 1011，其他的直接没了，结果是 43
即越界存储，得到的就是固定字节数的低位二进制

整型的范围边界：

以 1 字节举例子，其他同理：

unsigned char : 0 ~ 255

0000 0000	0111 1111	1000 0000	1000 0001	1111 1111
0	~	127	128	129 ~ 255

char : -128 ~ 127

0	~	127	-128	-127 ~ -1
---	---	-----	------	-----------

0000 0000 0111 1111 1000 0000 1000 0001 1111 1111

1000 0000这个数，在可表示的范围内
但是究竟让他表示多少？

-127~-1 0~127 范围连续

-0？数学逻辑就说不过去

128？最高位是1，按照规则就是负数

-128？合适，所以就让这个表示-128

浮点型转 2 进制: 跟补码不一样, 不要以补码的思维平移过来

-1234.456

77.25

1、符号不用管, 留着就行

2、正数部分直接转 2 进制:

10011010010.

1001101.

3、小数部分转 2 进制: 乘 2 取整数部分, 直到小数乘尽

$0.456 * 2 == 0.912$	0	$0.25 * 2 == 0.50$	0
$0.912 * 2 == 1.824$	1	$0.50 * 2 == 1.00$	1
$0.824 * 2 == 1.648$	1	$0.00 * 2 == 0.00$	乘尽了
$0.648 * 2 == 1.296$	1	77.25 的二进制 1001101.01	
$0.296 * 2 == 0.592$	0		
$0.592 * 2 == 1.184$	1		
$0.184 * 2 == 0.368$	0		
$0.368 * 2 == 0.736$	0		
$0.736 * 2 == 1.472$	1		
$0.472 * 2....$	乘不尽		

-1234.456 的二进制-10011010010.011101001....

浮点型存储是以科学记数法的形式存储各个属性的, 1.xxxxxx

-10011010010.011101001.... $-1.0011010010011101001 * 2^{10}$

1001101.01 $+1.00110101 * 2^6$

有三个属性: 符号: 正负号、阶数: 2 的次方数、尾数: 那串 2 进制数

最高位符号位, 负数为 1, 正数为 0

4 字节 32 位: float

1b	8b	23b
----	----	-----

8 字节 64 位: double

1b	11b	52b
----	-----	-----

--	--	--

10、12、16 字节的位: long double

1b	1xb	xxb
----	-----	-----

中间段是阶数位 指数位: ieee754

阶数位三个特殊设定:

1、阶数全是 0, 尾数随意, 表示+0.0 或者-0.0, 无限趋近于 0.0 的数

所以浮点型可以除 0.0

```
float c = 0.0;
```

```
printf("%e", 2.0/c); //是合法的, 结果是无穷大, 输出 inf
```

整型不能除 0, 直接异常中断

```
float a = 1.2f;
```

```
*(int*)&a = 0x80076ab3;
```

```
printf("%f", a);
```

2、阶数全是 1, 尾数全是 0 表示无穷, 符号位是 1, 表示无穷小, 符号位是 0, 表示无穷大, 输出符号 INF, inf, 全称 Infinite

float:

1111 1111 1000 0000 0000 0000 0000 0000 无穷小

0xff800000

0111 1111 1000 0000 0000 0000 0000 0000 无穷大

0x7f800000

```
float f = 0.0f;
```

```
*(int*)&f = 0xff800000;
```

```
printf("%e", f); //输出 inf
```

double:

1111 1111 1111 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000

无穷小 0xffff000000000000

0111 1111 1111 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000

无穷大 0x7ff0000000000000

```
double f = 0.0;
```

```
*(long long*)&f = 0xffff000000000000;
```

```
printf("%lf", f); //输出 inf
```

```
float c = 0.0;
```

```
float d = 2.0 / c;
```

```
printf("%x", *(int*)&d); //7f800000
```

3、阶数全是 1, 尾数非全是 0, 表示非法数, 输出符号 NAN, nan, 全称 not a number

float:

1111 1111 1001 0000 0000 0000 0000 0000

0xff900000

0111 1111 1001 0000 0000 0000 0000 0000

0x7f900000

```
float f = 0.0f;
```

```
*(int*)&f = 0xff900000;
printf("%e", f);    //输出-nan
```

double:

1111 1111 1111 1000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000
 无穷小 0xfff8000000000000
 0111 1111 1111 1000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000
 无穷大 0x7ff8000000000000

```
double f = 0.0;
*(long long*)&f = 0xffff800000000000;
printf("%lf", f);    //输出-nan(ind)   indeterminate 不确定
```

非正经数, 比如-1 开平方

```
float f = sqrt(-1);
printf("%x, %lf", *(int*)&f, f);    //ffc00000   -nan(ind)
```

指数范围中

0000 0000 与 1111 1111

000 0000 0000 与 111 1111 1111

用作表示特殊情况

所以

32 位浮点型阶数的 2 进制范围是: 0000 0001 到 1111 1110 2 的 8 次方 - 1

对应的 2 的指数是 -126 ~ 127, 即指数的存储从结果是个看就是 +127 后的 2 进制。

比如 -115, 其二进制为 -115 + 127 == 12, 所以其二进制为 0000 1100。127 叫中位数。

本质是非标准移码存储的, 即标准移码, 再减 1。移码是补码的符号位取反。 偏置值

-1234.456

-10011010010.011101001.... $-1.0011010010011101001 \times 2^{10}$

77.25

1001101.01 $+1.00110101 \times 2^6$

对于这两个数来说, 存 float 里

前者阶数位是: 10 + 127, 即 137, 10001001, 加上符号位, 前九位: 1100 0100 1

前者阶数位是: 6 + 127, 即 133, 10000101, 加上符号位, 前九位: 0100 0010 1

验证一下:

```
float f = -1234.456f, t = 77.25f;
printf("%x %x", *(int*)&f, *(int*)&t);
c49a4e98: 1100 0100 1001 1010 0100 1110 1001 1000
429a8000: 0100 0010 1001 1010 1000 0000 0000 0000
```

64 位浮点型阶数的 2 进制范围是：000 0000 0001 到 111 1111 1110

对应指数 10 进制形式是 2 的 $-1022 \sim 1023$ ，中位数 1023，也叫偏置值

对于这两个数来说, 存 double 里, 中位数 1023

前者阶数位是：10+1023，即 1033，100 0000 1001

加上符号位, 前 12 位: 1100 0000 1001

前者阶数位是：6+1023，即 1029，100 0000 0101

加上符号位, 前 12 位: 0100 0000 0101

验证一下：

```
double f = -1234.456, t = 77.25;
```

```
printf("%llx %llx", *(long long*)&f, *(long long*)&t);
```

c09349d2f1a9fbe7:

1100 0000 1001 0011010010011101001011110001101010011111101111100111

4053500000000000:

0100 0000 0101 0011010100

80, 96, 128 位浮点型阶数的 2 进制范围是，不同的系统环境设计略有差别，根据其位数来即可。咱就不做讨论了。原理规则一模一样。

尾数位:

-1234.456

$$-10011010010.011101001\dots \quad -1.0011010010011101001 \cdot 2^{10}$$

77.25

$$1001101.01 + 1.00110101 * 2^6$$

因为总是 1.xxxxxx, 所以这个 1 默认就有, 即隐藏了, 不需要存储

前者存储的就是 0011010010011101001.....共 23 个, 实际逻辑存储 24 位。因为是无
限的, 算不尽, 所以此时只能存储 24 个位, 即不精确存储。

后者存储的就是 00110101，这个因为算的尽，并且在 23 个内，所以是精确存储的，也就是说，不是所有的浮点型都不精确存储，但是统一当不精确存储处理。后面补够 23 位，都是 0。

float 里:

c49a4e98: 1100 0100 1001 1010 0100 1110 1001 1000

429a8000: 0100 0010 1001 1010 1000 0000 0000 0000

double 里:

1100 0000 1001 001101001001110100101111000110101001111101111100111

0100 0000 0101 0011010100

所以说 double 的存储精度更高

double 与 float 的最大值最小值怎么计算?

float

正最大值: 0111 1111 0111 1111 1111 1111 1111 1111 0x7f7fffff

负最小值: 1111 1111 0111 1111 1111 1111 1111 1111 0xff7fffff

验证:

```
float f = 0.0;
*(int*)&f = 0x7f7fffff;
printf("%e", f);          //输出 3.402823e+38 转定义 float.h 查看对比
```

正最小值: 0000 0000 1000 0000 0000 0000 0000 0000 0x00800000

负最大值: 1000 0000 1000 0000 0000 0000 0000 0000 0x80800000

验证:

```
float f = 0.0;
*(int*)&f = 0x00800000;
printf("%e", f);          //输出 1.175494e-38 转定义 float.h 查看对比
```

double

正最大值:

0111 1111 1110 1111 1111 1111 1111 1111 1111 1111 1111 1111 1111 1111 1111 1111
0x7fefffffffffffffff

负最小值:

1111 1111 1110 1111 1111 1111 1111 1111 1111 1111 1111 1111 1111 1111 1111 1111
0xffefffffffffffffffff

验证:

```
double f = 0.0;
*(long long*)&f = 0x7fefffffffffffffff;
printf("%e", f);          //输出 1.797693e+308 转定义 float.h 查看对比
```

正最小值:

0000 0000 0001 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000
0x0010000000000000

负最大值:

1000 0000 0001 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000
0x8010000000000000

验证:

```
double f = 0.0;
*(long long*)&f = 0x0010000000000000;
printf("%e", f);          //输出 2.225074e-308 转定义 float.h 查看对比
```

浮点型存储分精确存储与不精确存储

精确存储的数：2 的 n 次方，以及多个加和。其二进制中的 1 的排序刚好在 24 位或者 53 位尾数装的下。没有多于截断，所以精确存储。此时精确表示其精度，对应 10 进制数多少都能表示出来，比如：

```
float a = pow(2, 64);  
printf("%.20f\n", a); //18446744073709551616.00000000000000000000  
精确的输出其 10 进制形式，很多位有效数位。
```

不精确存储的数：由于其二进制个数无限，存储有限，所以不精确。

对于 float 类型，其尾数位是 23 位+1 位隐藏位，总 24 位，从 2 进制角度来说就是 24 个有效 2 进制位。从 10 进制角度来说，不管这个浮点数多大或者多小，都是存其前 24 个位，所以这 24 个位可以表示的 10 进制数，是多少位？最大 24 个 1，共 8 位 10 进制数，所以其精度是 8 位，最后一位四舍五入，所以前 7 位是精确的。标准规定不小于 6 位

double 类型同理，53 个尾数位，其表示的最大 10 进制数是 16 位，标准规定不小于 10 位，类比 2 进制与 16 进制，16 个 2 进制位，就是 4 个 16 进制，5 个 10 进制，6 个 8 进制，。

综上两种情况，由于平时给个小数，咱不能直接口算出其是否是精确存储，所以统一模糊处理，认为都是不精确存储。

浮点型做条件精度写法 比较

```
#define N 0.0001 //调整 N，以调整比较精度  
double f = 5.6;  
if (f >= 5.6 - N && f <= 5.6 + N)
```

判断一个浮点型运算结果是不是正常：

```
double d = 0.0;  
double f = 5.6 / d;  
double g = sqrt(-1);  
printf("%e %d\n", f, isinf(f)); //是 inf 返回 1，正常返回 0 math.h  
printf("%e %d", g, isnan(g)); //是 nan 返回 1，正常返回 0 math.h
```

%a 输出浮点型的 16 进制形式

浮点型 16 进制表示法：0xa.23p5 (c99 p 记法)

16 进制转 10 进制

p5 就是 2 的 5 次方，即 32

a	10×32	320
0.2	$2 \times (1/16) \times 32$	4
0.03	$3 \times (1/256) \times 32$	0.375
	$(1/16^3) \times 32$	
	$(1/16^4) \times 32$	
	...	

$320 + 4 + 0.375 == 324.375$

16 进制转 2 进制

0xa.23p5

a 1010

2 0010

3 0011

合起来： 1010.00100011

p5 是乘 2 的 5 次方，小数点后移 5 位

终局： 101000100.011

2 进制转 16 进制

101000100.011

先不看小数点： 1010 0010 0011 x 2 的-3 次方

4 个 2 进制是 1 个 16 进制： a 2 3 -> a23

即： 0xa23p-3 0xa.23p5 -3+8==5

10 进制转 16 进制：可以先转成 2 进制，再转 16 进制

当然实践工作中不会让大伙儿口算笔算，直接用计算器就行了。

8421 码：16 进制与 2 进制之间的转换方法

4 位 2 进制数，刚好是一个 16 进制数

0000 ~ 1111 对应 0 ~ F

一个字节刚好可以用两个 16 进制数表示

二进制 4 个位的基数是 8421

8421

0101 4+1==5

1100 8+4==12 c

1011 8+2+1==11 b

举例

10 进制：1234

2 进制： 0100 1101 0010

16 进制：0x4d2

421 码：8 进制与 2 进制之间的转化方法

3 位 2 进制数，刚好是一个 8 进制数

000 ~ 111 对应 0 ~ 7

二进制 3 个位的基数是 421

421

101 4+1==5

100 4 == 4

011 2+1==3

举例

10 进制：1234

2 进制： 010 011 010 010

8 进制： 2322

16 进制：0x4d2

类型转换：将一个数据转换成另外一种类型使用，不改变原数据。

隐式类型转换：C 语言编译器帮我们做的默默转换

情况 1、赋值(=)转换：

(1)、初始化：赋值左侧为目标类型

`int a = 12.6;` //将 double 的 12.3 转成 int 的 12，然后赋值给 a

`short b[3] = { 3.4f, 4.5 };` //同上

(2)、赋值：转换为左操作数的类型

`a = 5.6f;` //将 float 的 5.6f 转成 int 的 5，然后赋值给 a

`b[2] = 5.6;` //同上

(3)、函数实参会被转换成形参类型

`void fun(int a, double b);`

`fun(3.4, 5);` //将 double 的 3.4 转成 int 的 5，然后赋值给 a

 //将 int 的 5 转成 double 的 5.0，然后赋值给 b

(4)、函数 return 值会被转成返回值类型

```
int fun(void) { return 12.6; } //将 double 的 12.6 转成 int 的 12
double a = fun(); //将 int 的 12 转成 double 的 12.0 赋值给 a
```

情况 2：函数参数提升，在无函数声明的情况下

1、有无符号的 char、short 提升转换成 int 类型，实测 vs 提升，dev 不提升

```
void fun(int c, ...)
{
    va_list ap;
    va_start(ap, c);
    size_t a = sizeof(va_arg(ap, char));
    printf("%zd ", a); //4
    a = sizeof(va_arg(ap, short));
    printf("%zd ", a); //4
    va_end(ap);
}

char c = 'c';
short f = 4;
fun(2, c, f);
```

2、在 short 与 int 同大小的情况下，unsigned short 提升为 unsigned int

3、float 提升为 double，实测 vs 与 dev 均不提升。书上说 k&r c。

情况 3：表达式类型提升

1、出现在表达式里的有无符号的 char,short，会被提升为 int。在 short 与 int 同大小的情况下，unsigned short 提升为 unsigned int。为什么？字节对齐。

```
char c = 'a';
short t = 12;
printf("%zd\n", sizeof(c+t)); //输出 4
printf("%p\n", c + t); //格式字符串中的 “%p” 与 “int” 类型的参数 1 冲突
```

2、在包含两种不同的数据类型的计算中，会被提升为级别高的类型。级别如下：

在 long 大于 int 大小时，long > unsigned int 比如 64 位 gcc 下

long double > double > float > unsigned long long > long long > unsigned long > long > unsigned int > int

在 long 与 int 同大小时，unsigned int > long 比如 vs 下

long double > double > float > unsigned long long > long long > unsigned long > unsigned int > long > int

计算过程的提升是按照子表达式，运算一步转换一步，不是统一先转换。比如：

```
1. 2f + 7 / 2 - 2.1
//先执行 7/2 == 3
//然后执行 1.2f+3，此时 3 提升为 float 3.0f，再加，得到 4.2f
//然后执行 4.2f - 2.1，此时 4.2f 提升为 double 4.2，再减，得到 2.1
```

而不是先转换：1.2 + 7.0 / 2.0 - 2.1 错误的

3、在表达式计算中，类型是提升的，类型提升不会出现计算不精确的问题。在赋值表达式中，会出现类型降级，降级会出现一定的问题，比如降低精确度。

情况 4：指针的隐式转换

1、数组地址到指针的转换

```
int a[10];
int(*p)[10] = &a; //标准写法
int* p = &a;      //虽然有警告，但是没有问题，&a 转成了 int*
```

2、函数名字的转换：函数名就是函数地址

使用函数名 fun 时，自动转成&fun

3、void*的转换：

```
int a;
void* p = &a; //&a 是 int*, 转换 void*, 再赋值给 p, void*可以直接装任意
float* p1 = p; //p 转成 float*, 再赋值给 p1, void*可以赋值给任意
int* p2 = malloc(4); //malloc 的返回值是 void*, 可以直接这么写
int* p3 = (int*)malloc(4); //有的编译器不支持上，所以写成如此，强制类型转换
```

显式类型转换(强制类型转换)：书上有叫 指派 cast

形式： (类型)数据

基本数据类型，比如：

```
int a = 3.4; //隐式类型转换
int b = (int)3.4; //强制类型转换，二者都可，平时用上面即可
```

指针类型，大指针不要操作小空间，比如：

```
int a = 3; //4 字节
long long* p = &a; //大指针指向小空间
*p = 45; // *p 操作 8 字节，a 只有 4 字节，越界了，异常
```

小指针可以操作大空间，比如：

```
long long a = 4; //8 字节
int* p = &a; //小指针指向大空间
*p = 4; //操作前 4 字节, p[0]
*(p + 1) = 5; //操作后四字节, p[1], 相当于把 8 字节当 2 元素数组了
printf("%llx", a); //输出看赋值效果
```

空间，不管是任何空间，他都是空间，没有区别，可以 malloc，可以数组，可以结构体等等，通过其首地址的转换，只要不越界，我们可以任意访问其每个字节。也可以把其作为任何东西来使用。

强制转换也是一种运算，不改变原空间，仅仅是使用数据做一个运算，得到一个结果：

```
float c = 3.4f;
int a = (int)c; //这里就是将 3.4f 这个数据计算成 3，赋值给 a，c 本身还是 3.4f 没有一点
```

儿影响。

//只有赋值才影响结果

c = (int)c; //那就是把 3 赋值给 c, c 变成 3 了

下表运算: a[2] 与 2[a]

下表运算的过程:

1、a+2 得到偏移后的地址, 即要求两个操作数, 一个是地址, 一个是偏移量

2、a[2]就是该地址起始的空间的名字

a+2 与 2+a 是一样的, 所以可以写成 a[2] 2[a]

其过程与*(a+2)一样:

1、a+2 得到偏移后的地址,

2、对该地址取*操作, 就是该空间的名字。

所以: a[2] == 2[a] == *(a+2) == *(2+a)

*(a+2)与 a[2]

相同点: 作用结果一模一样, 用哪个都行, 习惯哪个用哪个

区别: 前者是 2 个运算符构成的表达式, 后者是一个运算符构成的表达式

```
p[2] = 4;
00007FF753514010  mov     eax, 4
00007FF753514015  imul    rax, rax, 2
00007FF753514019  mov     dword ptr p[rax], 4
    *(p + 2) = 5;
00007FF753514021  mov     dword ptr [rbp+10h], 5
```

复杂指针的解释: 按照运算符的优先级与结合性, 一步一步分析

以下定义中 pf 是什么?

int (*pf)(int)

第一步: (*pf) 先跟*结合, 所以 pf 是一个指针

第二步: int (int) 该指针的类型为该类型函数的地址

第三步: 这个函数的类型是: 返回值是 int 有一个 int 参数

```
int fun(int a) { return 0; }
```

```
int (*pf)(int) = &fun; //如果分析有误, 放编译器里就警告了
```

char *(*pf[3])(char *p)

第一步: pf[3]先跟方括号结合, 所以 pf 是一个 3 个元素数组

第二步: (*) 该数组元素都是是指针类型

第三步: char* (char *p) 元素指针是指向该类型函数的地址

第四步: 该函数类型是: 返回值为 char* 参数为 char*

```
char* fun1(char* p) { return NULL; }
char* fun2(char* p) { return NULL; }
char* fun3(char* p) { return NULL; }
char* (*pf[3])(char* p) = { fun1, fun2, fun3 };//如果分析有误，放编译器里就警告了
```

int (*pf)(int)(float)

第一步: **(*pf)**先跟*结合, 所以 **pf** 是一个指针

第二步: **(*(int))** 该指针指向该类型的函数

第三步: 该函数参数是一个 **int**, 返回值是一个指针 *

第四步: **int (float)** 表示第三步返回值指针指向该类型的函数

第五步: 该函数返回值是 **int**, 参数是一个 **float**

```
int fun(float a) { return 0; }
int (* fun2(int b))(float)
{
    return fun;
}
int ((*pf)(int))(float) = fun2;
```

int *(*pf(int))[3]

第一步: **pf(int)** 先跟小括号结合, 所以 **pf** 是一个函数

第二步: 函数参数有一个 **int** 参数, 返回值是一个指针*

第三步: **int* [3]** 该返回值指针指向 3 元素的指针数组的地址

```
int* (*pf(int a))[3]
{
    int* c[3];
    return &c;
}
```

int ((*pf(int *))[3])(int *)

第一步: **(*pf)**先跟*结合, **pf** 是一个指针

第二步: **(*(int *))**该指针指向函数类型

第三步: 该函数参数是一个 **int***, 返回值是一个指针 *

第四步: **(*[3])**该返回值指针指向 3 个元素的指针数组

第五步: **int (int *)**指针数组每个元素该函数类型指针

第六步: 该函数一个 **int***参数, 返回值为 **int** 类型

```
int fun1(int* p) { return 0; }
int fun2(int* p) { return 0; }
int fun3(int* p) { return 0; }
int (* pp[3])(int*) = {fun1, fun2, fun3};
int ((*fun(int* p))[3])(int*) { return &pp; }
int ((*pf)(int*))[3])(int*) = &fun;
```


内存分区总结

分区大概分 5 个区，有的资料分 6 个的，我这说下这 5 个区域。

栈区：局部变量

生命周期：所在大括号

作用域：所在大括号

空间特点：定义变量时系统申请空间，声明周期结束时系统检测释放

大小：默认 1M

修改大小：项目属性->连接器->系统->堆栈保留大小 改大，单位字节

堆区：malloc calloc 的空间

生命周期：从 malloc 到 free

当程序结束时，我们忘记 free 了，系统会自动回收空间，最好忘了它。

作用域：整个工程，只需地址传递

空间特点：malloc 申请，我们自己 free 释放。

无需系统额外的资源帮助我们管理释放

大小：默认很大，理论上可用的物理内存

修改大小：项目属性->连接器->系统->堆保留大小 改大，单位字节

静态存储区：全局变量，静态变量

生命周期：与程序共存亡

当程序结束时，释放。由于程序运行期间一直占用空间，所以不建议使用大空间的全局变量

作用域：整个工程

空间特点：自动初始化 0，系统申请，系统释放，无需像栈区一直检测

大小：默认很大，理论上可用的物理内存

代码区：存储每一行代码，函数调用就是跳到这里来执行代码

字符常量区：常量字符串，随叫随有。

命令行参数

命令行参数用来传递文件路径给软件，比如双击 stu.txt，系统就会自动用文本软件打开该文档，本质就是系统通过文件后缀得知用什么软件打开，然后将文件的绝对路径传递给软件，软件内通过 fopen 打开了 stu.txt，这就是用到了命令行参数。

另外拖动文件到软件，或者选择打开方式，或者通过控制台指令，虽然操作不一样，但是本质一样。

演示通过命令行传递参数：cmd->exe 的绝对路径(一拖一拉)->空格->文件的绝对路径

命令行主函数形式：

```
int main(int argc, char* argv[])
{
    return 0;
}
```

参数介绍：主函数的两个参数，

参数 1 是命令行参数的个数，

参数 2 是字符串数组，装命令行参数的，本质就是字符串，也可写成 `char**argv`
参数使用：

通过命令行传递，比如：`data.exe da1.txt da2.txt da3.txt`

`argc` 为 4

`argv[0]` 为 `data.exe`

`argv[1]` 为 `da1.txt`

`argv[2]` 为 `da2.txt`

`argv[3]` 为 `da3.txt`

调试参数传递：项目属性->调试->命令行参数

此时不用写 `exe` 文件了，默认就有，直接写文件 `da1.txt da2.txt da3.txt`，空格隔开。
用法同上，一模一样。

scanf 的返回值：成功写入变量的个数

```
int b;
double c;
int a = scanf_s("%d%lf", &b, &c);
//输入：12 34.5<回车> a==2
//输入：12 s<回车> a==1 输入字符 s 错误，不匹配%lf
//输入：w 34.5<回车> a==0 输入字符 w 错误，不匹配%d，有错误直接中断输入
```

printf 的返回值：成功输出的字节数

```
int a = printf("hello world\n");
//hello world a==12, 算上\n, 不算\0
int b = printf("hello %d,%lf\n", 123, 34.5);
//hello 123,34.500000 b==20, 算上\n, 不算\0
int c = printf("hello %d,%.2lf\n", 12, 34.5);
//hello 123,34.50 c==16, 算上\n, 不算\0
```

scanf 返回值应用

大家经常在写案例时需求：输入成绩，正常录入，输入错误字符，直接退出循环

原理：输入数据与变量类型不同，直接中断输入，并返回数值

```
int score = 0, sum = 0;
while (0 != scanf("%d", &score)) //输入字符就会退出循环
{
    sum += score;
}
//清空缓冲区，因为上边输入字符，还在残留
rewind(stdin);
//输入：1 2 3 4 5 6 q sum==21
```

输出格式控制：

格式控制：%8.3lf 浮点型

```
printf("%8.2lf", 12.34); //右对齐
```

输出结果：<空格><空格><空格>12.34

8：表示 12.34 的输出占控制台 8 个字符位，默认右对齐，左侧补空格。凑齐 8 个字符

.：表示小数点

2：表示 2 位小数

格式控制：%+8.3lf 浮点型

```
printf("%+8.2lf", 12.34); //显示符号
```

输出结果：<空格><空格>+12.34

8：表示 12.34 的输出占控制台 8 个字符位，默认右对齐，左侧补 2 个空格，凑齐 8 位

+：显示符号，正号

.：表示小数点

2：表示 2 位小数

格式控制：%+08.3lf %08.3lf 浮点型

```
printf("%*+08.2lf*\n", 12.34); //右对齐
```

```
printf("%*08.2lf*\n", 12.34); //右对齐
```

输出结果：+0012.34 00012.34

8：表示 12.34 的输出占控制台 8 个字符位，默认右对齐

+：显示符号，正号

0：默认补空格，0 就是补 0

.：表示小数点

2：表示 2 位小数

格式控制：% -8.3lf 浮点型

```
printf("%-8.2lf", 12.34); //左对齐
```

输出结果：12.34<空格><空格><空格>

8：表示 12.34 的输出占控制台 8 个字符位，-负号指示左对齐，右侧补空格，凑齐 8 位

-：左对齐

.：表示小数点

2：表示 2 位小数

格式控制：%+-8.3lf 浮点型

```
printf("%+-8.2lf", 12.34); //显示符号，左对齐
```

输出结果：+12.34<空格><空格>

8：表示 12.34 的输出占控制台 8 个字符位，左对齐，右侧补 2 个空格，凑齐 8 位

+：显示符号，正号

-：左对齐

.：表示小数点

2：表示 2 位小数

格式控制: %6d 整型

```
printf("%6d", 12);
```

输出结果: <空格><空格><空格><空格>12

6 : 表示 12 的输出占控制台 6 个字符位, 右对齐, 左侧补 4 个空格, 凑齐 6 位

格式控制: %+6d 整型

```
printf("%+6d", 12);
```

输出结果: <空格><空格><空格>+12

6 : 表示 12 的输出占控制台 6 个字符位, 右对齐

+ : 表示显示符号, 左补 3 个空格, 凑够 6 位

格式控制: %+06d 整型

```
printf("%+06d", 12);
```

输出结果: +00012

6 : 表示 12 的输出占控制台 6 个字符位, 右对齐

+ : 表示显示符号

0 : 表示用 0 补够位数, 不用空格了

格式控制: %-6d 整型

```
printf("%-6d", 12);
```

输出结果: 12<空格><空格><空格><空格>

6 : 表示 12 的输出占控制台 6 个字符位

- : 表示左对齐

格式控制: %+.6d 整型

```
printf("%+.6d", 12);
```

输出结果: +000012

.6 : 表示输出 6 个数字位, 不够用 0 补够, 不算符号位

+ : 表示显示符号, 总共显示 7 位

输入格式控制

格式控制: %2d 整型

```
int a = 0;
```

```
scanf_s("%2d", &a);
```

输入: 123456<回车> 结果: a==12

2 : 表示将 2 的字符宽度的数字组合成整数, 所以转换了 12

格式控制: %2lf 浮点型

```
double a = 0;
```

```
scanf_s("%5lf", &a);
```

输入: 12.3456<回车> 结果: a==12.34

5 : 表示将 5 个字符宽度的数字组合成浮点数, 所以转换了 12.34

输入没有%.4lf 的小数点控制形式

格式控制: `scanf_s("%*2d", &a);` 跳过

```
int a = 0;
scanf_s("%*2d%2d", &a);
```

输入: 123456<回车> 结果: a==34

`%*2d` : 表示跳过两个数字字符, 即*就是跳, 所以 a 装的 34

`%*d` : 表示跳过一段连续的数字字符

`%*lf` : 表示跳过一段连续的浮点数字符

`%*c` : 表示跳过 1 个字符

格式控制: `scanf_s("%*[^\\n]", &a);` 表示跳过所有字符, 直到遇见\\n 字符

```
int a = 0;
scanf_s("%*[^3]%d", &a);
```

输入: 123456<回车> 结果: a==3456

`%*[^3]` : 跳, 遇到 3 停止, 然后%d 从 3 处读入整数

类型修饰符:

有三个: `const`(常量) `volatile`(易变) `restrict`(唯一)

`volatile` : 易变的, 影响编译器优化

如果一个变量 a, 在程序中使用的频率很高, 那么就会被优化存储入缓存, 这样可以增加程序的执行效率, 当声明成 `volatile` 变量, 该变量就是易变的, 表示在程序运行过程中会经常发生改变, 所以就不能被优化存入缓存了。

形式:

```
struct Node { int a; double b; };
volatile int a = 12;           //a 是易变的
volatile int b[5] = {1, 2, 3, 4, 5}; //每个元素都是易变的
volatile struct Node c = { 12, 3.5 }; //每个成员都是易变的
```

`restrict` 修饰指针变量, 表示该变量是指向空间唯一、初始的, 即该空间仅与此变量关联, 一一对应。

修饰变量:

C 语言里 : `restrict`(老), `__restrict`(C99)
C++里 : `__restrict`

修饰函数:

`__declspec(restrict)`

为了通用性, 大家使用下划线版本, 仅能修饰变量

```
int* __restrict p1 = (int*)malloc(20);
__declspec(restrict) void fun(void) { } //看看头文件
```

它的作用是允许编译器对所修饰的变量进行优化, 如下:

```
int* __restrict p = (int*)malloc(20);
p[0] = 5;
p[0] += 6;
p[0] += 3;
free(p);
```

由于 p 是该空间的唯一的访问方式，所以会优化成 p[0] += 9;

```
int* __restrict p = (int*)malloc(20);
p[0] = 5;
p[0] += 9;
free(p);
```

转义字符： \

转义字符就是将字符原本的含义进行转换，比如\n，n 本身就是字母，\n 就是换行符了，把 n 原本的含义转换了。

C 语言支持的转义后字符，也叫控制字符：

0	NUL	\0	字符串结尾
7	BEL	\a	响铃
8	BS	\b	退1格
9	HT	\t	tab 水平制表符
10	LF	\n	换行
11	VT	\v	垂直制表符
12	FF	\f	换页
13	CR	\r	光标到首

其他的字符也是转义字符，但是 C 语言没有定义其含义，所以其无意义

'\c' '\x' '\y'

特殊：想要输出如下字符，一般加上转义字符

```
printf("\\ \\' \\' ");
```

另外：想输出%，写%%，这个不是转义字符

```
printf("%%");
```

数字转义：

\+数字： '\000' '\123'，该数字默认是 8 进制，其范围是 '\000' - '\377'，对应 10 进制的 0-255，刚好 1 字节，转义字符都是 1 字节

\+x+数字： '\xa3' '\x12'，该数字是 16 进制，其范围是 '\x00' - '\xff'，对应 10 进制的 0-255，刚好 1 字节，转义字符都是 1 字节

单引号中放多个字符：

单引号里理论上只能放 1 个字符，放多个字符叫字符串，要用双引号。但是一些编译器对单引号里放多个字符也进行了相应处理，但是由于此行为是 C 语言的未定义行为，属于编译器行为，所以该行为可能是不通用的。

咱们把 VS 下的处理方式教给大家。

```
char a = 'abcd';
```

a 里装的 d, 默认最后一个字符，原因如下；

字符的本质就是 int 整数，4 个字节，每个字符是 1 字节，所以单引号内最多放四个字符。多于 4 个就会报错。所以

```
'abcd'
```

四个字节： 97 98 99 100

01100001 01100010 01100011 01100100

小端存储：数据的低位存在内存的低位

100 99 98 97

0x10 0x11 0x12 0x13

a = 四字节空间，只能装一个字节，就是 0x10 这个字节的数据，即 100

位域，位字段

位的操作可以使用位运算符，第二种方式就是位字段，也叫位域。位运算符可以实现任何操作，而且操作灵活简单快捷，所以一般大家看资料就很少详细介绍位域的。甚至平时看一些库，也是位运算使用居多，位域很少见到。

书上说，由于存储模式取决于系统，所以位域的移植性会有问题，即在当前环境下是 123，代码原模原样移到另一个环境，就变成 789 了，程序运行出现二义性，造成不可预知的错误，即移植性很差。

形式：

```
struct hh
{
    unsigned int a : 1;
    unsigned int b : 1;
    unsigned int c : 1;
    unsigned int d : 1;
    unsigned int e : 1;
    unsigned int f : 1;
    unsigned int g : 1;
    unsigned int h : 1;
}bit;
```

形式跟结构体差不多

- 1、成员类型用 unsigned int，或者 signed int，C99 后可以用 bool 别的不行
- 2、a b c d e f g h 叫做标签，跟 switch 那个标签意思一样
- 3、1 表示 1 位，即 a 这个标签代表 1 个 2 进制位，该标签只能赋值 1, 0
- 4、连续的 8 个位，即 1 字节的空间数据，可以 bit.b=0 操作专门对 1 个位进行读写。
- 5、该 bit 变量本质是 unsigned int 类型变量，上 8 位是 4 字节中的低八位

```

struct hh c = {0};
c.a = 1;
c.b = 0;
c.c = 1;
printf("%x", *(int*)&c); //输出 5

```

也可以用 1 个标签代表多个位, 如下:

```

struct hhh
{
    unsigned int a : 1;
    unsigned int b : 3;
    unsigned int c : 5;
    unsigned int d : 4;
}bit;

```

a 字段赋值: 0 - 1

b 字段赋值: 000 - 111

c 字段赋值: 00000 - 11111

d 字段赋值: 0000 - 1111

总字段数是 13 位, 即 unsigned int 空间的低 13 位

位字段对齐

不允许一个字段跨两个 unsigned int 边界, 第二个字段会自动移到下一个 unsigned int, 相当于 4 字节对齐

```

struct hhhh
{
    unsigned int a : 24;
    unsigned int b : 4;
    unsigned int c : 16;
    unsigned int d : 24;
}bit;

```

a b 在第 1 个 unsigned int 的 4 字节

c 在第 2 个 unsigned int 的 4 字节

d 在第 3 个 unsigned int 的 4 字节

所以 bit 总字段是 12 字节 96 个位

位字段控制:

```

struct hhhh
{
    unsigned int a : 1;
    unsigned int   : 3; //无名 3 位字段, 空闲
    unsigned int   : 0; //0 无名, 将 d 字段推到下一个四字节
    unsigned int d : 4;
};

```


预处理：在程序预处理阶段进行执行

利用宏创建字符串：#

```
#define X(a) #a
printf(X(qwerty));
```

参数 a 是 qwerty，#将这个参数 qwerty 变成了字符串"qwerty"

宏拼接：##

```
#define X(a) #a##"asdf"
printf(X(qwerty));          //输出：qwertyasdf

#define Y(a,b) #a###b
printf(Y(qwerty,asdf));    //输出：qwertyasdf
```

可变参数宏

```
#define PRINT(...) printf(__VA_ARGS__)
PRINT("%d,%lf", 12, 34.5);
```

...表示参数可变，__VA_ARGS__ 用来获取可变参数，即传递的所有东西，直接替换该宏处

```
#define PRINT(...) printf("%d,%lf\n", __VA_ARGS__)
PRINT(12, 34.5);
```

取消宏定义：#undef

```
#define X 123
#undef X //取消 X，后面不能用了
```

条件编译：#if #else #ifdef #ifndef #endif

条件编译影响代码的有无：

```
#if 0          //假
int b = 0;
#elif 1//真
int a = 0;
#else
int c = 4;
#endif        //结尾，必须要有

b = 2;    //未定义错误
a = 2;    //正常
c = 2;    //未定义错误
```

#if 0 也常用于全文件注释

#ifdef //如果定义了

#endif //结尾

#ifndef __H_ //如果定义了宏__H_ 则该代码有效

int a = 12;

```
#endif      //结尾
a = 2;      //未声明标识符
```

对应

```
#define _H_ //定义宏
#ifdef _H_  //如果定义了宏_H_ 则该代码有效
int a = 12;
#endif     //结尾
a = 2;     //可以正常使用
```

ifndef 如果没有定义宏
#endif 结尾

```
#ifndef _H_ //如果定义了宏_H_ 则该代码有效
int a = 12;
#endif     //结尾
a = 2;     //可以正常使用
```

```
#define _H_ //定义宏
#ifndef _H_  //如果定义了宏_H_ 则该代码有效
int a = 12;
#endif     //结尾
a = 2;     //未声明标识符
```

预定义宏名：C 语言定义的一些具有特定功能的宏

```
printf(__TIME__); //当前时间 15:04:36
printf(__DATE__); //当前日期 May 22 2022
printf(__FILE__); //当前源文件的完整路径
printf(__LINE__); //当前宏所在行号

__STDC_VERSION__ //当前 C 语言标准版本，工程属性->C 语言版本 老旧版本不支持
199409L(C95) 201112L(C11) 201112L(C11) 201710L(C17)
```

__STDC__ //如果当前有它并指定为 1，说明是符合 C 标准，但是没有这个东西

__STDC_HOSTED__ //如果当前环境有它并为 1，说明支持完整的标准库

当然这几个是有一定意义的学习标准库里有数千个预定义宏，常量宏，那个就没什么意思

#line：用来重置 **__LINE__** **__FILE__**

```
#line 5 "text.t" 但是尽量不要改，改了行号就乱了，调试功能就不能用了
该行的下一行就是第 5 行，依次往下查
```

#error 报错，发出错误信息

```
#if 1
#error not hello //后边是错误描述
#endif
```

编译报错：fatal error C1189: #error: not hello

```
#if 0
#error not hello    //后边是错误描述
#endif
这行代码无效，就不报错了
```

#pragma 预处理指令，该指令部分不是通用的，不同的编译器之间移植性不一，并且不同的 C 语言环境有很多各自的用法，大家使用的时候如果涉及，就查具体文档即可。咱们这讲解几个常见的，比较常用的。

#pragma once 防止头文件重复包含

```
#ifndef _H_
#define _H_
#endif
```

#pragma pack(4) 设置内存字节对齐数

```
#pragma pack(4)      将当前对齐设置为值 4。该数 1, 2, 4, 8, 16
#pragma pack()        将当前对齐设置为默认值
#pragma pack(push)    将当前对齐的值存储
#pragma pack(push, 4) 将当前对齐的值存储，然后再将当前对齐设置为 4
#pragma pack(pop)     取出栈顶存的对齐数，并设置为当前对齐数
```

#pragma message("hello world ") 编译时显示双引号的内容

#pragma comment(lib, 库名)

```
#pragma comment(lib, "Winmm.lib")    #include <Mmsystem.h>
#pragma comment(lib, "Ws2_32.lib")   #include <winsock2.h>
```

#pragma warning

```
#pragma warning(disable: 4996 4477) //编译信息窗口，不显示 4996 和 4477 号警告
#pragma warning(once: 4996 )       //编译信息窗口，4996 号警告仅显示 1 次
#pragma warning(error: 4996)        //编译信息窗口，把 4996 号警告提升为 error
```

#pragma section 设置其下变量的使用权限

```
#pragma section("mycode", read)
__declspec(allocate("mycode"))
int i = 0;    //变量 i 只能 read 操作

#pragma section("mycode", read, write)
__declspec(allocate("mycode"))
int a = 0;    //变量 a 可读可写
```

#pragma push_macro(" macro-name ") 以栈结构存储宏名

```
#define X 1
#pragma push_macro("X")    //入栈
```

```
#pragma pop_macro("X")    //出栈
```

#pragma region 设置编译器内代码折叠

```
#pragma region  
    code //该部分可以折叠  
#pragma endregion
```

#pragma data_seg 共享数据段

```
#pragma data_seg("data")  
    int a = 12;  
#pragma endregion
```

应用在动态链接库里，咱们在高级应用里讲这个东西。设置成共享数据，那么 a 这个变量就可以在多进程间共享使用，实现进程间通信。

宏拼接: \

一个宏名仅是其同一行的名字，跟别的行没有关系了

如下：

```
#define PRINT    int i;
                for (i = 0; i < 5; i++)
                {
                    printf("%d ", i);
                }
```

我想让 PRINT 代替整个 5 行代码，其做不到，其本体就是 int i;，因为宏只把同一行的代码作为本体，所以可以写到一行，如下：

```
#define PRINT    int i;for (i = 0; i < 5; i++){ printf("%d ", i); }
```

如此便可

但是，这样写法代码完全看不出结构，代码一多，那就瞎了，这就有了宏拼接符：\

代码如下：

```
#define PRINT    int i;\
                for (i = 0; i < 5; i+)\
                {\
                    printf("%d ", i);\
                }
```

他的意义就是告诉编译器上一行与下一行是链接的，紧挨着的，仅此而已

内容参考: <https://en.cppreference.com/w/>

MSDN: <https://docs.microsoft.com/en-US/search/?terms=wsastartup>

库函数: C 语言的一部分, 在标准 C 语言使用场景下是随时可以使用的。

库函数学习思路: 了解每个头文件的主要功能, 以及主要有哪些函数。不用每个函数都试一遍, 知道功能即可, 一是太多了, 二是 90%都用不到, 三是一类一类的。

1、<stdarg.h> 可变参数头文件

转到头文件带大家看下。

```
#define va_start __crt_va_start
#define va_arg __crt_va_arg
#define va_end __crt_va_end
#define va_copy(destination, source) ((destination) = (source))

#include <stdarg.h>
void fun(int c, ...) //函数头, 参数 1 指示参数个数, 后面用三个点, 表示可变
{
    va_list ap, ad; //定义参数装填结构
    va_start(ap, c); //将 c 个参数装进 ap, 不算 c, 只算后面不确定的
    va_copy(ad, ap); //将 ap 又复制了一份
    char a = va_arg(ap, char); //第一个
    short b = va_arg(ap, short); //第二个 依次自动拿
    va_end(ap); //结束, 释放 ap
}

int main(void)
{
    char c = 'c';
    short f = 4;
    fun(2, c, f);

    return 0;
}
```

printf, scanf 系列的函数都是可变参数, 其参数 1 是字符串, 其内的格式说明符个数用来指示不确定参数个数。

2、<stdbool.h> (C99): bool 类型的定义

布尔类型只有两个逻辑值, 真与假, 输出 1/0, 常用做函数的返回值类型。

C 语言的布尔类型是 _Bool。C++ 中的布尔是 bool, 为了保证二者的兼容以及移植性, C99 标准新增了该头文件, 对 _Bool 新命名为 bool, 并增加 true 与 false 为 1,0 的名字。

转进去看看, 如下:

```
#define bool _Bool
```

```
#define false 0
#define true 1
```

我们也可以不使用这个头文件，在自己的代码里，自己写上这三个宏，一样的作用。

3、<time.h>：时间、日期类函数

转定义看头文件，里边主要是类型的声明，宏，函数

经验：

头文件里有一些以下划线起始的函数，有非下划线起始的函数，非下划线起始的函数内部调用下划线起始的函数，比如 ctime 这个函数，内部调用 _ctime 函数，演示下。

我们平时用的函数是 ctime，不带下划线的。下划线函数，我的理解，是编译器厂商实现的函数，因为 ctime 的功能，参数都是标准固定的，通用的，但是该函数的具体实现不同的系统理论上是不一样的，所以厂商做了这层调用，如果某一天操作系统重新优化，厂商直接修改 _ctime 这个函数即可，ctime 没有一点儿影响，当然这些头文件里的都能调用，下不下划线的都能使用。

就像是汽车，方向盘，换挡，刹车控制汽车，换了发动机，变速箱，还得是方向盘，换挡，刹车控制汽车而且不能换位置，换花样。

time()：获取当前系统时间

函数原型: `time_t time(time_t* const _Time)`

`time_t`: `typedef __int64 time_t;` 64 位的整形

返回值：返回系统日历时间，参数是时间传址调用，失败返回-1。该时间是一个计数，从纪元元年(1970 年 1 月 1 日)开始,0,1,2,3...(秒)一直至今,早期计算机是 32 位的时间,到 2038 年就算尽了，所以现在的时间都是 64 位的了，这个要 2900 亿年，所以不愁了。

```
time_t tt;
time_t t = time(&tt); //tt t 都是装的时间的计数，
time_t t = time(NULL); //参数可以 NULL，只用返回值返回，方便
```

ctime()：将 time_t 转成日历时间的文本显示

函数原型: `char* ctime(time_t const* const _Time)`

`#define _CRT_SECURE_NO_WARNINGS` //老版加宏

`#include <time.h>`

```
time_t tt;
time_t t = time(&tt);
char *s = ctime(&tt);
puts(s); //Tue May 10 11:37:29 2022
```

ctime_s()(C11):同上

函数原型: `errno_t ctime_s(char* const Buf, size_t const Size, time_t const* const Time)`

老版通过函数返回值返回，新版通过参数返回。

返回值：错误码。成功返回 0。

参数：1 是装字符串的字符数组，2 是字节数，3 是日历时间。

ctime_r()(C23) //vs2022 还不支持它，只是支持部分 23 标准

函数原型：char* ctime_r(const time_t * timer, char* buf);

返回值：日历时间字符串首地址，失败返回 NULL

参数：1 日历时间点，2 装字符串的字符数组，更像是前两者的结合版，方便。

difftime():计算两个时间点的时间差，以秒为单位

函数原型：double difftime(time_t const _Time1, time_t const _Time2)

clock():用来获取当前程序占用了 CPU 多长时间，非精确。

函数原型：clock_t clock(void);

返回值：clock_t typedef long clock_t; 毫秒

该值除以宏：CLOCKS_PER_SEC ((clock_t)1000)，得到的是秒，即 1 秒==1000 毫秒

timespec_get(C11): 返回基于给定时间基底的日历时间，各国家地区的时间记录是不一样的，比如咱们国家此时中午 12:00，某个国家的时间是早晨 8:00，那么这个参数 2 就是时间基底，填对应的，那么得到的对应国家的此时时间。国际时间的基底是 1。

函数原型：int timespec_get(struct timespec* const _Ts, int const _Base)

返回值：成功参数 2，失败返回 0

参数 1：时间结构体，精确到纳秒

1,000,000,000 纳秒 = 1000,000 微秒 = 1000 毫秒 = 1 秒

```
struct timespec
{
    time_t tv_sec; // 秒
    long tv_nsec; // 纳秒
}; //头文件看下这个结构体
```

参数 2：时间基底

#define TIME_UTC 1 国际时间基底，获取北京时间直接 time 函数

timespec_getres(C23): 目前 vs2022 还不支持。同上

函数原型：int timespec_getres(struct timespec *ts, int base);

localtime() :将 time_t 的时间转换成 struct tm 类型的时间。

```
struct tm
{
    int tm_sec; // seconds after the minute - [0, 60] including leap second
    int tm_min; // minutes after the hour - [0, 59]
    int tm_hour; // hours since midnight - [0, 23]
    int tm_mday; // day of the month - [1, 31]
    int tm_mon; // months since January - [0, 11]
```



```

    int tm_year; // years since 1900
    int tm_wday; // days since Sunday - [0, 6]
    int tm_yday; // days since January 1 - [0, 365]
    int tm_isdst; // daylight savings time flag
};

```

函数原型: `struct tm* localtime(time_t const* const _Time)`

`localtime_s()(C11)`: 作用同上, 参数变了

`errno_t localtime_s(struct tm* const _Tm, time_t const* const _Time)`

老版是通过返回值返回, 新版是通过参数传址调用返回。

`localtime_r()(C23)`: 目前 vs2022 还不支持。同上

`struct tm* localtime_r(const time_t *timer, struct tm* buf);`

`gmtime()`: 将给定的时间转换成国际时间的 `struct tm` 格式

函数原型: `struct tm* __CRTDECL gmtime(time_t const* const _Time)`

`gmtime_s(C11)`: 作用同上

函数原型: `errno_t gmtime_s(struct tm* const Tm, time_t const* const Time)`

`gmtime_r(C23)` 目前 vs2022 还不支持。同上

`struct tm *gmtime_r(const time_t *timer, struct tm *buf);`

`asctime()`: 将 `struct tm` 类型的时间转换成文本字符串

`char* asctime(struct tm const* _Tm);`

`asctime_s(C11)`

`errno_t asctime_s(char* _Buf, size_t _Size, struct tm const* _Tm);`

`asctime_r(C23)` 目前 vs2022 还不支持。同上

`char* asctime_r(const struct tm* time_ptr, char* buf);`

`strftime()`: 将 `struct tm` 类型的时间转换成自定义文本字符串

`size_t strftime(char* Buf, size_t Size, char const* Format, struct tm const* Tm);`

功能类似 `printf`, 通过 `Tm` 访问成员, 即参数 3: `"%A %p"` 这种写法, 其格式说明符有专门的,

<https://docs.microsoft.com/en-us/cpp/c-runtime-library/reference/strftime-wcsftime-strftime-l-wcsftime-l?view=msvc-170>

<https://zh.cppreference.com/w/c/chrono/strftime>

`mktime()`: 将 `struct tm` 格式的时间转换成 `time_t` 类型的时间

`time_t mktime(struct tm* const _Tm)`

4、<float.h> 浮点型的属性，转定义

```
#define DBL_DECIMAL_DIG 17 //四舍五入位
#define DBL_DIG 15 //精确位
#define DBL_EPSILON 2.2204460492503131e-016 //最比较小精度 1.0+DBL_EPSILON != 1.0
#define DBL_HAS_SUBNORM 1 //是否支持非正规数，-1 不确定，1 支，0 不
#define DBL_MANT_DIG 53 //尾数位
#define DBL_MAX 1.7976931348623158e+308 //最大正值
#define DBL_MAX_10_EXP 308 //最大 10 进制指数
#define DBL_MAX_EXP 1024 //最大 2 进制指数
#define DBL_MIN 2.2250738585072014e-308 //最小正值
#define DBL_MIN_10_EXP (-307) //最小 10 进制指数
#define DBL_MIN_EXP (-1021) //最小 2 进制指数
#define _DBL_RADIX 2 //指数基数，即 2 的多少次方
#define DBL_TRUE_MIN 4.9406564584124654e-324 //最小正值，这就是非规格化数
0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0001

#define FLT_DECIMAL_DIG 9 //四舍五入位
#define FLT_DIG 6 //精确位
#define FLT_EPSILON 1.192092896e-07F //最比较小精度 1.0+DBL_EPSILON != 1.0
#define FLT_HAS_SUBNORM 1 //是否支持非正规数，-1 不确定，1 支，0 不
#define FLT_GUARD 0
#define FLT_MANT_DIG 24 //尾数位
#define FLT_MAX 3.402823466e+38F //最大正值
#define FLT_MAX_10_EXP 38 //最大 10 进制指数
#define FLT_MAX_EXP 128 //最大 2 进制指数
#define FLT_MIN 1.175494351e-38F //最小正值
#define FLT_MIN_10_EXP (-37) //最小 10 进制指数
#define FLT_MIN_EXP (-125) //最小 2 进制指数
#define FLT_NORMALIZE 0
#define FLT_RADIX 2 //指数基数，即 2 的多少次方
#define FLT_TRUE_MIN 1.401298464e-45F //最小正值，非规格化数
0000 0000 0000 0000 0000 0000 0000 0001
FLT_EVAL_METHOD 计算中是否拓展精度，0 不，1 拓展为 double，-1 拓展为 long double

double _copysign(double _Number, double _Sign); //把参数 2 的符号放参数 1 值上，返回
double _chgsign(double _X); //负的变正的，正的变负的
double _scalb(double _X, long _Y); //x 乘 2 的 y 次方
double _logb(double _X); //返回浮点型的 2 进制指数值，125.3 返回 6
double _nextafter(double _X, double _Y); //返回离 x 最近的下一个浮点数
int _finite(double _X); //判断是否是无穷 inf
int _isnan(double _X); //判断是否是 nan
int _fpclass(double _X); //返回浮点型状态
```

_FPCLASS_SNAN	信号非数 NaN 指数全 1, 尾数最高位不为 1, 其余有 1
_FPCLASS_QNAN	静态非数 NAN sqrt(-1) 指数全 1, 尾数最高位是 1
_FPCLASS_NINF	-inf 负无穷
_FPCLASS_NN	负的, 指数全为 0, 尾数不全 0
_FPCLASS_ND	负的非规格化数, - DBL_TRUE_MIN
_FPCLASS_NZ	-0
_FPCLASS_PZ	+0
_FPCLASS_PD	正的非规格化数, DBL_TRUE_MIN
_FPCLASS_PN	正的, 指数全为 0, 尾数不全 0
_FPCLASS_PINF	+inf 正无穷

非规格化数可以表示正常范围以外的数, 主要是靠近 0

正常浮点型 $1.01011001 * 2^{-126}$

此时有: $1.01011001 * 2^{-130}$ 但是 float 存不了 -130

所以加上前导 0, 即: $0.000101011001 * 2^{-126}$ 这就是非规格化存储

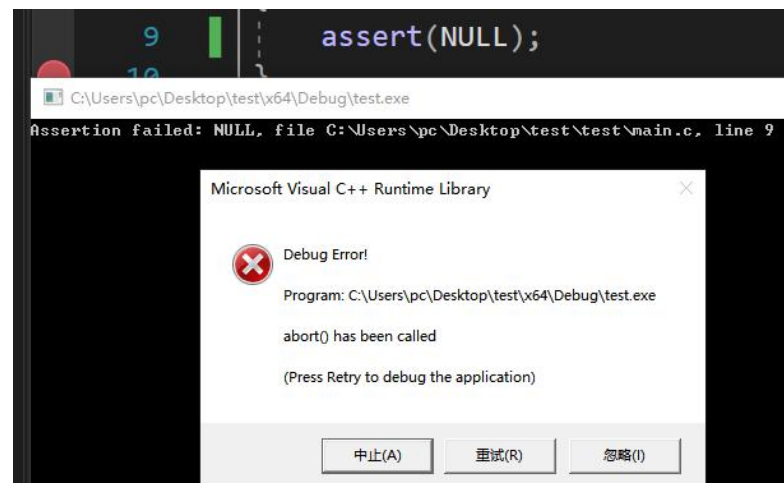
比如

```
#define FLT_TRUE_MIN      1.401298464e-45F    //最小正值, 非规格化数
                        0000 0000 0000 0000 0000 0000 0000 0001
                        本质:  $1.0 * 2^{-150}$  因为最多存  $-126$ , 存不了  $-150$ 
                        所以:  $0.0000 0000 0000 0000 0000 0000 0000 0001 * 2^{-126}$ 
```

5、<assert.h> 断言

assert(exp); 用来中断可能产生的错误

当参数 exp 为 NULL 时, 弹出中断框



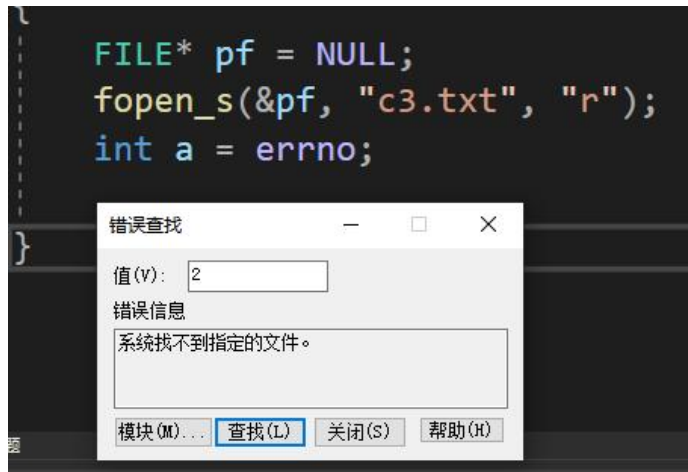
这个框经常遇见, 比如_s 版本函数越界, 传递指针 NULL 时, 都是这个断言的应用。

6、<errno.h>: 错误码获取

用来获取某个标准函数执行失败时的错误码, 错误码用来明晰错误原因

```
_ACRTIMP int* __cdecl _errno(void);
#define errno (*_errno())
```

举例:



通过 `a` 得到错误码，点击工具->错误查找，将错误码 2 放进去，显示错误信息。
该代码的错误是，我没有创建 `c3.txt` 文件，而 `r` 需要文件存在，否则失败。
检测错误时，一定要在该函数后立即检查，不能隔着别的函数，`errno` 是只检测最近的那个。

```
errno_t _set_errno(int _Value);  
//清零，正常使用 errno 前，设置 0，防止没错瞎出个错误码  
errno_t _get_errno(int* _Value);  
//获取错误码，同 errno，这个是传址调用
```

```
unsigned long* __doserrno(void);  
#define _doserrno (*__doserrno()) //获取原始错误码  
程序执行错误，首先操作系统知道，然后操作系统将错误传递给咱们的程序，这期间将错误码进行转换，转换成我们程序专属的错误码。
```

```
errno_t _set_doserrno(unsigned long _Value);  
//同上，清零错误码  
errno_t _get_doserrno(unsigned long * _Value);  
//同_doserrno
```

另外还有一堆错误码，提前定义好的，这个不用看。错误码一般都是在出错时候查，通过 `errno` 查到错误码，然后进行相应的代码处理。

7、<limits.h>: 整型的范围定义

```
#define CHAR_BIT      8  
#define SCHAR_MIN    (-128)  
#define SCHAR_MAX     127  
#define UCHAR_MAX    0xff  
#define MB_LEN_MAX   5  
#define SHRT_MIN     (-32768)  
#define SHRT_MAX     32767
```

```

#define USHRT_MAX    0xffff
#define INT_MIN      (-2147483647 - 1)
#define INT_MAX      2147483647
#define UINT_MAX     0xffffffff
#define LONG_MIN     (-2147483647L - 1)
#define LONG_MAX     2147483647L
#define ULONG_MAX    0xffffffffUL
#define LLONG_MAX    9223372036854775807i64
#define LLONG_MIN    (-9223372036854775807i64 - 1)
#define ULLONG_MAX   0xfffffffffffffffffui64

#define _I8_MIN      (-127i8 - 1)    //后缀: i8   i16   i32   i64
#define _I8_MAX      127i8          //后缀: ui8  ui16  ui32  ui64
#define _UI8_MAX     0xffui8        //咱们之前说 char short 没有后缀，这是不对的，作为整
                                     型，是有的

#define _I16_MIN     (-32767i16 - 1)
#define _I16_MAX     32767i16
#define _UI16_MAX    0xffffui16

#define _I32_MIN     (-2147483647i32 - 1)
#define _I32_MAX     2147483647i32
#define _UI32_MAX    0xffffffffui32

#define _I64_MIN     (-9223372036854775807i64 - 1)
#define _I64_MAX     9223372036854775807i64
#define _UI64_MAX    0xfffffffffffffffffui64

#ifndef SIZE_MAX    //意思是这个宏的大小随着系统的变换而变化
    #ifdef _WIN64
        #define SIZE_MAX 0xfffffffffffffffffui64
    #else
        #define SIZE_MAX 0xffffffffui32
    #endif
#endif

#if __STDC_WANT_SECURE_LIB__
    #ifndef RSIZE_MAX
        #define RSIZE_MAX (SIZE_MAX >> 1)
    #endif
#endif

```

8、<stdint.h> C99: 再次对整型进行功能性定义

转定义进去看看:

这是一些对基本整型的重命名, 凸显其符号以及字节数的命名。

```
typedef signed char      int8_t;
typedef short            int16_t;
typedef int              int32_t;
typedef long long        int64_t;
typedef unsigned char    uint8_t;
typedef unsigned short    uint16_t;
typedef unsigned int      uint32_t;
typedef unsigned long long uint64_t;

typedef signed char      int_least8_t;
typedef short            int_least16_t;
typedef int              int_least32_t;
typedef long long        int_least64_t;
typedef unsigned char    uint_least8_t;
typedef unsigned short    uint_least16_t;
typedef unsigned int      uint_least32_t;
typedef unsigned long long uint_least64_t;

typedef signed char      int_fast8_t;
typedef int              int_fast16_t;
typedef int              int_fast32_t;
typedef long long        int_fast64_t;
typedef unsigned char    uint_fast8_t;
typedef unsigned int      uint_fast16_t;
typedef unsigned int      uint_fast32_t;
typedef unsigned long long uint_fast64_t;

typedef long long        intmax_t;
typedef unsigned long long uintmax_t;
```

以下再次定义整型范围

```
#define INT8_MIN      (-127i8 - 1)
#define INT16_MIN     (-32767i16 - 1)
#define INT32_MIN     (-2147483647i32 - 1)
#define INT64_MIN     (-9223372036854775807i64 - 1)
#define INT8_MAX       127i8
#define INT16_MAX      32767i16
#define INT32_MAX      2147483647i32
#define INT64_MAX      9223372036854775807i64
#define UINT8_MAX      0xffui8
#define UINT16_MAX     0xffffui16
```

```
#define UINT32_MAX    0xffffffffui32
#define UINT64_MAX    0xffffffffffffffffui64
```

以下对范围宏再次重命名

```
#define INT_LEAST8_MIN    INT8_MIN
#define INT_LEAST16_MIN   INT16_MIN
#define INT_LEAST32_MIN   INT32_MIN
#define INT_LEAST64_MIN   INT64_MIN
#define INT_LEAST8_MAX    INT8_MAX
#define INT_LEAST16_MAX   INT16_MAX
#define INT_LEAST32_MAX   INT32_MAX
#define INT_LEAST64_MAX   INT64_MAX
#define UINT_LEAST8_MAX   UINT8_MAX
#define UINT_LEAST16_MAX  UINT16_MAX
#define UINT_LEAST32_MAX  UINT32_MAX
#define UINT_LEAST64_MAX  UINT64_MAX
```

```
#define INT_FAST8_MIN     INT8_MIN
#define INT_FAST16_MIN    INT32_MIN
#define INT_FAST32_MIN    INT32_MIN
#define INT_FAST64_MIN    INT64_MIN
#define INT_FAST8_MAX     INT8_MAX
#define INT_FAST16_MAX    INT32_MAX
#define INT_FAST32_MAX    INT32_MAX
#define INT_FAST64_MAX    INT64_MAX
#define UINT_FAST8_MAX    UINT8_MAX
#define UINT_FAST16_MAX   UINT32_MAX
#define UINT_FAST32_MAX   UINT32_MAX
#define UINT_FAST64_MAX   UINT64_MAX
```

```
#ifdef _WIN64
    #define INTPTR_MIN     INT64_MIN
    #define INTPTR_MAX     INT64_MAX
    #define UINTPTR_MAX    UINT64_MAX
#else
    #define INTPTR_MIN     INT32_MIN
    #define INTPTR_MAX     INT32_MAX
    #define UINTPTR_MAX    UINT32_MAX
#endif
```

```
#define INTMAX_MIN        INT64_MIN
#define INTMAX_MAX        INT64_MAX
#define UINTMAX_MAX       UINT64_MAX
```

```
#define PTRDIFF_MIN      INTPTR_MIN
#define PTRDIFF_MAX      INTPTR_MAX
```

MAX_SIZE 的定义

```
#ifndef SIZE_MAX
    // SIZE_MAX definition must match exactly with limits.h for modules support.
    #ifdef _WIN64
        #define SIZE_MAX 0xffffffffffffffffui64
    #else
        #define SIZE_MAX 0xffffffffui32
    #endif
#endif
```

以下定义点儿新东西

```
#define SIG_ATOMIC_MIN    INT32_MIN
#define SIG_ATOMIC_MAX    INT32_MAX

#define WCHAR_MIN         0x0000
#define WCHAR_MAX         0xffff

#define WINT_MIN          0x0000
#define WINT_MAX          0xffff

#define INT8_C(x)         (x)
#define INT16_C(x)        (x)
#define INT32_C(x)        (x)
#define INT64_C(x)        (x ## LL)    //加了后缀

#define UINT8_C(x)        (x)
#define UINT16_C(x)       (x)
#define UINT32_C(x)       (x ## U)     //加了后缀
#define UINT64_C(x)       (x ## ULL)   //加了后缀

#define INTMAX_C(x)       INT64_C(x)
#define UINTMAX_C(x)      UINT64_C(x)
```

9、<inttypes.h>(C99) 整型格式说明符，转换

转定义看看

有以下转换函数

```
intmax_t imaxabs(intmax_t _Number);
//计算绝对值 intmax_t long long
imaxdiv_t imaxdiv(intmax_t _Numerator, intmax_t _Denominator);
//计算参数 1 与参数 2 相除结果的整数与余数，通过返回值返回
typedef struct
```



```

{
    intmax_t quot; //整数
    intmax_t rem; //余数
} _Lldiv_t;
typedef _Lldiv_t imaxdiv_t;
//返回值是结构体,

intmax_t strtoumax(char const* _String, char** _EndPtr, int _Radix);
//将数字字符串转成无符号整数, 并通过参数 3 指定进制, 返回值返回整数
//返回值: 装着转换后的整数
    char* s = "10101";
    char* p ;
    intmax_t a = strtoumax(s, &p, 2);
//参数 1 是被转换后的数字字符串首地址
//参数 2 装着转换停止时的位置, 比如"10a01", 只转换 10, a 停止, p 就指着 a 的地址
//参数 3, 指示字符串的进制, 2 表示 10101 是 2 进制, 10 表示 10101 是 10 进制

intmax_t _strtoumax_l(char const* _String, char** _EndPtr, int _Radix, _locale_t _Locale);
//将数字字符串转成无符号整数作用同上
//前三个参数, 返回值都是同上, 参数 4 指定编译环境字符集
// _locale_t c = _create_locale(LC_ALL, "zh-CN"); 填 c

uintmax_t strtoumax(char const* _String, char** _EndPtr, int _Radix);
//将数字字符串转成无符号整数, 参数跟上面一样 strtoumax

uintmax_t _strtoumax_l(char const* _String, char** _EndPtr, int _Radix, _locale_t _Locale);
//将数字字符串转成无符号整数, 参数跟上面一样 _strtoumax_l

//以下四个是对应宽字符的函数, 作用跟上面一模一样, 以及参数的作用。
intmax_t wcstoumax(wchar_t const* _String, wchar_t** _EndPtr, int _Radix);
intmax_t _wcstoumax_l(wchar_t const*String, wchar_t**EndPtr, int Radix, _locale_t Locale);
uintmax_t wcstoumax(wchar_t const* _String, wchar_t**_EndPtr, int _Radix);
uintmax_t _wcstoumax_l(wchar_t const*String, wchar_t**EndPtr, int Radix, _locale_t Locale);

```

有以输出格式定义: printf 用的, 各自还有很多名字, 这里只列一个

```

#define PRId8      "hhd"
#define PRId16     "hd"
#define PRId32     "d"
#define PRId64     "lld"
#define PRIi8      "hhi"
#define PRIi16     "hi"
#define PRIi32     "i"
#define PRIi64     "lli"
#define PRId8      "hho"

```

```
#define PRIo16      "ho"
#define PRIo32      "o"
#define PRIo64      "llo"
#define PRIu8        "hhu"
#define PRIu16       "hu"
#define PRIu32       "u"
#define PRIu64       "llu"
#define PRIx8        "hhx"
#define PRIx16       "hx"
#define PRIx32       "x"
#define PRIx64       "llx"
#define PRIX8        "hhX"
#define PRIX16       "hX"
#define PRIX32       "X"
#define PRIX64       "llX"
```

有以输入格式定义：scanf 用的，各自还有很多名字，这里只列一个

```
#define SCNd8        "hhd"
#define SCNd16       "hd"
#define SCNd32       "d"
#define SCNd64       "lld"
#define SCNi8        "hhi"
#define SCNi16       "hi"
#define SCNi32       "i"
#define SCNi64       "lli"
#define SCNo8        "hho"
#define SCNo16       "ho"
#define SCNo32       "o"
#define SCNo64       "llo"
#define SCNu8        "hhu"
#define SCNu16       "hu"
#define SCNu32       "u"
#define SCNu64       "llu"
#define SCNx8        "hhx"
#define SCNx16       "hx"
#define SCNx32       "x"
#define SCNx64       "llx"
```

10、<fenv.h>(C99) 浮点型环境获取与设置

```
#define FE_TONEAREST _RC_NEAR //最近舍入，理解为四舍五(向偶)入
#define FE_UPWARD _RC_UP //向正无穷舍入 3.4 -> 4 -3.4 -> -3
#define FE_DOWNWARD _RC_DOWN //向正负无穷舍入 3.8 -> 3 -3.8 -> -4
#define FE_TOWARDZERO _RC_CHOP //向 0 舍入 3.8 -> 3 -3.8 -> -3
```

```
int fegetround(void);
```

//返回当前取舍入方向，默认是 FE_TONEAREST，四舍五入

```
int fesetround(int _Round);
```

//设置舍入方向，会影响运算，参数填上述 4 个宏之一

```
fesetround(FE_TONEAREST);
```

```
printf("%.4f\n", 12.34567f); //12.3457
```

```
fesetround(FE_UPWARD);
```

```
printf("%.4f\n", 12.34562f); //12.3457
```

```
fesetround(FE_DOWNWARD);
```

```
printf("%.4f\n", 12.34567f); //12.3456
```

```
fesetround(FE_TOWARDZERO);
```

```
printf("%.4f\n", 12.34567f); //12.3456
```

浮点型的舍入控制

```
#define FE_TONEAREST _RC_NEAR //最近舍入，理解为四舍五入
#define FE_UPWARD _RC_UP //向正无穷舍入
#define FE_DOWNWARD _RC_DOWN //向正负无穷舍入
#define FE_TOWARDZERO _RC_CHOP //向 0 舍入，截断
```

浮点型的状态

```
#define FE_INEXACT _SW_INEXACT // 不精确的结果，有舍入
#define FE_UNDERFLOW _SW_UNDERFLOW // 结果向下溢出，即超过下边界了
#define FE_OVERFLOW _SW_OVERFLOW // 结果向上溢出，即超过上边界了
#define FE_DIVBYZERO _SW_ZERODIVIDE // 分母是 0
#define FE_INVALID _SW_INVALID // 无效的运算
#define FE_ALL_EXCEPT (FE_DIVBYZERO | FE_INEXACT | FE_INVALID | FE_OVERFLOW |
FE_UNDERFLOW)
// 5 个状态按位或，表示代表所有
```

```
typedef struct fenv_t
```

```
{
```

```
    unsigned long _Fe_ctl, //浮点型舍入控制
```

```
    _Fe_stat; //当前浮点型环境状态
```

```
} fenv_t;
```

```

int fegetenv(fenv_t* _Env);
int fesetenv(fenv_t const* _Env);
//设置/获取当前浮点型环境，参数是上边那个结构体变量地址，设置完通过下面
fetestexcept 函数测试设置结果，否则会出现异常

int feclearexcept(int _Flags);
//清空浮点型的状态，可以指定，上面五个宏之一，也可以 FE_ALL_EXCEPT，清空所有状态

int fetestexcept(int _Flags);
//检测某个状态，参数 5 个宏之一，如果是当前这个状态，则返回 1，不是则返回 0

int fegetexceptflag(fexcept_t* _Except, int _TestFlags);
//获取到当前浮点型的所有状态，其或的结果装在_Except，参数 2 填 FE_ALL_EXCEPT

int fesetexceptflag(fexcept_t const* _Except, int _SetFlags);
//设置浮点型的状态，其或的结果装在_Except，参数 2 填 FE_ALL_EXCEPT

int feraiseexcept(int _Except)
//用来增加浮点型的状态，比如原有 1 个，他可以再增加一个

int feholdexcept(fenv_t* _Env);
//将当前的浮点型环境状态保存在 env 里，然后清空浮点型状态

int feupdateenv(const fenv_t *_Penv)
//更新恢复浮点型的默认状态

```

11、<iso646.h> 运算符的宏命名

```

#define and &&           //2 and 3
#define and_eq &=        //a and_eq 2
#define bitand &         //2 bitand 3
#define bitor |          //2 bitor 3
#define compl ~          // compl 2
#define not !            // not 2
#define not_eq !=        // 2 not_eq 3
#define or ||           // 2 or 3
#define or_eq |=         // a or_eq 2
#define xor ^           // 2 xor 3
#define xor_eq ^=        // a xor_eq 2

```

首字符下划线的函数，单下划线，两个下划线，三个下划线，叫 CRT 函数，即内部调用的标准 C 语言函数，其实现随着版本的更迭而变化，不建议我们使用，我们使用无下划线版本。所以大家要知道，我是刚知道的。

12、<locale.h> 本地化设置函数

语言环境：

不同的国家与地区对于一些信息的显示有各自的习惯与特点

比如钱的符号 \$, ¥, £...

比如整数的显示 123455，有的三个一组中间用逗号 123,456...

比如日期的显示 年月日，日月年，有的显示上午下午，有的显示年号...

语言环境的根本就是字符集(字符合集)，每个字符(汉字，字母，标点，特殊符号...)都对应一个编号，就像 ASCII 码一样(最小的字符集)，不同国家的字符集的内容是不同，因为各个国家地区的语言文字，表示习惯是不同的，所以不同的字符集是不一样的，比如 A 这个符号，在这里是 65，在那里可能是 85，也可能没有这个字符。

所以，一般在某个编辑器或者网页复制的代码，放在 VS 里直接各种乱码符号，就是这个原因。

```
char* setlocale(int _Category, char const* _Locale);
```

//功能：设置语言环境，

//参数 1，影响范围设置，即在哪个应用上起作用，以下 6 宏之一，不同的编译环境，可能会增加很多选项

范围宏

```
#define LC_ALL 0 //整个 C 语言环境
#define LC_COLLATE 1 //字符比较，不同地区国家的语言应用环境不同，所以同一个字符
//比如 A，在不同的语言环境下，不一定是 65，所以会影响字符串
//的比较。以下函数受影响
//strcoll, _stricoll, wcsoll, _wcsicoll, strxfrm, _st
//rncoll, _strnicoll, _wcsncoll, _wcsnicoll, wcsxfrm
#define LC_CTYPE 2 //字符字符串的转换以及操作
#define LC_MONETARY 3 //货币显示格式
#define LC_NUMERIC 4 //数字显示格式，比如有的是 123,456,789 三个一组，中间用逗号
#define LC_TIME 5 //时间显示格式，主要是 strftime 函数，影响函数如下：
//strftime wcsftime
```

//参数 2，语言代号，“zh-CN”，“C”表示 C 语言环境，ASCII 码，“”表示使用默认本地的

//我在 MSDN 上找到如下表，这些是常见的编码形式

Language string	Equivalent Locale Name
american	en-US
american english	en-US
american-english	en-US
australian	en-AU
belgian	nl-BE
canadian	en-CA
chh	zh-HK

Language string	Equivalent Locale Name
chi	zh-SG
chinese	zh
chinese-hongkong	zh-HK
chinese-simplified	zh-CN
chinese-singapore	zh-SG
chinese-traditional	zh-TW
dutch-belgian	nl-BE
english-american	en-US
english-aus	en-AU
english-belize	en-BZ
english-can	en-CA
english-caribbean	en-029
english-ire	en-IE
english-jamaica	en-JM
english-nz	en-NZ
english-south africa	en-ZA
english-trinidad y tobago	en-TT
english-uk	en-GB
english-us	en-US
english-usa	en-US
french-belgian	fr-BE
french-canadian	fr-CA
french-luxembourg	fr-LU
french-swiss	fr-CH
german-austrian	de-AT
german-lichtenstein	de-LI
german-luxembourg	de-LU
german-swiss	de-CH
irish-english	en-IE
italian-swiss	it-CH
norwegian	no
norwegian-bokmal	nb-NO
norwegian-nynorsk	nn-NO
portuguese-brazilian	pt-BR
spanish-argentina	es-AR
spanish-bolivia	es-BO
spanish-chile	es-CL
spanish-colombia	es-CO
spanish-costa rica	es-CR

Language string	Equivalent Locale Name
spanish-dominican republic	es-DO
spanish-ecuador	es-EC
spanish-el salvador	es-SV
spanish-guatemala	es-GT
spanish-honduras	es-HN
spanish-mexican	es-MX
spanish-modern	es-ES
spanish-nicaragua	es-NI
spanish-panama	es-PA
spanish-paraguay	es-PY
spanish-peru	es-PE
spanish-puerto rico	es-PR
spanish-uruguay	es-UY
spanish-venezuela	es-VE
swedish-finland	sv-FI
swiss	de-CH
uk	en-GB
us	en-US
usa	en-US

```

struct lconv* localeconv(void);
//获取当前语言环境下的语言环境信息，返回值装着，是个结构体，MSDN 上有
    setlocale(LC_ALL, "zh-CN");
    struct lconv* t = localeconv();
//结构体介绍，通识知识
struct lconv
{
    char*    decimal_point;    //非货币数量的小数点字符，有的是. 有的是逗号,
    char*    thousands_sep;    //非货币数量的小数点前面的分位字符，千分符
    char*    grouping;         //一组多少个数，非货币数量
                                举例：1.234.567,89          //8 前的是小数点，再前面的是分位符
                                1 • 234 • 567,89
                                1'234'567,89
                                1,234,567.89
                                1,234,567 • 89
                                1 234 567.89
                                1 234 567,89
    char*    int_curr_symbol;   //当前语言环境的国际货币符号,CNY,USD,HNL...
    char*    currency_symbol;   //当前语言环境的本地货币符号,¥,$,L...
    char*    mon_decimal_point; //货币数量的小数点字符
    char*    mon_thousands_sep; //货币数量的分位字符

```

```

char*    mon_grouping;        //货币数量每组数量
char*    positive_sign;      //正号 +，有的没有
char*    negative_sign;      //负号 -
char     int_frac_digits;     //国际格式化货币数量中小数点右侧的位数
char     frac_digits;         //格式化货币数量中小数点右侧的位数
char     p_cs_precedes;       //如果货币符号在非负格式货币数量的值之前，则设置为 1。
                                如果符号跟随值，则设置为 0。
char     p_sep_by_space;      //如果货币符号与非负格式货币数量的值用空格分隔，则设置为
                                1。如果没有空格分隔，则设置为 0。
char     n_cs_precedes;       //如果货币符号在负格式货币数量的值之前，则设置为 1。如果
                                符号成功值，则设置为 0。
char     n_sep_by_space;      //如果货币符号与负格式货币数量的值用空格分隔，则设置为
                                1。如果没有空格分隔，则设置为 0
char     p_sign_posn;         //正号在非负格式货币数量中的位置
char     n_sign_posn;         //负格式货币数量中正号的位置
wchar_t* _W_decimal_point;    //非货币数量的小数点字符，有的是. 有的是逗号，
wchar_t* _W_thousands_sep;    //非货币数量的小数点前面的分位字符，千分符
wchar_t* _W_int_curr_symbol;   //当前语言环境的国际货币符号,CNY,USD,HNL...
wchar_t* _W_currency_symbol;  //当前语言环境的本地货币符号,¥,$,L...
wchar_t* _W_mon_decimal_point;//货币数量的小数点字符
wchar_t* _W_mon_thousands_sep;//货币数量的分位字符
wchar_t* _W_positive_sign;     //正号 +，有的没有
wchar_t* _W_negative_sign;     //负号 -
};

void _lock_locales(void);
void _unlock_locales(void);
//多线程环境中操作是否带锁
int _configthreadlocale(int _Flag);
//设置多线程中使用语言环境选项
// _ENABLE_PER_THREAD_LOCALE 使用 setlocal 仅影响该线程的语言环境
// _DISABLE_PER_THREAD_LOCALE 线程使用全局语言环境，使用 setlocal 影响所有线程
//0 恢复当前线程语言环境设置

_locale_t _get_current_locale(void);
//获取表示当前语言环境的语言环境对象
_locale_t _create_locale(int _Category, char const* _Locale);
//创建语言环境对象，参数跟 setlocal 一样
void _free_locale(_locale_t _Locale);
//将上面 create 创建的语言环境对象释放掉

wchar_t* _wsetlocale(int _Category, wchar_t const* _Locale);
//宽字符版，本质就是参数用宽字符 wchar_t 原来的是 char，
_locale_t _wcreate_locale(int _Category, wchar_t const* _Locale);

```


//宽字符版, 本质就是参数用宽字符 wchar_t 原来的是 char

```
wchar_t** __lc_locale_name_func(void);
```

//获取当前线程的环境名字, 内部 CRT 函数(标准 C 函数), 随着版本的变化而变化, 不建议使用, 其功能跟 _get_current_locale, localeconv 一样

```
unsigned int __lc_codepage_func(void);
```

//获取当前线程的字符集(代码页), 内部 CRT 函数(标准 C 函数), 随着版本的变化而变化, 不建议使用, _get_current_locale 可以得到相应信息 LC_CTYPE, 字符转换环境

```
unsigned int __lc_collate_cp_func(void);
```

//获取当前线程的字符集(代码页), 内部 CRT 函数(标准 C 函数), 随着版本的变化而变化, 不建议使用, _get_current_locale 可以得到相应信息 LC_COLLATE, 字符比较环境, 大小

```
char* _Getdays(void);
```

//获取当前语言环境天的名字,

zh-cn: 周日:星期日:周一:星期一:周二:星期二:周三:星期三:周四:星期四:周五:星期五:周六:星期六

en-us: Sun: Sunday: Mon: Monday: Tue: Tuesday: Wed: Wednesday: Thu: Thursday: Fri: Friday: Sat: Saturday

```
char* _Getmonths(void);
```

//获取当前语言环境月的名字,

zh-cn: 1月:一月:2月:二月:3月:三月:4月:四月:5月:五月:6月:六月:7月:七月:8月:八月:9月:九月:10月:十月:11月:十一月:12月:十二月

en-us: Jan: January: Feb: February: Mar: March: Apr: April: May: May: Jun: June: Jul: July: Aug: August: Sep: September: Oct: October: Nov: November: Dec: December

```
void* _Gettnames(void); //没搜到, 应该是获取年的名字, 比如千禧年, 快乐年什么的
```

//以下三个对应宽字符版

```
wchar_t* _W_Getdays(void);
```

```
wchar_t* _W_Getmonths(void);
```

```
void* _W_Gettnames(void);
```

```
size_t _Strftime( char* _Buffer,
                  size_t _Max_size,
                  char const* _Format,
                  struct tm const* _Timeptr,
                  void* _Lc_time_arg );
```

//时间信息格式化函数

```
size_t _Wcsftime( wchar_t* _Buffer,
                  size_t _Max_size,
                  wchar_t const* _Format,
                  struct tm const* _Timeptr,
                  void* _Lc_time_arg );
```

//时间信息格式化函数, 宽字符版

13、<ctype.h> 用来确定包含于字符数据中的类型的函数

语言环境参数设置方式如下，意为在这个语言环境下

```
_locale_t t = _create_locale(LC_ALL, "zh-CN");
```

```
int _isctype( int _C, int _Type);
```

//判断参数 1 的字符类型

//参数 2 默认 ASCII 码表上的

```
#define _UPPER 0x01 // 大写字母 A-Z
```

```
#define _LOWER 0x02 // 小写字母 a-z
```

```
#define _DIGIT 0x04 // 数字[0-9]
```

```
#define _SPACE 0x08 // ' ', \t, \r, \n, 垂直制表\v, 换页\f
```

```
#define _PUNCT 0x10 // 标点符号
```

```
#define _CONTROL 0x20 // 合法转义字符\0 \b \t \v \f \r \n
```

```
#define _BLANK 0x40 // 专门空格 不包括制表符
```

```
#define _HEX 0x80 // 16 进制数字 '\x30'-' \x39' 即 '0'-'9'，0x30-0x39 48-57
```

```
#define _LEADBYTE 0x8000 // 标识符的首字符，大小写字母或者下划线
```

```
#define _ALPHA (0x0100 | _UPPER | _LOWER) // 大小写字母
```

//返回值，如果是就返回上面的宏常量数

```
int _isctype_l( int _C, int _Type, _locale_t _Locale);
```

//作用同上，多了语言环境

```
int isalpha( int _C);
```

//判断参数是否是大小写字母，大写返回 1，小写返回 2，非大小写返回 0，就是上边的宏

```
int _isalpha_l( int _C, _locale_t _Locale);
```

//在指定的语言环境下，判断参数是否是大小写字母，前方定义的 t

```
int isupper( int _C);
```

//判断参数是否是大写字母，是返回 1，不是返回 0，就是上边的宏

```
int _isupper_l( int _C, _locale_t _Locale);
```

//在指定的语言环境下，判断参数是否是大写字母，前方定义的 t

```
int islower( int _C);
```

//判断参数是否是大小写字母，是返回 2，不是返回 0，就是上边的宏

```
int _islower_l( int _C, _locale_t _Locale);
```

//在指定的语言环境下，判断参数是否是大小写字母，前方定义的 t

```
int isdigit( int _C);
```

//判断参数是否是数字字符，是返回 4，不是返回 0，就是上边的宏

```
int _isdigit_l( int _C, _locale_t _Locale);
```

//在指定的语言环境下，判断参数是否是数字字符，前方定义的 t

```
int isxdigit( int _C);
```

//判断参数是否是 16 进制数字字符，是返回 0x80，不是返回 0，就是上边的宏

```
int _isxdigit_l( int _C, _locale_t _Locale);
```

//在指定的语言环境下，判断参数是否是 16 进制数字字符，前方定义的 t

```
int isspace( int _C);
```

//判断参数是否是空格类，是返回 0x8，不是返回 0，就是上边的宏_SPACE

```
int _isspace_l( int _C, _locale_t _Locale);
```

//在指定的语言环境下，判断参数是否是空格类字符，前方定义的 t

```
int ispunct( int _C);
```

//判断参数是否是标点符号，是返回 0x10，不是返回 0，就是上边的宏_PUNCT

```
int _ispunct_l( int _C, _locale_t _Locale);
```

//在指定的语言环境下，判断参数是否是标点符号

```
int isblank( int _C);
```

//判断参数是否是空格，是返回 0x40，不是返回 0，就是上边的宏_BLANK

```
int _isblank_l( int _C, _locale_t _Locale);
```

//在指定的语言环境下，判断参数是否是空格

```
int isalnum( int _C);
```

//判断参数是否是小写字母数字，大写返回 1，小写返回 2，数字返回 4，非返回 0

```
int _isalnum_l( int _C, _locale_t _Locale);
```

//在指定的语言环境下，判断参数是否是小写字母数字

```
int isprint( int _C);
```

//判断参数是否是可打印字符，32~127 号，成功返回对应分组的那个宏，转义字符是不可输出的

```
int _isprint_l( int _C, _locale_t _Locale);
```

//在指定的语言环境下，判断参数是否是可输出字符

```
int isgraph( int _C);
```

//判断参数是否是可画出字符，有样貌，不包括空格，失败返回 0

```
int _isgraph_l( int _C, _locale_t _Locale);
```

//在指定的语言环境下，判断参数是否是可输出字符

```
int iscntrl( int _C);
```

//判断参数是否是控制字符，即转义字符合法转义字符\0 \b \t \v \ a \f \r \n，成功_CONTROL

```
int _iscntrl_l( int _C, _locale_t _Locale);
```

//在指定的语言环境下，判断参数是否是控制字符

```
int toupper( int _C);
```

//转换成大写字母，返回值返回

```
int tolower( int _C);
```

//转换成小写字母，返回值返回

```
int _tolower( int _C);
```

//转换成小写字母，

```
int _tolower_l( int _C, _locale_t _Locale);
```

```

//转换成小写字母，转头文件有 CRT 标识，
//参数 2 是在指定的语言环境下
int  _toupper( int _C);
//转换成大写字母，
int  _toupper_l( int _C, _locale_t _Locale);
//转换成大写字母，参数 2 是在指定的语言环境下

int  __isascii( int _C);
#define isascii __isascii
//判断 c 是不是 ASCII 字符，即 0x00 - 0x7f，如果不是返回非 0，是返回 0

int  __toascii( int _C);
#define toascii __toascii
//通过截断将字符转换为 7 位 ASCII。 %128

int  __iscsymf( int _C);
#define iscsymf __iscsymf
//判断字符是否可以作为标识符的第一个字符，大小写字母，下划线返回 1，其余返回 0
int  __iscsym( int _C);
#define iscsym __iscsym
//判断字符是否可以作为标识符的非第一个字符，大小写字母，下划线，数字返回 1，其余返回 0

```

14、<wchar.h> (C95) 扩展多字节和宽字符工具

ASCII 码字符集下，默认 1 个字符是 1 字节，因为就那么 128 个。如果说‘包’这个中文，其不是 1 字节 ASCII 码范围的，所以其编号一定大于 1 字节，那就是 2 字节，ASCII 码下，2 字节以上就是字符串了，所以要以字符串形式处理“包”，即 3 个字符，包含\0。

```

char str[3] = "包";
puts(str);

```

那么如何能用‘包’单引号处理 1 个汉字呢？即将汉字作为 1 个字符处理呢？那就是更改字符集，将字符集改成包含中文的字符集。

```

setlocale(LC_ALL, "zh-CN"); //设置为中文简体的字符集环境

```

更改环境字符集后，就要忘了 ASCII 码了。原理跟 ASCII 码一样，每个字符 1 个 ID。

此时所有的字符都是用 2 字节编号，即 0~65535，转定义 wchar.h

```

#define WCHAR_MIN 0x0000    0
#define WCHAR_MAX 0xffff    65535

```

那么此时的字符，也不能用 char 来装了，装不开。有专属类型 wchar_t

```

setlocale(LC_ALL, "zh-cn");//设置字符集
wchar_t wc = L'包';           //单引号前加 L，表示宽字符
putwchar(wc);                 //函数也是专属带'W'的那一系列函数，putchar putwchar
wc = _T('东');                //单引号加 _T()，表示宽字符，意同 L，<TCHAR.h>
putwc(wc, stdout);            //函数带 W  putc putwc
wc = TEXT('华');              //单引号加 TEXT()，表示宽字符，意同 L，<windows.h>
wprintf(L"%wc", wc);          //wprintf 与 printf 常量字符串前一定加 L，%c 与%wc 都行

```

```
wprintf(L"\n%d", sizeof(wc)); //输出 2
```

字符串操作:

```
setlocale(LC_ALL, "zh-cn");  
//_wsetlocale(LC_ALL, L"zh-cn"); 都行  
wchar_t wc[4] = L"包东华"; //双引号前加L, 表示宽字符  
wprintf(L"%wc %wc %wc\n", wc[0], wc[1], wc[2]); //双引号前加L, 表示宽字符  
wscanf_s(L"%ws", wc, 8); //双引号前加L, 表示宽字符  
wprintf(L"%wc %wc %wc\n", wc[0], wc[1], wc[2]); //双引号前加L, 表示宽字符  
//一定要用宽字符 wchar_t 对应的 w 专属函数, 功能与意义跟多字节 char 函数是一模一样的。  
//只是涉及到字符串参数与返回值的函数, 才有 wchar_t 专属, 别的没有。  
比如 int _configthreadlocale(int _Flag); 参数与返回值都没有 char, 所以没有 w 版本函数
```

头文件有以下两个函数, 前面说了, 只不过参数返回值有 wchar_t

```
wchar_t* _wsetlocale(int _Category, wchar_t const* _Locale); L"zh-cn"  
setlocale(LC_ALL, "zh-cn");  
_wsetlocale(LC_ALL, L"zh-cn");  
_wsetlocale(LC_ALL, L ""); //恢复 ASCII 的 C 语言环境  
_wsetlocale(LC_ALL, L"C"); //恢复 ASCII 的 C 语言环境
```

```
_locale_t _wcreate_locale(int _Category, wchar_t const* _Locale); L"zh-cn"
```

其他函数

```
wint_t btowc(int _Ch);  
//将单字节字符转换成对应的宽字符形式, 因为单字节字符是 1 字节存储, 改变语言环境后, 单字节  
字符也需要两个字节存储, 一个字符如何用两个字节存储? 这就是转换的意义。
```

```
size_t mbrlen(char const* _Ch, size_t _SizeInBytes, mbstate_t* _State);  
//判断 ch 所指的字符, 是几个字节的字符, ASCII 就是 1 字节, 中文就是 2 字节, 并返回
```

```
size_t mbrtowc(wchar_t* Dst, char const* Src, size_t Size, mbstate_t* State);  
//将多字节字符串 src 中指定位置转换成宽字符, 并存储在 dst 中, size 是 dst 的字节数  
//如果 src 指的是单字节字符, 即 ASCII, 返回值为 1  
//如果 src 指的是非单字节字符, 即中文等等, 返回值为 2
```

```
errno_t mbsrtowcs_s(size_t* _RetVal,  
wchar_t* _Dst,  
size_t _Size,  
char const** _PSrc,  
size_t _N,  
mbstate_t* _State );
```

//举例说明

```
char ws[15] = "he 好 lo";
```

```

char* p = ws;
wchar_t wc[15];
mbstate_t mbstate = {0};
size_t ee = 4;
errno_t a = mbsrtowcs_s(&ee, wc, 15, &p, 4, &mbstate);

```

将 p ~ p+4 的字符转换成宽字符装在 wc 中, 15 是 wc 的元素数。
ee 返回实际转换的字节数, 即 5

```

errno_t wctomb_s( size_t*    _Retval,
                  char*      _Dst,
                  size_t     _SizeInBytes,
                  wchar_t    _Ch,
                  mbstate_t* _State );
//将 1 个宽字符 ch 转换成多字节字符, 放在 Dst 里。
//retaval 装着实际转换后的字节数, _SizeInBytes 是 dst 的字节数

```

```

errno_t wcsrtombs_s(size_t*    _Retval,
                   char*      _Dst,
                   size_t     _SizeInBytes,
                   wchar_t const** _Src,
                   size_t     _Size,
                   mbstate_t* _State );
//将宽字符串 src 转换成多字节字符串, 放在 Dst 里。
//_Retval 是实际转换的字符数, _SizeInBytes 是 dst 的字节数, _Size 是想要转几个

```

```

int wctob(wint_t _WCh);
//将 1 个宽字符 wch 转换成 1 个多字节字符

```

```

wchar_t* wmemchr(wchar_t const* _S, wchar_t _C, size_t _N)
//在 _s 串中查找 _C 字符, 找到返回地址, 没找到返回 0

```

```

int wmemcmp(wchar_t const* _S1, wchar_t const* _S2, size_t _N)
//宽字符串比较前_N 个字符, 相等返回 0, 前大返回 1, 后大返回-1

```

```

wchar_t* wmemcpy(wchar_t* _S1, wchar_t const* _S2, size_t _N)
errno_t wmemcpy_s(wchar_t* S1, rsize_t N1, wchar_t const* S2, rsize_t N);
//宽字符串复制, 将 s2 的前_N 个字符复制进 s1

```

```

wchar_t* wmemmove(wchar_t* _S1, wchar_t const* _S2, size_t _N)
errno_t wmemmove_s(wchar_t* S1, rsize_t N1, wchar_t const* _S2, rsize_t N);
//宽字符串移动, 将 s2 的前_N 个字符移动进 s1
wchar_t* wmemset(wchar_t* _S, wchar_t _C, size_t _N)
//宽字符串设置, 将 _s 的前_N 个字符设置成 _c 字符

```

15、<wctype.h> (C95) 用来确定包含于宽字符数据中的类型的函数

跟 ctype.h 功能一样，装宽字符操作的函数。

```
#define _UPPER    0x01    // 大写字母 A-Z
#define _LOWER    0x02    // 小写字母 a-z
#define _DIGIT    0x04    // 数字[0-9]
#define _SPACE    0x08    // ' ', \t, \r, \n, 垂直制表\v, 换页\f
#define _PUNCT    0x10    // 标点符号
#define _CONTROL  0x20    // 合法转义字符\0 \b \t \v \f \r \n
#define _BLANK    0x40    // 专门空格 不包括制表符
#define _HEX      0x80    // 16 进制数字 '\x30'-' \x39' 即 '0'-'9', 0x30-0x39 48-57
#define _LEADBYTE 0x8000    // 标识符的首字符, 大小写字母或者下划线
#define _ALPHA    (0x0100 | _UPPER | _LOWER) // 大小写字母与数字

int iswalnum (wint_t _C);
//判断参数是否是宽字母数字, 大写返回 1, 小写返回 2, 数字返回 4, 非返回 0
int iswalpna (wint_t _C);
//判断参数是否是大小写字母, 大写返回 1, 小写返回 2, 非大小写返回 0, 就是上边的宏
int iswascii (wint_t _C);
//判断 c 是不是 ASCII 字符, 即 0x00 - 0x7f, 如果不是返回非 0, 是返回 0
int iswblank (wint_t _C);
//判断参数是否是空格, 是返回 0x40, 不是返回 0, 就是上边的宏 _BLANK
int iswcntrl (wint_t _C);
//判断参数是否是控制字符, 即转义字符合法转义字符\0 \b \t \v \f \r \n, 成功 _CONTROL
int iswdigit (wint_t _C);
//判断参数是否是数字字符, 是返回 4, 不是返回 0, 就是上边的宏
int iswgraph (wint_t _C);
//判断参数是否是可画出字符, 有样貌, 不包括空格, 失败返回 0
int iswlower (wint_t _C);
//判断参数是否是宽小写字母, 是返回 2, 不是返回 0, 就是上边的宏
int iswprint (wint_t _C);
//判断参数是否是可打印字符, 32~127 号, 成功返回对应分组的那个宏, 转义字符是不可输出的
int iswpunct (wint_t _C);
//判断参数是否是标点符号, 是返回 0x10, 不是返回 0, 就是上边的宏 _PUNCT
int iswspace (wint_t _C);
//判断参数是否是空格类, 是返回 0x8, 不是返回 0, 就是上边的宏 _SPACE
int iswupper (wint_t _C);
//判断参数是否是宽大写字母, 是返回 1, 不是返回 0, 就是上边的宏
int iswxdigit (wint_t _C);
//判断参数是否是 16 进制数字字符, 是返回 0x80, 不是返回 0, 就是上边的宏
int __iswcsymf(wint_t _C);
//判断字符是否可以作为标识符的第一个字符, 大小写字母, 下划线返回 1, 其余返回 0
int __iswcsym (wint_t _C);
//判断字符是否可以作为标识符的非第一个字符, 大小写字母, 下划线, 数字返回 1, 其余返回 0
int _iswalnum_l (wint_t _C, _locale_t _Locale);
```

```

//判断参数是否是字母数字，大写返回 1，小写返回 2，数字返回 4，非返回 0
int _iswalpha_l (wint_t _C, _locale_t _Locale);
//判断参数是否是大小写字母，大写返回 1，小写返回 2，非大小写返回 0，就是上边的宏
int _iswblank_l (wint_t _C, _locale_t _Locale);
//判断参数是否是空格，是返回 0x40，不是返回 0，就是上边的宏 _BLANK
int _iswcntrl_l (wint_t _C, _locale_t _Locale);
//判断参数是否是控制字符，即转义字符合法转义字符 \0 \b \t \v \f \r \n，成功 _CONTROL
int _iswdigit_l (wint_t _C, _locale_t _Locale);
//判断参数是否是数字字符，是返回 4，不是返回 0，就是上边的宏
int _iswgraph_l (wint_t _C, _locale_t _Locale);
//判断参数是否是可画出字符，有样貌，不包括空格，失败返回 0
int _iswlower_l (wint_t _C, _locale_t _Locale);
//判断参数是否是字母，是返回 2，不是返回 0，就是上边的宏
int _iswprint_l (wint_t _C, _locale_t _Locale);
//判断参数是否是可打印字符，32~127 号，成功返回对应分组的那个宏，转义字符是不可输出的
int _iswpunct_l (wint_t _C, _locale_t _Locale);
//判断参数是否是标点符号，是返回 0x10，不是返回 0，就是上边的宏
int _iswspace_l (wint_t _C, _locale_t _Locale);
//判断参数是否是空格类，是返回 0x8，不是返回 0，就是上边的宏
int _iswupper_l (wint_t _C, _locale_t _Locale);
//判断参数是否是大写字母，是返回 1，不是返回 0，就是上边的宏
int _iswxdigit_l(wint_t _C, _locale_t _Locale);
//判断参数是否是 16 进制数字字符，是返回 0x80，不是返回 0，就是上边的宏
int _iswcsymf_l (wint_t _C, _locale_t _Locale);
//判断字符是否可以作为标识符的第一个字符，大小写字母，下划线返回 1，其余返回 0
int _iswcsym_l (wint_t _C, _locale_t _Locale);
//判断字符是否可以作为标识符的非第一个字符，大小写字母，下划线，数字返回 1，其余返回 0

wint_t towupper(wint_t _C);
//转换成大写字母
wint_t towlower(wint_t _C);
//转换成小写字母
wint_t _towupper_l(wint_t _C, _locale_t _Locale);
//指定语言环境下，转换成大写字母
wint_t _tolower_l(wint_t _C, _locale_t _Locale);
//指定语言环境下，转换成小写字母
int isleadbyte(int _C);
//判断是否是多字节的首字符
int _isleadbyte_l(int _C, _locale_t _Locale);
//指定语言环境下，判断是否是多字节的首字符

```



```

wctrans_t wctrans(const char *name);
"toupper" "tolower"
//生成这两功能的映射值，用做下面这个函数的参数 2
wint_t towctrans(wint_t c, wctrans_t value);
//生成这两功能的映射值，用做下面这个函数的参数 2
wctrans_t t = wctrans("toupper");
wint_t tt = towctrans(L'a', t); //返回值的到大写的值

wctype_t wctype(const char *name);
"alnum" "alpha" "blank" "cntrl" "cntrl"
"digit" "graph" "lower" "print" "space" "xdigit"
生成这些字符串对应的一个值，用做下面这个函数的参数
int iswctype(wint_t _C, wctype_t _Type); //同 isctype
int _iswctype_l(wint_t _C, wctype_t _Type, _locale_t _Locale); //指定环境下
#define _UPPER 0x01 // 大写字母 A-Z
#define _LOWER 0x02 // 小写字母 a-z
#define _DIGIT 0x04 // 数字[0-9]
#define _SPACE 0x08 // ' ', \t, \r, \n, 垂直制表\v, 换页\f
#define _PUNCT 0x10 // 标点符号
#define _CONTROL 0x20 // 合法转义字符\0 \b \t \v \f \r \n
#define _BLANK 0x40 // 专门空格 不包括制表符
#define _HEX 0x80 // 16 进制数字 '\x30'-' \x39' 即 '0'-'9'，0x30-0x39 48-57
#define _LEADBYTE 0x8000 // 标识符的首字符，大小写字母或者下划线
#define _ALPHA (0x0100 | _UPPER | _LOWER) // 大小写字母与数字

#define iswalpha(_c) (iswctype(_c, _ALPHA))
#define iswupper(_c) (iswctype(_c, _UPPER))
#define iswlower(_c) (iswctype(_c, _LOWER))
#define iswdigit(_c) (iswctype(_c, _DIGIT))
#define iswxdigit(_c) (iswctype(_c, _HEX))
#define iswspace(_c) (iswctype(_c, _SPACE))
#define iswpunct(_c) (iswctype(_c, _PUNCT))
#define iswblank(_c) (((_c) == '\t') ? _BLANK : iswctype(_c, _BLANK))
#define iswalnum(_c) (iswctype(_c, _ALPHA | _DIGIT))
#define iswprint(_c) (iswctype(_c, _BLANK | _PUNCT | _ALPHA | _DIGIT))
#define iswgraph(_c) (iswctype(_c, _PUNCT | _ALPHA | _DIGIT))
#define iswcntrl(_c) (iswctype(_c, _CONTROL))
#define iswascii(_c) ((unsigned)(_c) < 0x80)

#define _iswalpha_l(_c, _p) (iswctype(_c, _ALPHA))
#define _iswupper_l(_c, _p) (iswctype(_c, _UPPER))
#define _iswlower_l(_c, _p) (iswctype(_c, _LOWER))
#define _iswdigit_l(_c, _p) (iswctype(_c, _DIGIT))
#define _iswxdigit_l(_c, _p) (iswctype(_c, _HEX))

```

```

#define _iswspace_l(_c, _p) (iswctype(_c, _SPACE))
#define _iswpunct_l(_c, _p) (iswctype(_c, _PUNCT))
#define _iswblank_l(_c, _p) (iswctype(_c, _BLANK))
#define _iswalnum_l(_c, _p) (iswctype(_c, _ALPHA | _DIGIT))
#define _iswprint_l(_c, _p) (iswctype(_c, _BLANK | _PUNCT | _ALPHA | _DIGIT))
#define _iswgraph_l(_c, _p) (iswctype(_c, _PUNCT | _ALPHA | _DIGIT))
#define _iswcntrl_l(_c, _p) (iswctype(_c, _CONTROL))
#define isleadbyte(_c) (__PCTYPE_FUNC[(unsigned char)(_c)] & _LEADBYTE)

```

16、<uchar.h> (C11) UTF-16 和 UTF-32 字符工具

wchar_t 是两字节的类型，本质跟 unsigned short 一样，最多 65536 个字符，但是字符的个数是超过 65536 个的，比如汉字，实际有 10 几个万个，一般字符集收录的是常用的，大概 3 4 万个，如果想做个汉字大全的，或者世界字符大全的字符集，那字符的数量远远超过 65536 个，此时需要更大的字节数存储，那就是 4 字节的字符，本质就是 int

```

typedef unsigned short _Char16_t; //16 位 2 字节的字符
typedef unsigned int _Char32_t;  //32 位 4 字节的字符
//以下函数参数传递要初始化
size_t mbrtoc16(char16_t *_Pc16, const char *_S, size_t _N, mbstate_t *_Ps);
//将多字节字符 s，转换成 16 位字符的字符装在 pc16 里，n 限制 s 中检索字节数
size_t c16rtomb(char *_S, char16_t _C16, mbstate_t *_Ps);
//将 16 位字符的字符 c16，转换成多字节字符装在 s 里，n 限制 _C16 中检索字节数
size_t mbrtoc32(char32_t *_Pc32, const char *_S, size_t _N, mbstate_t *_Ps);
//将多字节字符 s，转换成 32 位字符的字符装在 pc16 里，n 限制 s 中检索字节数
size_t c32rtomb(char *_S, char32_t _C32, mbstate_t *_Ps);
//将 32 位字符的字符 c16，转换成多字节字符装在 s 里，n 限制 _C16 中检索字节数

```

头文件里出现，意思就是 `wcslwr` 已经被 `_wcslwr` 代替，当然原版也能用

```
_CRT_NONSTDC_DEPRECATE(_wcslwr)
_Ret_z_
_ACRTIMP wchar_t* wcslwr(
    _Inout_z_ wchar_t* _String
);
```

17、<string.h> 字符串操作库

字符串拷贝

将 `src` 字符串复制进 `des` 字符数组内。

`desbytes` 是 `des` 的字节数

`count` 是设置拷贝几个字符

```
char * strcpy(char* des, const char *src);
char * strncpy(char* des, const char *src, size_t count);
errno_t strcpy_s(char* des, rsize_t desbytes, char const* src);
errno_t strncpy_s(char* des, rsize_t desbytes, char const* src, rsize_t count);
errno_t wcsncpy_s(wchar_t* des, rsize_t desbytes, wchar_t const* src);
errno_t wcsncpy_s(wchar_t* des, rsize_t desbytes, wchar_t const* src, rsize_t count);
```

字符串拼接

将 `src` 字符串拼接到 `des` 字符串尾巴。

`desbytes` 是 `des` 的字节数

`count` 是设置拼接几个字符

```
char* strcat(char* des, const char* src);
char* strncat(char* des, const char* src, size_t count);
errno_t strcat_s(char* des, rsize_t desbytes, char const* src);
errno_t strncat_s(char* des, rsize_t desbytes, char const* src, rsize_t count);
errno_t wcsat_s(wchar_t* des, rsize_t desbytes, wchar_t const* src);
errno_t wcsncat_s(wchar_t* des, rsize_t desbytes, wchar_t const* src, rsize_t count);
```

字符串比较

比较 `str1` 与 `str2`，区分大小写的字符串比较

`count` 是比较指定个数

```
int strcmp(char const* str1, char const* str2);
int strncmp(char const* str1, char const* str2, size_t count);
int wcscmp(wchar_t const* str1, wchar_t const* str2);
int wcsncmp(wchar_t const* str1, wchar_t const* str2, size_t count);
```

字符串比较

可以设置语言环境，上边那个不行

```
int strcoll(char const* str1, char const* str2);
int _strcoll_l(char const* str1, char const* str2, _locale_t locale);
int _strncoll(char const* str1, char const* str2, size_t count);
```

```
int _strncoll_l(char const* str1, char const* str2, size_t count, _locale_t locale);
```

字符串比较

比较 str1 与 str2，不区分大小写的字符串比较

count 是比较指定个数

locale 是设置语言环境，_locale_t locale = _create_locale("LC_ALL", "zh - cn")

```
int strcmpi(char const* str1, char const* str2);
```

```
int _strcmpi(char const* str1, char const* str2);
```

已经弃用，这俩的新形式是下边这俩，stricmp_stricmp 替代

```
int stricmp(char const* str1, char const* str2);
```

```
int _stricmp(char const* str1, char const* str2);
```

```
int _stricmp_l(char const* str1, char const* str2, _locale_t locale);
```

```
int strnicmp(char const* str1, char const* str2, size_t count);
```

```
int _strnicmp(char const* str1, char const* str2, size_t count);
```

```
int _strnicmp_l(char const* str1, char const* str2, size_t count, _locale_t locale);
```

```
int _wscmp(wchar_t const* str1, wchar_t const* str2);
```

```
int _wscmp_l(wchar_t const* str1, wchar_t const* str2, _locale_t locale);
```

```
int _wcsnicmp(wchar_t const* str1, wchar_t const* str2, size_t count);
```

```
int _wcsnicmp_l(wchar_t const* str1, wchar_t const* str2, size_t count, _locale_t locale);
```

比较字符串的大小，不区分大小写

```
int _stricoll(char const* str1, char const* str2);
```

```
int _stricoll_l(char const* str1, char const* str2, _locale_t locale);
```

```
int _strnicoll(char const* str1, char const* str2, size_t count);
```

```
int _strnicoll_l(char const* str1, char const* str2, size_t count, _locale_t locale);
```

求字符串的长度，到\0 截止，不算\0

count 指示字符串的元素个数，比如字符数组没有\0 结尾，使用 strlen 会计算越界

_s 版本比 n 版本多了 str 的 NULL 检测，更加完善

```
size_t strlen(char const* str);
```

```
size_t strlen(char const* str, size_t count);
```

```
size_t strlen_s(char const* str, size_t count)
```

```
size_t wcslen(wchar_t const* str);
```

```
size_t wcslen(wchar_t const* str, size_t count);
```

```
size_t wcslen_s(wchar_t const* str, size_t count)
```

_s 版本比非_s 版本多了参数 str 不为 NULL 的检测

```
char* strlwr(char* str); //被代替 字符串转换成小写
```

```
char* _strlwr(char* str); //代替上
```

```
errno_t _strlwr_s(char* str, size_t count);
```

```
errno_t _strlwr_s_l(char* str, size_t count, _locale_t locale);
```

```
wchar_t* wslwr(wchar_t* str); //被代替
```

```
wchar_t* _wslwr(wchar_t* str); //代替上
```

```
errno_t _wslwr_s(wchar_t* str, size_t count);
```

```
errno_t _wslwr_s_l(wchar_t* str, size_t count, _locale_t locale);
```

查找子串

```
char *strchr(char const* str, int ch);    //在 str 中找 ch, 找到返回该字符地址  
char *strrchr(char const* str, int ch);   //在 str 中找 ch, 找到返回该字符的下一个字符地址  
char *strstr(char const* str, char const* SubStr); //在 str 中查找子串 SubStr  
wchar_t *wcschr(wchar_t const* str, wchar_t ch);  
wchar_t *wcsrchr(wchar_t const* str, wchar_t ch);  
wchar_t *wcsstr(wchar_t const* str, wchar_t const* SubStr);
```

字符串元素设置

```
//_strset, 将字符串 str 全部设置成 ch  
char* strset(char* str, int ch);  
errno_t _strset_s(char* str, size_t strbytes, int ch);  
//将字符串 str 前 count 个设置成 ch  
char* strnset(char* str, int ch, size_t count);  
errno_t _strnset_s(char* str, size_t strbytes, int ch, size_t count);  
//宽字符版  
errno_t _wcsset_s(wchar_t* str, size_t strbytes, wchar_t ch);  
errno_t _wcsnset_s(wchar_t* str, size_t strbytes, wchar_t ch, size_t count);
```

删除字符数组中指定字符, 该位置设置为\0, 只删第一个

```
char* strtok(char* str, char const* del);  
char* strtok_s(char* str, char const* del, char** next);  
//参数 3 是指向后一个字符为首的字符  
wchar_t* wcstok(wchar_t* str, wchar_t const* del, wchar_t** next);  
wchar_t* wcstok_s(wchar_t* str, wchar_t const* del, wchar_t** next);
```

返回参数 1 中第一个不在参数 2 内的字符的下标

```
size_t strspn(char const* str, char const* charset);
```

返回参数 1 中第一个在参数 2 内的字符的下标

```
size_t strcspn(char const* str, char const* charset);  
size_t wcsspn(wchar_t const* str, wchar_t const* charset);  
size_t wcsnspn(wchar_t const* str, wchar_t const* charset);
```

浏览字符串 str 中的字符, 找到在参数 2 中的第一个字符, 返回该地址

```
char *strpbrk(char const* str, char const* charset );  
wchar_t* wcpbrk(wchar_t const* str, wchar_t const* charset );
```

将字符串转换成大写

```
char* strupr(char* str);    //新名字_strupr  
errno_t _strupr_s(char* str, size_t bytes);  
errno_t _strupr_s_l(char* str, size_t bytes, _locale_t locale);  
wchar_t* wcsupr(wchar_t* str);    //新名字_wcsupr
```

```

errno_t _wcsupr_s(wchar_t* str, size_t bytes);
errno_t _wcsupr_s_l(wchar_t* str, size_t bytes, _locale_t locale);

```

将字符串 `strSource` 转换成当前语言环境的字符串，存在 `strDest` 里。比如字符 `a` 在 ASCII 下是 97，在某个地区的字符集下 `a` 是 200，那么该函数就是将 97 的 `a` 转成 200 的 `a`。

```

size_t strxfrm(char* strDest, const char* strSource, size_t count);
size_t _strxfrm_l(char* strDest, const char* strSource, size_t count, _locale_t locale);
size_t wcsxfrm(wchar_t* strDest, const wchar_t* strSource, size_t count);
size_t wcsxfrm_l(wchar_t* strDest, const wchar_t* strSource, size_t count, _locale_t locale);

```

复制字符串，返回值返回复制的字符串的存储空间，该空间由 `malloc` 内在申请。所以返回值使用完需要 `free` 释放。

```

char* strdup(char const* str); //旧名字
char* _strdup(char const* str); //新名字
wchar_t* _wcsdup(wchar_t const* str);

```

翻转字符串

```

char* strrev(char* str);
char* _strrev(char* str); //新名字
wchar_t* _wcsrev(wchar_t* str);

```

获取系统错误信息，其功能、使用规则与 `errno` 是一样，实用工具错误查找

```

char* strerror(int _ErrorMessage); //errno, 获取错误码对应的错误描述
char* _strerror(char const* _ErrorMessage); //在默认描述上，增加自定义描述信息
errno_t strerror_s(char* _Buffer, size_t _SizeInBytes, int _ErrorNumber);
errno_t _strerror_s(char* _Buffer, size_t _SizeInBytes, char const* _ErrorMessage);
wchar_t* _wcserror(int _ErrorNumber);
errno_t _wcserror_s(wchar_t* _Buffer, size_t _SizeInWords, int _ErrorNumber);
wchar_t* __wcserror(wchar_t const* _String);
errno_t __wcserror_s(wchar_t* _Buffer, size_t _SizeInWords, wchar_t const* _ErrorMessage);

```

将 `src` 内的字符复制进 `dst`，直到字符 `val`，如果没有这个字符，那就全复制过去

```

void* memcpy(void* _Dst, void const* _Src, int _Val, size_t _Size); //旧名字
void* _memcpy(void* _Dst, void const* _Src, int _Val, size_t _MaxCount); //新名字

```

参数 4 是字节数

将缓冲区 `src` 的字符拷贝进 `dest` 中，复制 `count` 个

```

void* memcpy(void* dest, const void* src, size_t count);
errno_t memcpy_s(void* dest, size_t destSize, const void* src, size_t count);
wchar_t* wmemcpy(wchar_t* dest, const wchar_t* src, size_t count);
errno_t wmemcpy_s(wchar_t* dest, size_t destSize, const wchar_t* src, size_t count);

```

与 `strncpy` 的区别，`strncpy` 到 `\0` 截止，而 `memcpy` 跟 `count` 有关，不受 `\0` 限制。

`hello\0world`，前者只赋值 `hello`，后者 `memcpy` 全过去了

将缓冲区 src 内的内容移动进 dst,

```
void* memmove(void* _Dst, void const* _Src, size_t _Size);  
errno_t memmove_s(void* dest, size_t numberOfElements, const void* src, size_t count);  
功能效果跟 memcpy 一样, 区别, memcpy 不允许空间重叠, memmove 可以重叠  
wchar_t* wmemmove(wchar_t* dest, const wchar_t* src, size_t count);  
errno_t wmemmove_s(wchar_t* dest, size_t numberOfElements, const wchar_t* src, size_t count);
```

内存设置值, 按字节设置

```
void* memset(void* _Dst, int _Val, size_t _Size);  
wchar_t* wmemset(wchar_t* dest, wchar_t c, size_t count);
```

比较两个缓冲区的字符, 不区分大小写

```
int memicmp(void const* _Buf1, void const* _Buf2, size_t _Size);  
int _memicmp(void const* _Buf1, void const* _Buf2, size_t _Size);  
int _memicmp_l(void const* _Buf1, void const* _Buf2, size_t _Size, _locale_t _Locale);
```

比较两个缓冲区的字符, 区分大小写

```
int memcmp(void const* _Buf1, void const* _Buf2, size_t _Size);  
int wmemcmp(const wchar_t* buffer1, const wchar_t* buffer2, size_t count);
```

查找字符

```
void* memchr(void const* _Buf, int _Val, size_t _MaxCount);  
wchar_t* wmemchr(const wchar_t* buffer, wchar_t c, size_t count);
```

18、<stdio.h>

标准输入输出头文件: standard input output . head

输入输出两个部分:

控制台的输入输出: printf scanf 为主

文件的输入输出: 文件操作为主体

printf 系函数, 输出到控制台函数:

```
int printf(const char* format[, argument]...); //不会检查格式说明符  
int _printf_l(const char* format, _locale_t locale[, argument]...);  
int wprintf(const wchar_t* format[, argument]...);  
int _wprintf_l(const wchar_t* format, _locale_t locale[, argument]...);
```

printf_s(c11)系函数, 输出到控制台函数: 会检查格式说明符的对错

```
int printf_s(const char* format[, argument]...); //会检查格式说明符的对错  
int _printf_s_l(const char* format, _locale_t locale[, argument]...);  
int wprintf_s(const wchar_t* format[, argument]...);  
int _wprintf_s_l(const wchar_t* format, _locale_t locale[, argument]...);
```

sprintf 系函数，输出到字符串中，格式化字符串到 buf 里

```
int sprintf(char* buf, const char* format[, argument] ...);  
int _sprintf_l(char* buf, const char* format, _locale_t locale[, argument] ...);  
int swprintf(wchar_t* buf, size_t count, const wchar_t* format[, argument]...);  
int _swprintf_l(wchar_t* buf, size_t count, const wchar_t* format, _locale_t  
locale[, argument] ...);  
int __swprintf_l(wchar_t* buf, const wchar_t* format, _locale_t locale[, argument] ...);
```

sprintf_s(c11)系函数，输出到字符串中，格式化字符串到 buf 里

```
int sprintf_s(char* buf, size_t Bufbytes, const char* format,...);  
int _sprintf_s_l(char* buf, size_t Bufbytes, const char* format, _locale_t locale,...);  
int swprintf_s(wchar_t* buf, size_t bytes, const wchar_t* format,...);  
int _swprintf_s_l(wchar_t* buf, size_t bytes, const wchar_t* format, _locale_t locale,...);
```

snprintf 系函数，输出到字符串中，格式化字符串到 buf 里，count 个字符

```
int snprintf(char* buf, size_t count, const char* format[, argument] ...); //旧名字  
int _snprintf(char* buf, size_t count, const char* format[, argument] ...); //新名字  
int _snprintf_l(char* buf, size_t count, const char* format, _locale_t  
locale[, argument] ...);  
int _snwprintf(wchar_t* buf, size_t count, const wchar_t* format[, argument] ...);  
int _snwprintf_l(wchar_t* buf, size_t count, const wchar_t* format, _locale_t  
locale[, argument] ...);
```

_snprintf_s 系函数，输出到字符串中，格式化字符串到 buf 里，count 个字符

```
int _snprintf_s(char* buf, size_t bytes, size_t count, const char* format[, argument] ...);  
int _snprintf_s_l(char* buf, size_t bytes, size_t count, const char* format, _locale_t  
locale[, argument] ...);  
int _snwprintf_s(wchar_t* buf, size_t bytes, size_t count, const wchar_t*  
format[, argument] ...);  
int _snwprintf_s_l(wchar_t* buf, size_t bytes, size_t count, const wchar_t* format, _locale_t  
locale[, argument] ...);
```

fprintf 系函数，将格式化字符串输出到文件中

```
int fprintf(FILE* stream, const char* format[, argument]...);  
int _fprintf_l(FILE* stream, const char* format, _locale_t locale[, argument]...);  
int fwprintf(FILE* stream, const wchar_t* format[, argument]...);  
int _fwprintf_l(FILE* stream, const wchar_t* format, _locale_t locale[, argument]...);
```

fprintf_s 系函数，将格式化字符串输出到文件中

```
int fprintf_s(FILE* stream, const char* format[, argument_list]);  
int _fprintf_s_l(FILE* stream, const char* format, _locale_t locale[, argument_list]);  
int fwprintf_s(FILE* stream, const wchar_t* format[, argument_list]);  
int _fwprintf_s_l(FILE* stream, const wchar_t* format, _locale_t locale[, argument_list]);
```


vprintf 系函数，从可变参数中获取数据进行输出到控制台

```
int vprintf(const char* format, va_list argptr);
int _vprintf_l(const char* format, _locale_t locale, va_list argptr);
int vwprintf(const wchar_t* format, va_list argptr);
int _vwprintf_l(const wchar_t* format, _locale_t locale, va_list argptr);
```

举例如下

```
void fun(int a, ...)
{
    va_list ap;
    va_start(ap, a);
    vprintf("%d, %lf", ap);
    va_end(ap);
}
fun(2, 12, 3.4);
```

vprintf_s 系函数，从可变参数中获取数据进行输出到控制台，检测说明符正误，如果有误我们会给报断言。

```
int vprintf_s(const char* format, va_list argptr);
int _vprintf_s_l(const char* format, _locale_t locale, va_list argptr);
int vwprintf_s(const wchar_t* format, va_list argptr);
int _vwprintf_s_l(const wchar_t* format, _locale_t locale, va_list argptr);
```

vsprintf 系函数，从可变参数中获取数据装进字符数组 buffer 中

```
int vsprintf(char* buffer, const char* format, va_list argptr);
int _vsprintf_l(char* buffer, const char* format, _locale_t locale, va_list argptr);
int vswprintf(wchar_t* buffer, size_t count, const wchar_t* format, va_list argptr);
int _vswprintf_l(wchar_t* buffer, size_t count, const wchar_t* format, _locale_t locale, va_list argptr);
int __vswprintf_l(wchar_t* buffer, const wchar_t* format, _locale_t locale, va_list argptr);
```

vsprintf_s 系函数，从可变参数中获取数据装进字符数组 buffer 中，参数 2 是 buffer 字节

```
int vsprintf_s(char* buffer, size_t numberOfElements, const char* format, va_list argptr);
int _vsprintf_s_l(char* buffer, size_t numberOfElements, const char* format, _locale_t locale, va_list argptr);
int vswprintf_s(wchar_t* buffer, size_t numberOfElements, const wchar_t* format, va_list argptr);
int _vswprintf_s_l(wchar_t* buffer, size_t numberOfElements, const wchar_t* format, _locale_t locale, va_list argptr);
```

vsnprintf 系函数，从可变参数中获取 count-1 个字符，装进字符数组 buffer 中

```
int vsnprintf(char* buffer, size_t count, const char* format, va_list argptr);
int _vsnprintf(char* buffer, size_t count, const char* format, va_list argptr);
int _vsnprintf_l(char* buffer, size_t count, const char* format, _locale_t locale, va_list argptr);
```

```

argptr);
int _vsnwprintf(wchar_t* buffer, size_t count, const wchar_t* format, va_list argptr);
int _vsnwprintf_l(wchar_t* buffer, size_t count, const wchar_t* format, _locale_t
locale, va_list argptr);

```

vsnprintf_s 系函数，从可变参数中获取 count-1 个字符，装进字符数组 buffer 中

```

int vsnprintf_s(char* buffer, size_t sizeOfBuffer, size_t count, const char* format, va_list
argptr);
int _vsnprintf_s(char* buffer, size_t sizeOfBuffer, size_t count, const char* format, va_list
argptr);
int _vsnprintf_s_l(char* buffer, size_t sizeOfBuffer, size_t count, const char*
format, _locale_t locale, va_list argptr);
int _vsnwprintf_s(wchar_t* buffer, size_t sizeOfBuffer, size_t count, const wchar_t*
format, va_list argptr);
int _vsnwprintf_s_l(wchar_t* buffer, size_t sizeOfBuffer, size_t count, const wchar_t*
format, _locale_t locale, va_list argptr);

```

fprintf 系函数，从可变参数中获取数据，格式化字符串后装进文件中

```

int fprintf(FILE* stream, const char* format, va_list argptr);
int _fprintf_l(FILE* stream, const char* format, _locale_t locale, va_list argptr);
int fwprintf(FILE* stream, const wchar_t* format, va_list argptr);
int _fwprintf_l(FILE* stream, const wchar_t* format, _locale_t locale, va_list argptr);

```

fprintf_s 系函数，从可变参数中获取数据，格式化字符串后装进文件中

```

int fprintf_s(FILE* stream, const char* format, va_list argptr);
int _fprintf_s_l(FILE* stream, const char* format, _locale_t locale, va_list argptr);
int fwprintf_s(FILE* stream, const wchar_t* format, va_list argptr);
int _fwprintf_s_l(FILE* stream, const wchar_t* format, _locale_t locale, va_list argptr);

```

scanf 系函数，从控制台输入缓冲区拿数据装进变量参数

```

int scanf(const char* format[, argument]...);
int _scanf_l(const char* format, _locale_t locale[, argument]...);
int wscanf(const wchar_t* format[, argument]...);
int _wscanf_l(const wchar_t* format, _locale_t locale[, argument]...);

```

scanf_s 系函数，从控制台输入缓冲区拿数据装进变量参数，安全版本

```

int scanf_s(const char* format[, argument]...);
int _scanf_s_l(const char* format, _locale_t locale[, argument]...);
int wscanf_s(const wchar_t* format[, argument]...);
int _wscanf_s_l(const wchar_t* format, _locale_t locale[, argument]...);

```

sscanf 系函数，从字符串 buffer 拿数据装进变量参数

```
int sscanf(const char* buffer, const char* format[, argument] ...);  
int _sscanf_l(const char* buffer, const char* format, _locale_t locale[, argument] ...);  
int swscanf(const wchar_t* buffer, const wchar_t* format[, argument] ...);  
int _swscanf_l(const wchar_t* buf, const wchar_t* format, _locale_t locale[, argument] ...);
```

sscanf_s 系函数，从字符串 buffer 拿数据装进变量参数，安全版本

```
int sscanf_s(const char* buffer, const char* format[, argument] ...);  
int _sscanf_s_l(const char* buffer, const char* format, _locale_t locale[, argument] ...);  
int swscanf_s(const wchar_t* buffer, const wchar_t* format[, argument] ...);  
int _swscanf_s_l(const wchar_t* buf, const wchar_t* format, _locale_t locale[, argument]...);
```

fscanf 系函数，从文件中拿数据装进变量参数

```
int fscanf(FILE* stream, const char* format[, argument]...);  
int _fscanf_l(FILE* stream, const char* format, _locale_t locale[, argument]...);  
int fwscanf(FILE* stream, const wchar_t* format[, argument]...);  
int _fwscanf_l(FILE* stream, const wchar_t* format, _locale_t locale[, argument]...);
```

fscanf_s 系函数，从文件中拿数据装进变量参数，安全版本

```
int fscanf_s(FILE* stream, const char* format[, argument]...);  
int _fscanf_s_l(FILE* stream, const char* format, _locale_t locale[, argument]...);  
int fwscanf_s(FILE* stream, const wchar_t* format[, argument]...);  
int _fwscanf_s_l(FILE* stream, const wchar_t* format, _locale_t locale[, argument]...);
```

vscanf 系函数，控制台输入进 va_list 的可变参数变量里

```
int vscanf(const char* format, va_list arglist);  
int vwscanf(const wchar_t* format, va_list arglist);  
int vscanf_s(const char* format, va_list arglist);  
int vwscanf_s(const wchar_t* format, va_list arglist);
```

vfscanf 系函数，从文件中输入进 va_list 的可变参数变量里

```
int vfscanf(FILE* stream, const char* format, va_list argptr);  
int vfwscanf(FILE* stream, const wchar_t* format, va_list argptr);  
int vfscanf_s(FILE* stream, const char* format, va_list arglist);  
int vfwscanf_s(FILE* stream, const wchar_t* format, va_list arglist);
```

vsscanf 系函数，从字符串中输入进 va_list 的可变参数变量里

```
int vsscanf(const char* buffer, const char* format, va_list arglist);  
int vswscanf(const wchar_t* buffer, const wchar_t* format, va_list arglist);  
int vsscanf_s(const char* buffer, const char* format, va_list argptr);  
int vswscanf_s(const wchar_t* buffer, const wchar_t* format, va_list arglist);
```

控制台输入 1 个字符

```
int getchar();  
wint_t getwchar();
```

控制台输入 1 个字符，功能同上

```
int getc(FILE* stream);    //参数 stdin  
wint_t getwc(FILE* stream);
```

文件中读取 1 个字符，功能同上

```
int fgetc(FILE* stream);  
wint_t fgetwc(FILE* stream);
```

控制台输入字符串

```
char* gets(char* buffer);    //c11 舍弃了，用 gets_s  
wchar_t* _getws(wchar_t* buffer);  
char* gets_s(char* buffer, size_t sizeInCharacters);  
wchar_t* _getws_s(wchar_t* buffer, size_t sizeInCharacters);
```

文件读取字符串

```
char* fgets(char* str, int n, FILE* stream);  
wchar_t* fgetws(wchar_t* str, int n, FILE* stream);
```

向输入缓冲区 stdin 装一个字符

```
int ungetc(int c, FILE* stream);  
wint_t ungetwc(wint_t c, FILE* stream);
```

控制台输出 1 个字符

```
int putchar(int c);  
wint_t putwchar(wchar_t c);
```

控制台输出 1 个字符，参数 2 : stdout

```
int putc(int c, FILE* stream);  
wint_t putwc(wchar_t c, FILE* stream);
```

控制台输出字符串

```
int puts(const char* str);  
int _putws(const wchar_t* str);
```

将 1 个字符输入进文件里

```
int fputc(int c, FILE* stream);  
wint_t fputwc(wchar_t c, FILE* stream);
```

将 1 个字符串输入进文件里

```
int fputs(const char* str, FILE* stream);  
int fputws(const wchar_t* str, FILE* stream);
```

文件操作

fopen 系函数，打开文件

```
FILE* fopen(const char* filename, const char* mode);  
FILE* _wfopen(const wchar_t* filename, const wchar_t* mode);  
errno_t fopen_s(FILE** pFile, const char* filename, const char* mode);  
errno_t _wfopen_s(FILE** pFile, const wchar_t* filename, const wchar_t* mode);
```

freopen 系函数，重新打开文件，

```
FILE* freopen(const char* path, const char* mode, FILE* stream);  
FILE* _wfreopen(const wchar_t* path, const wchar_t* mode, FILE* stream);  
errno_t freopen_s(FILE** stream, const char* fileName, const char* mode, FILE* oldStream);  
errno_t _wfreopen_s(FILE** str, const wchar_t* file, const wchar_t* mode, FILE* oldStr);
```

关闭指定文件，关闭所有打开中的问题

```
int fclose(FILE* stream);  
int _fcloseall(void);
```

刷新缓冲区，老版本编译器可以清空输入缓冲区，新版本的就没有这个功能了。

```
int fflush(FILE * stream);
```

比如文件写入操作，fputs 之后，数据装在文件缓冲区里，文件操作完毕，数据会写入磁盘，fflush 调用后，会理解将文件写入磁盘。因为文件操作过程不能二次打开，所以咱也看不到具体过程。

设置文件缓冲区为自己的空间

```
void setbuf(FILE * stream, char* buffer); //被 setvbuf 代替  
int setvbuf(FILE * stream, char* buffer, int mode, size_t size);
```

举例

```
char buf[100] = { 0 };  
int a = setvbuf(stdin, buf, _IOLBF, 100);  
int c, d;  
scanf_s("%d%d", &c, &d); //buf 成了输入缓冲区，控制台输入的都在 buf 里存着
```

_IOFBF 全缓冲

输入的数据装满缓冲区，才进行 I/O 操作，比如文件操作，而不是按回车

_IOLBF 行缓冲

数据以回车作为 i/o 操作开始标记，比如 scanf

_IONBF 不使用缓冲区

数据直接进变量，不使用缓冲区

设置文件操作的基准，>0 是按照字符为单位操作，<0 是按照字节为单位操作。

```
int fwide(FILE * stream, int mode);
```

vs 未实现该函数

以二进制数据形式读文件

```
size_t fread(void* buffer, size_t size, size_t count, FILE* stream);
```

以二进制数据形式写文件

```
size_t fwrite(const void* buffer, size_t size, size_t count, FILE* stream);
```

获取文件指针的下标

```
int fgetpos(FILE* stream, fpos_t* pos);
```

设置文件指针的下标

```
int fsetpos(FILE* stream, const fpos_t* pos);
```

获取文件指针的下标

```
long ftell(FILE* stream);
```

```
__int64 _ftelli64(FILE* stream);
```

设置文件指针的下标

```
int fseek(FILE* stream, long offset, int origin);
```

```
int _fseeki64(FILE* stream, __int64 offset, int origin);
```

SEEK_CUR 当前文件位置

SEEK_END 文件结尾位置

SEEK_SET 文件开头位置

设置文件指针指向文件首部，对 stdin 有清空的作用

```
void rewind(FILE* stream);
```

获取文件流操作的错误码，errno 是获取所有标准函数的

```
int ferror(FILE* stream);
```

设置错误描述信息

```
void perror(const char* message);
```

```
void _wperror(const wchar_t* message);
```

重置错误信息

```
void clearerr(FILE* stream);
```

```
errno_t clearerr_s(FILE* stream);
```

检测文件是否到结尾了，到了返回真，不到返回假

```
int feof(FILE* stream);
```

删除文件

```
int remove(const char* path);  
int _wremove(const wchar_t* path);
```

文件重命名

```
int rename(const char* oldname, const char* newname);  
int _wrename(const wchar_t* oldname, const wchar_t* newname);
```

创建临时文件，返回值文件名，配合 fopen 使用

```
char* _tempnam(const char* dir, const char* prefix);  
wchar_t* _wtempnam(const wchar_t* dir, const wchar_t* prefix);  
char* tmpnam(char* str);  
wchar_t* _wtmpnam(wchar_t* str);
```

创建临时文件，直接返回临时文件指针，比上边的方便了

```
FILE* tmpfile(void);  
errno_t tmpfile_s(FILE** pFilePtr);
```

另一类函数，名字带 **nolock**，表示非锁死函数，在多线程操作时，是否对数据加锁，**nolock** 表示不加锁，默认的函数带锁

```
int _fclose_nolock(FILE* _Stream);
```

19、<stdlib.h> 基础工具库: 内存管理、程序工具、字符串转换、随机数、算法

内存管理，在 **malloc.h** 也可以，因为它俩都包含 **<corecrt_malloc.h>** 这个内部头文件，声明都在这里。

申请空间

```
void* malloc(size_t _Size);  
void* calloc(size_t _Count, size_t _Size);
```

重新对已有空间改变大小

```
void* realloc(void* _Block, size_t _Size);
```

释放空间

```
void free(void* _Block);
```

计算 malloc 空间的字节数

```
size_t _msize(void* _Block);
```

以上函数是可移植的，通用的

依据字节对齐申请空间，vs 还不支持这个函数

```
void* aligned_alloc(size_t alignment, size_t size )
```

以指定字节对齐数申请空间, alignment 必须是 2 的 n 次方

该空间的首地址是 alignment 的整数倍

```
void* _aligned_malloc(size_t size, size_t alignment);
void* _aligned_realloc(void* _Block, size_t _Count, size_t _Size, size_t _Alignment);
void* _aligned_realloc(void* _Block, size_t _Size, size_t _Alignment);
void _aligned_free(void* _Block);
size_t _aligned_msize(void* _Block, size_t _Alignment, size_t _Offset);
```

以指定字节对齐数申请空间, alignment 必须是 2 的 n 次方

该空间的首地址+_Offset 是 alignment 的整数倍

```
void* _aligned_offset_malloc(size_t _Size, size_t _Alignment, size_t _Offset);
void* _aligned_offset_realloc(void* Block, size_t Size, size_t Alignment, size_t Offset);
void* _aligned_offset_realloc(void* Block, size_t Count, size_t Size, size_t
Alignment, size_t _Offset);
void _aligned_free(void* _Block);
```

程序工具

终止进程

```
void exit(int const status);
void _Exit(int const status);
void _exit(int const status);
```

参数: 退出状态, 返回给调用对象

```
EXIT_SUCCESS 1    不正常退出
EXIT_FAILURE 0    正常退出
```

exit: 函数调用线程局部对象的析构函数, 然后以后进先出的顺序调用 atexit 和 _onexit 注册的函数, 然后在终止进程之前刷新所有文件缓冲区

_Exit _exit: 函数终止进程而不破坏线程本地对象或处理 atexit 或 _onexit 函数, 并且不刷新流缓冲区。

注册函数, 参数是函数地址, 注册的函数在 exit 调用时执行

```
int atexit(void (__cdecl*)(void));
```

正常终止进程, 他不会给调用对象任何返回, 行为同 exit

```
void quick_exit(int status);
int at_quick_exit(void (__cdecl*)(void));
```

```
_onexit_t __cdecl _onexit(_In_opt_ _onexit_t _Func);
```

```
typedef int (__CRTDECL* _onexit_t)(void);
```

也是注册退出时要调用的函数, 微软拓展

终止进程，返回错误信息，

```
void abort(void);
```

设置 abort 的错误信息

```
unsigned int _set_abort_behavior(unsigned int _Flags, unsigned int _Mask);
```

```
#define _WRITE_ABORT_MSG 0x1 //行为类似 assert，有三个按钮
```

```
#define _CALL_REPORTFAULT 0x2 //显示异常中断窗口
```

传递控制台指令：dos 指令集

```
int system(const char* command);
```

```
int _wsystem(const wchar_t* command);
```

获取当前系统的环境变量设置

```
char* getenv(const char* varname);
```

```
wchar_t* _wgetenv(const wchar_t* varname);
```

```
errno_t getenv_s(size_t* pValue, char* buffer, size_t count, const char* varname);
```

```
errno_t _wgetenv_s(size_t* pValue, wchar_t* buffer, size_t count, const wchar_t* varname);
```

参数 varname 是环境变量的名字，举例如下

```
const char* name = "PATH";  
const char* env = getenv(name);  
printf("%s : %s \n", name, env);
```

环境变量名字：

APPDATA: 应用程序存储数据的目录

OS: 操作系统的名称

PATH: 指定可执行文件的搜索路径，常用的环境目录，比如安装 java，mysql 等要配置这

RANDOM: 返回 0 到 32767 随机数

TIME: 返回当前时间

... 大家自行百度

功能同上，获取环境变量的信息

```
errno_t _dupenv_s(char** buffer, size_t* numberOfElements, const char* varname);
```

```
errno_t _wdupenv_s(wchar_t** buffer, size_t* numberOfElements, const wchar_t* varname);
```

对环境变量添加变量

```
int _putenv(const char* envstring); // "varname=value"
```

```
int _wputenv(const wchar_t* envstring);
```

```
errno_t _putenv_s(const char* varname, const char* value_string);
```

```
errno_t _wputenv_s(const wchar_t* varname, const wchar_t* value_string);
```

在指定的环境变量中查找文件名，参数 3 返回完整路径

```
void _searchenv(const char* filename, const char* varname, char* pathname);
```

```
void _wsearchenv(const wchar_t* filename, const wchar_t* varname, wchar_t* pathname);
```

```
errno_t _searchenv_s(const char* filename, const char* varname, char* pathname, size_t  
count);
```

```
errno_t _wsearchenv_s(const wchar_t* filename, const wchar_t* varname, wchar_t*
```

```
pathname, size_t count);
```

为参数 2 的相对路径，创建一个绝对路径，存放在参数 1 中。默认就是当前的工程目录

```
char* _fullpath(char* absPath, const char* relPath, size_t maxLength);
```

```
wchar_t* _wfullpath(wchar_t* absPath, const wchar_t* relPath, size_t maxLength);
```

创建绝对路径

```
void _makepath(char* path, const char* drive, const char* dir, const char* fname, const char* ext);
```

```
void _wmakepath(wchar_t* path, const wchar_t* drive, const wchar_t* dir, const wchar_t* fname, const wchar_t* ext);
```

```
errno_t _makepath_s(char* path, size_t size, const char* drive, const char* dir, const char* fname, const char* ext);
```

```
errno_t _wmakepath_s(wchar_t* path, size_t size, const wchar_t* drive, const wchar_t* dir, const wchar_t* fname, const wchar_t* ext);
```

参数：

path 完整的路径缓冲区：E:\\mode\\er\\rt\\data.txt

drive 盘符名字 E:\\

dir 包含目录的路径 mode\\er\\rt\\

fname 文件名字，不包括拓展名 data

ext 拓展名 .txt

将完整路径拆分，跟_makepath 函数正好对应

```
void _splitpath(  
    const char* path,  
    char* drive,  
    char* dir,  
    char* fname,  
    char* ext  
);
```

```
void _wsplitpath(  
    const wchar_t* path,  
    wchar_t* drive,  
    wchar_t* dir,  
    wchar_t* fname,  
    wchar_t* ext  
);
```

```
errno_t _splitpath_s(  
    const char* path,  
    char* drive,  
    size_t driveNumberOfElements,  
    char* dir,  
    size_t dirNumberOfElements,  
    char* fname,
```

```

    size_t nameNumberOfElements,
    char* ext,
    size_t extNumberOfElements
);
errno_t _wsplitpath_s(
    const wchar_t* path,
    wchar_t* drive,
    size_t driveNumberOfElements,
    wchar_t* dir,
    size_t dirNumberOfElements,
    wchar_t* fname,
    size_t nameNumberOfElements,
    wchar_t* ext,
    size_t extNumberOfElements);

```

```

void _seterrormode(int _Mode);
void _beep(unsigned _Frequency, unsigned _Duration);
void _sleep(unsigned long _Duration);

```

这三个函数没有了，被以下三个代替，头文件 windows.h

//设置进程错误模式

```

UINT SetErrorMode(UINT uMode);

```

//制造一段声音，参数 1 是频率，参数 2 是持续时间，单位毫秒

```

BOOL Beep(DWORD dwFreq, DWORD dwDuration);

```

//休息停住，参数单位是毫秒

```

VOID Sleep(DWORD dwMilliseconds);

```

字符串转换

将整数字符串转成 int 类型

```

int atoi(const char* str);
int _wtoi(const wchar_t* str);
int _atoi_l(const char* str, _locale_t locale);
int _wtoi_l(const wchar_t* str, _locale_t locale);

```

将整数字符串转成 long 类型

```

long atol(const char* str);
long _atol_l(const char* str, _locale_t locale);
long _wtol(const wchar_t* str);
long _wtol_l(const wchar_t* str, _locale_t locale);

```

将整数字符串转成 long long 类型

```

long long atoll(const char* str);
long long _wtoll(const wchar_t* str);
long long _atoll_l(const char* str, _locale_t locale);
long long _wtoll_l(const wchar_t* str, _locale_t locale);

```

将整数字符串转成 long 类型

```
long strtol(const char* string, char** end_ptr, int base);
long wctol(const wchar_t* string, wchar_t** end_ptr, int base);
long _strtol_l(const char* string, char** end_ptr, int base, _locale_t locale);
long _wctol_l(const wchar_t* string, wchar_t** end_ptr, int base, _locale_t locale);
```

参数 1: 被转换字符串

参数 2: 转换终止位的地址, 比如 123d4a, 指向 d 的地址

参数 3: 指示字符串的进制

参数 4: 语言环境, _create_locale 函数创建

将整数字符串转成 long long 类型

```
long long strtoll(const char* nptr, char** endptr, int base);
long long wctoll(const wchar_t* nptr, wchar_t** endptr, int base);
long long _strtoll_l(const char* nptr, char** endptr, int base, _locale_t locale);
long long _wctoll_l(const wchar_t* nptr, wchar_t** endptr, int base, _locale_t locale);
```

参数 1: 被转换字符串

参数 2: 转换终止位的地址, 比如 123d4a, 指向 d 的地址

参数 3: 指示字符串的进制

参数 4: 语言环境, _create_locale 函数创建

将整数字符串转成 i64 类型

```
__int64 _strtoi64(const char* strSource, char** endptr, int base);
__int64 _wctoi64(const wchar_t* strSource, wchar_t** endptr, int base);
__int64 _strtoi64_l(const char* strSource, char** endptr, int base, _locale_t locale);
__int64 _wctoi64_l(const wchar_t* strSource, wchar_t** endptr, int base, _locale_t locale);
```

将整数字符串转成 ui64 类型

```
unsigned __int64 _strtoui64(const char* nptr, char** endptr, int base);
unsigned __int64 _wcstoui64(const wchar_t* nptr, wchar_t** endptr, int base);
unsigned __int64 _strtoui64_l(const char* nptr, char** endptr, int base, _locale_t locale);
unsigned __int64 _wcstoui64(const wchar_t* nptr, wchar_t** endptr, int base, _locale_t locale);
```

将整数字符串转成 unsigned long 类型

```
unsigned long strtoul(const char* nptr, char** endptr, int base);
unsigned long _strtoul_l(const char* nptr, char** endptr, int base, _locale_t locale);
unsigned long wcstoul(const wchar_t* nptr, wchar_t** endptr, int base);
unsigned long _wcstoul_l(const wchar_t* nptr, wchar_t** endptr, int base, _locale_t locale);
```

参数 1: 被转换字符串

参数 2: 转换终止位的地址, 比如 123d4a, 指向 d 的地址

参数 3: 指示字符串的进制

参数 4: 语言环境, _create_locale 函数创建

将整数字符串转成 unsigned long long 类型

```
unsigned long long strtoull(const char* nptr, char** endptr, int base);  
unsigned long long _strtoull_l(const char* nptr, char** endptr, int base, _locale_t locale);  
unsigned long long wstrtoull(const wchar_t* nptr, wchar_t** endptr, int base);  
unsigned long long _wstrtoull_l(const wchar_t* nptr, wchar_t** endptr, int base, _locale_t locale);
```

参数 1: 被转换字符串

参数 2: 转换终止位的地址, 比如 123d4a, 指向 d 的地址

参数 3: 指示字符串的进制

参数 4: 语言环境, _create_locale 函数创建

将小数字符串转成 double 类型

```
double atof(const char* str);  
double _atof_l(const char* str, _locale_t locale);  
double _wtof(const wchar_t* str);  
double _wtof_l(const wchar_t* str, _locale_t locale);
```

将小数字符串转成 float 类型

```
float strtof(const char* nptr, char** endptr);  
float _strtof_l(const char* nptr, char** endptr, _locale_t locale);  
float wctof(const wchar_t* nptr, wchar_t** endptr);  
float wctof_l(const wchar_t* nptr, wchar_t** endptr, _locale_t locale);
```

参数 1: 被转换字符串

参数 2: 转换终止位的地址, 比如 123.3d4a, 指向 d 的地址

将小数字符串转成 double 类型

```
double strtod(const char* strSource, char** endptr);  
double _strtod_l(const char* strSource, char** endptr, _locale_t locale);  
double wcstod(const wchar_t* strSource, wchar_t** endptr);  
double wcstod_l(const wchar_t* strSource, wchar_t** endptr, _locale_t locale);
```

参数 1: 被转换字符串

参数 2: 转换终止位的地址, 比如 123.3d4a, 指向 d 的地址

将小数字符串转成 long double 类型

```
long double strtold(const char* strSource, char** endptr);  
long double _strtold_l(const char* strSource, char** endptr, _locale_t locale);  
long double wcstold(const wchar_t* strSource, wchar_t** endptr);  
long double wcstold_l(const wchar_t* strSource, wchar_t** endptr, _locale_t locale);
```

参数 1: 被转换字符串

参数 2: 转换终止位的地址, 比如 123.3d4a, 指向 d 的地址

整数转换成字符串，radix 参数是转换后的进制

参数 1: 被转换整数

参数 2: 转换成字符串装这里

参数 3: 转换成字符串的进制

```
char* itoa(int value, char* buffer, int radix); //被舍弃了
```

```
char* _itoa(int value, char* str, int radix);
```

```
errno_t _itoa_s(int value, char* buffer, size_t size, int radix);
```

```
wchar_t* _itow(int value, wchar_t* str, int radix);
```

```
errno_t _itow_s(int value, wchar_t* buffer, size_t size, int radix);
```

```
char* ltoa(long value, char* buffer, int radix); //被舍弃了
```

```
char* _ltoa(long value, char* str, int radix);
```

```
wchar_t* _ltow(long value, wchar_t* str, int radix);
```

```
errno_t _ltoa_s(long value, char* str, size_t sizeOfstr, int radix);
```

```
errno_t _ltow_s(long value, wchar_t* str, size_t sizeOfstr, int radix);
```

```
char* ultoa(unsigned long value, char* buffer, int radix); //被舍弃了
```

```
char* _ultoa(unsigned long value, char* buffer, int radix);
```

```
wchar_t* _ultow(unsigned long value, wchar_t* buffer, int radix);
```

```
errno_t _ultoa_s(unsigned long value, char* buffer, size_t size, int radix);
```

```
errno_t _ultow_s(unsigned long value, wchar_t* buffer, size_t size, int radix);
```

```
char* _i64toa(__int64 value, char* str, int radix);
```

```
errno_t _i64toa_s(__int64 value, char* buffer, size_t size, int radix);
```

```
wchar_t* _i64tow(__int64 value, wchar_t* str, int radix);
```

```
errno_t _i64tow_s(__int64 value, wchar_t* buffer, size_t size, int radix);
```

```
char* _ui64toa(unsigned __int64 value, char* str, int radix);
```

```
errno_t _ui64toa_s(unsigned __int64 value, char* buffer, size_t size, int radix);
```

```
wchar_t* _ui64tow(unsigned __int64 value, wchar_t* str, int radix);
```

```
errno_t _ui64tow_s(unsigned __int64 value, wchar_t* buffer, size_t size, int radix);
```

浮点型转字符串

value: 被转换小数

count: 存几位有效数字

dec: 小数点的位置，下标

sign: 符号，整数 0，负数 1

返回值/buffer 存有效数位字符串

```
char* _ecvt(double value, int count, int* dec, int* sign);
```

```
errno_t _ecvt_s(char* buffer, size_t bytes, double value, int count, int* dec, int* sign);
```

```
char* _fcvt(double value, int count, int* dec, int* sign);
```

```
errno_t _fcvt_s(char* buffer, size_t bytes, double value, int count, int* dec, int* sign);
```

浮点型转字符串

value: 被转换小数

count: 存几位有效数字

buffer: 存完整的小数字符串, 上边那个不带. 这个带

```
char* _gcvt(double value, int digits, char* buffer);
```

```
errno_t _gcvt_s(char* buffer, size_t bytes, double value, int digits);
```

获取某多字节字符的字节数

```
int mblen(const char* mbstr, size_t count);
```

```
int _mblen_l(const char* mbstr, size_t count, _locale_t locale);
```

将 1 个多字节字符, 转成宽字符字符, count 是检测数量, 一般是 2

```
int mbtowc(wchar_t* wchar, const char* mbchar, size_t count);
```

```
int _mbtowc_l(wchar_t* wchar, const char* mbchar, size_t count, _locale_t locale);
```

将 1 个宽字符, 转成多字节字符, 参数 1 装该字符的多字节的字节数

```
int wctomb(char* mbchar, wchar_t wchar);
```

```
int _wctomb_l(char* mbchar, wchar_t wchar, _locale_t locale);
```

```
errno_t wctomb_s(int* pRetVal, char* mbchar, size_t Bytes, wchar_t wchar);
```

```
errno_t _wctomb_s_l(int* pRetVal, char* mbchar, size_t Bytes, wchar_t wchar, _locale_t locale);
```

多字节字符串转成宽字符串

```
size_t mbstowcs(wchar_t* wstr, const char* mbstr, size_t count);
```

```
size_t _mbstowcs_l(wchar_t* wstr, const char* mbstr, size_t count, _locale_t locale);
```

```
errno_t mbstowcs_s(size_t* pReturnValue, wchar_t* wstr, size_t sizeInWords, const char* mbstr, size_t count);
```

```
errno_t _mbstowcs_s_l(size_t* pReturnValue, wchar_t* wstr, size_t sizeInWords, const char* mbstr, size_t count, _locale_t locale);
```

宽字符串转成多字节字符串

```
size_t wcstombs(char* mbstr, const wchar_t* wstr, size_t count);
```

```
size_t _wcstombs_l(char* mbstr, const wchar_t* wstr, size_t count, _locale_t locale);
```

```
errno_t wcstombs_s(size_t* pReturnValue, char* mbstr, size_t sizeInBytes, const wchar_t* wstr, size_t count);
```

```
errno_t _wcstombs_s_l(size_t* pReturnValue, char* mbstr, size_t sizeInBytes, const wchar_t* wstr, size_t count, _locale_t locale);
```

旋转 value, shift 是旋转几位

逆时针转

```
unsigned int _rotl(unsigned int _Value, int _Shift);
```

```
unsigned long _lrotl(unsigned long _Value, int _Shift);
```

```
unsigned __int64 _rotl64(unsigned __int64 _Value, int _Shift);
```

顺时针转

```
unsigned int _rotr(unsigned int _Value, int _Shift);
unsigned long _lrotr(unsigned long _Value, int _Shift);
unsigned __int64 _rotr64(unsigned __int64 _Value, int _Shift);
```

随机数

```
#define RAND_MAX 0x7fff
void srand(unsigned int seed);    //种子
int rand(void);                  //产生随机数
errno_t rand_s(unsigned int* randomValue);
```

随机的应用场景

种子：随机数计算的基数，同一个基数计算得到的随机数是一样的，所以一般要把随机数种子设置成不一样的值，即当前系统时间，time(NULL)

```
srand((unsigned int)time(NULL)); //整个工程仅需在程序开始调用一次
```

种子一旦种下，随它生根发芽

产生随机数：10 个，每个都不一样

```
int i = 0;
for (i = 0; i < 10; i++)
{
    int a = rand();    //0 - 65536
    printf("%d ", a);
}
```

指定范围随机数

比如生成 25~55 的随机数：

第一步：生成 0~30 的随机数 rand()%31

第二步：加 25 rand()%31+25

```
int i = 0;
for (i = 0; i < 10; i++)
{
    int a = rand()%31 + 25;
    printf("%d ", a);
}
```

一堆数中的随机

```
srand((unsigned int)time(NULL));
int arr[10] = { 12, 45, 78, 111, 333, 78, 999, 123, 345, 567 };
int a = rand()%10;    //随机产生 0 - 9，作为下标，下标随机，元素就相当于随机访问
printf("%d ", arr[a]);
```


算法

排序函数:

```
void qsort(  
    void* base,      //被排序的数组  
    size_t number,   //元素个数  
    size_t width,    //元素类型  
    int(__cdecl* compare)(const void*, const void*) //排序函数  
);
```

举例如下

```
int compare(const void* a, const void* b)  
{  
    int aa = *(const int*)a;  
    int bb = *(const int*)b;  
    //升序  
    //if (aa > bb) return 1;  
    //if (aa < bb) return -1;  
    //return 0;  
    //降序  
    if (aa > bb) return -1;  
    if (aa < bb) return 1;  
    return 0;  
}  
  
int compare(const void* a, const void* b) //看似简单点儿的写法  
{  
    return (*(const int*)b - *(const int*)a); //降序  
    return (*(const int*)a - *(const int*)b); //升序  
}  
  
int main(void)  
{  
    int arr[10] = { 3, 1, 5, 3, 6, 7, 4, 2, 8, 1 };  
    qsort(arr, 10, sizeof(int), compare); //可以对结构体排序  
}
```

排序函数(C11):

```
void qsort_s(  
    void* base,      //被排序的数组  
    size_t num,      //元素个数  
    size_t width,    //元素类型  
    int(__cdecl* compare)(void*, const void*, const void*), //排序函数  
    void* context    //附加参数, 传递进比较函数, 我们自己使用, 比如语言环境, 控制升降  
);
```

在数组中查找指定元素，要求数组必须有序

```
void* bsearch(  
    const void* key,  
    const void* base,  
    size_t num,  
    size_t width,  
    int(__cdecl* compare) (const void* key, const void* datum)  
);  
  
void* bsearch_s(  
    const void* key,  
    const void* base,  
    size_t num,  
    size_t width,  
    int(__cdecl* compare) (void*, const void* key, const void* datum),  
    void* context  
);
```

在数组中查找指定元素，数组无需有序。

```
void* _lfind(  
    const void* key,  
    const void* base,  
    unsigned int* num,  
    unsigned int width,  
    int(__cdecl* compare) (const void*, const void*)  
);  
  
void* _lfind_s(  
    const void* key,  
    const void* base,  
    unsigned int* num,  
    size_t size,  
    int(__cdecl* compare) (void*, const void*, const void*),  
    void* context  
);
```

在数组中查找指定元素，数组无需有序。如果没找到，就把该元素接入数组尾部

```
void* _lsearch(  
    const void* key,  
    void* base,  
    unsigned int* num,  
    unsigned int width,  
    int(__cdecl* compare) (const void*, const void*)  
);
```

```

void* _lsearch_s(
    const void* key,
    void* base,
    unsigned int* num,
    size_t size,
    int(__cdecl* compare)(void*, const void*, const void*),
    void* context
);

```

数学函数

整数求绝对值：

```

int      abs   ( int      _Number);
long     labs  ( long     _Number);
long long llabs ( long long _Number);
__int64  _abs64( __int64  _Number);

```

改变字节序，可以用于大小端存储转换

```

unsigned short _byteswap_ushort(unsigned short _Number);
unsigned long  _byteswap_ulong (unsigned long  _Number);
unsigned __int64 _byteswap_uint64(unsigned __int64 _Number);

```

求整求余，参数1 除参数2

```

div_t  div  ( int      _Numerator,  int      _Denominator);
ldiv_t ldiv  ( long     _Numerator,  long     _Denominator);
lldiv_t lldiv( long long _Numerator,  long long _Denominator);

```

```

typedef struct _div_t
{
    int quot; //整数部分
    int rem;  //余数部分
} div_t;

```

20、<stddef.h> 常用宏定义

以下在 error.h 里见过一模一样的

```
int* _errno(void);
#define errno (*_errno())
errno_t _set_errno(int _Value); //一般用在 get 之前，用来对错误码清零
errno_t _get_errno(int* _Value);
```

计算结构体成员的偏移量

```
#define offsetof(s,m) (((size_t)&(((s*)0)->m))
```

举例

```
struct aa
{
    double a;
    float c;
};
int main(void)
{
    struct aa a;
    printf("%zd\n", offsetof(struct aa, c)); //8
}
```

```
extern unsigned long __threadid(void);
#define _threadid (__threadid()) //获取当前线程 ID
extern uintptr_t __threadhandle(void); //获取当前线程句柄
```

21、<stdnoreturn.h> 便利宏 这个头文件 vs2022 不支持了，因为被舍弃了

`_Noreturn` C23 已被弃用

`__declspec(noreturn)` 类似功能有此，修饰函数，表示函数无返回值

22、<stdalign.h> 便利宏 这个头文件 vs2022 不支持了

内容被代替

`alignas` `_Alignas` 类型修饰符，设置单一变量对齐方式，`pack` 设置整体
`alignof` `_Alignof` C11 运算符，获取结构体最大对齐方式

`__alignas_is_defined` 这两个符号未定义
`__alignof_is_defined`

```
#include <stdio.h>
#include <stddef.h>
struct aa
{
    _Alignas(32) int a; //32 字节对齐
    double d;
};
struct bb
```

```

{
    _Alignas(16) char c;
    _Alignas(64) char str[32];
    //最大的 64 字节对齐
};

int main(void)
{
    printf("字节数: sizeof(aa) = %zu\n", sizeof(struct aa));
    printf("字节对齐数: _Alignof(aa) == %zu\n", _Alignof(struct aa));
    printf("字节数: sizeof(bb) = %zu\n", sizeof(struct bb));
    printf("字节对齐数: _Alignof(bb) == %zu\n", _Alignof(struct bb));
}

```

23、<math.h> 常用数学函数

浮点型绝对值

```

double    fabs(double x);
float     fabsf(float x);      //转定义本质调用 fabs
long double fabsl(long double x); //转定义本质调用 fabs

```

计算余数

```

double    fmod(double x, double y);
float     fmodf(float x, float y);
long double fmodl(long double x, long double y);

```

余数函数计算 x/y 的浮点余数 f , $x = i * y + f$ 其中 i 是整数, f 与 x 的符号相同, 并且 f 的绝对值小于 y 的绝对值。

比如: $5.0 / 2.8 == 1 \dots 2.2$

```

double    remainder(double x, double y);
float     remainderf(float x, float y);
long double remainderl(long double x, long double y);

```

余数函数计算 x/y 的浮点余数 r , 使得 $x = n * y + r$, 其中 n 是在值上最接近 x/y 的偶数
比如: $5.0 / 2.8 == 1.7857\dots$ 最接近偶数 2, 所以 n 的值是 2, 所以 r 的值是 -0.6

```

double    remquo(double numer, double denom, int* quo);
float     remquof(float numer, float denom, int* quo);
long double remquol(long double numer, long double denom, int* quo);

```

同 remainder, n 存在 $*quo$ 的传址调用中, 返回值是 r

```

double    fma(double x, double y, double z);
float     fmaf(float x, float y, float z);
long double fmal(long double x, long double y, long double z);

```

$x*y+z$, 以无限精度计算, 然后对结果进行舍入 1 次, 不会因中间舍入而损失任何精度。

```
double    fmax(double x, double y);
float      fmaxf(float x, float y);
long double fmaxl(long double x, long double y);
```

得到两个数的最大值

```
double    fmin(double x, double y);
float      fminf(float x, float y);
long double fminl(long double x, long double y);
```

得到两个数的最小值

```
double    fdim(double x, double y);
float      fdimf(float x, float y);
long double fdiml(long double x, long double y);
```

x 到 y 的正差, 比如 4-1 正差是 3。5-8 的正差是 0。

获得一个静态 nan 的浮点数, 指数全为 1, 尾数最高位是 1。

```
double nan(const char* input);
float nanf(const char* input);
long double nanl(const char* input);
```

vs 下参数忽略。c 网站上是把参数填入尾数

指数运算函数

计算 e 指数, 2.718281828...

```
double    exp(double x);
float      expf(float x);
long double expl(long double x);
```

计算 2 指数

```
double    exp2(double x);
float      exp2f(float x);
long double exp2l(long double x);
```

计算 e 指数减 1

```
double    expm1(double x);
float      expm1f(float x);
long double expm1l(long double x);
```

对数运算函数

计算 e 为底的对数

```
double    log(double x);
float      logf(float x);
long double logl(double x);
```

计算以 10 为底的对数

```
double    log10(double x);  
float     log10f(float x);  
long double log10l(double x);
```

计算以 2 为底的对数

```
double    log2(double x);  
float     log2f(float x);  
long double log2l(long double x);
```

计算以 e 为底(1+x)的对数

```
double log1p(double x);  
float  log1pf(float x);  
long double log1pl(long double x);
```

幂运算函数

x 的 y 次幂

```
double    pow(double x, double y);  
float     powf(float x, float y);  
long double powl(long double x, long double y);
```

x 的开平方

```
double    sqrt(double x);  
float     sqrtf(float x);  
long double sqrtl(long double x);
```

x 的开立方

```
double    cbrt(double x);  
float     cbrtf(float x);  
long double cbrtl(long double x);
```

三角函数

给定直角三角形的两边，计算斜边的长度

```
double    hypot(double x, double y);  
float     hypotf(float x, float y);  
long double hypotl(long double x, long double y);  
double    _hypot(double x, double y);  
float     _hypotf(float x, float y);  
long double _hypotl(long double x, long double y);
```

正弦值计算, 参数是弧度

```
double    sin(double x);  
float     sinf(float x);  
long double sinl(long double x);
```

余弦值计算, 参数是弧度

```
double    cos(double x);  
float     cosf(float x);  
long double cosl(long double x);
```

正切值计算, 参数是弧度

```
double    tan(double x);  
float     tanf(float x);  
long double tanl(long double x);
```

反正弦函数

```
double    asin(double x);  
float     asinf(float x);  
long double asinl(long double x);
```

反余弦函数

```
double    acos(double x);  
float     acosf(float x);  
long double acosl(long double x);
```

反正切函数

```
double    atan(double x);  
float     atanf(float x);  
long double atanl(long double x);
```

反正切函数, 使用正负号来确定象限

```
double    atan2(double y, double x);  
float     atan2f(float y, float x);  
long double atan2l(long double y, long double x);
```

双曲函数

双曲正弦函数

```
double    sinh(double x);  
float     sinhf(float x);  
long double sinhl(long double x);
```

双曲余弦函数

```
double    cosh(double x);  
float     coshf(float x);  
long double coshl(long double x);
```


双曲正切函数

```
double      tanh(double x);  
float       tanhf(float x);  
long double tanhl(long double x);
```

双曲反正弦函数

```
double      asinh(double x);  
float       asinhf(float x);  
long double asinhl(long double x);
```

双曲反余弦函数

```
double      acosh(double x);  
float       acoshf(float x);  
long double acoshl(long double x);
```

双曲反正切函数

```
double      atanh(double x);  
float       atanhf(float x);  
long double atanhl(long double x);
```

高斯误差函数与伽马函数

返回 x 的高斯误差

```
double      erf(double x);  
float       erff(float x);  
long double erfl(long double x);
```

返回 x 的互补高斯误差

```
double      erfc(double x);  
float       erfcf(float x);  
long double erfc1(long double x);
```

返回 x 的 gamma

```
double      tgamma(double x);  
float       tgammaf(float x);  
long double tgamma1(long double x);
```

返回 x 的 gamma 函数绝对值的自然对数。

```
double      lgamma(double x);  
float       lgammaf(float x);  
long double lgamma1(long double x);
```

浮点型的最近整数

大于等于 x 的最小整数 向上舍入

```
double    ceil(double x);
float     ceilf(float x);
long double ceill(long double x);
```

小于等于 x 的最大整数 向下舍入

```
double    floor(double x);
float     floorf(float x);
long double floorl(long double x);
```

x 的最近整数, 向 0 舍入, -13.x -> -13 13.x -> 13

```
double    trunc(double x);
float     truncf(float x);
long double truncf(long double x);
```

将浮点型舍入为整数, 四舍五入(向偶)入, 默认舍入

```
double    nearbyint( double x );
float     nearbyintf(float x);
long double nearbyintl(long double x);
```

将浮点值四舍五入为最接近的整数值, 纯的, 代码实现

```
double    round(double x);
float     roundf(float x);
long double roundl(long double x);
long      lround(double x);
long      lroundf(float x);
long      lroundl(long double x);
long long llround(double x);
long long llroundf(float x);
long long llroundl(long double x);
```

使用当前舍入模式和方向将指定的浮点值舍入到最接近的整数值, 随模式变化

```
double    rint(double x);
float     rintf(float x);
long double rintl(long double x);
long int   lrint(double x);
long int   lrintf(float x);
long int   lrintl(long double x);
long long int llrint(double x);
long long int llrintf(float x);
long long int llrintl(long double x);
```

浮点型操作函数

获取浮点数的尾数与指数，指数是 2 进制的指数

```
double      frexp(double x, int* expptr);
float       frexpf(float x, int* expptr);
long double frexpl(long double x, int* expptr);
```

x 乘以 2 的 exp 次方

```
double      ldexp(double x, int exp);
float       ldexpf(float x, int exp);
long double ldexpl(long double x, int exp);
```

将浮点数拆分成整数部分与小数部分

```
double      modf(double x, double* intptr);
float       modff(float x, float* intptr);
long double modfl(long double x, long double* intptr);
```

x 乘以 FLT_RADIX(二进制系统是 2)的 exp 次方

```
double      scalbn(double x, int exp);
float       scalbnf(float x, int exp);
long double scalbnl(long double x, int exp);
double      scalbln(double x, long exp);
float       scalblnf(float x, long exp);
long double scalblnl(long double x, long exp);
```

2 的 a 次方是 x，求 a

```
int  ilogb(double x);
int  ilogbf(float x);
int  ilogbl(long double x);
```

提取浮点型的指数值，2 为底的

```
double      logb(double x);
float       logbf(float x);
long double logbl(long double x);
```

返回下一个可表示的浮点值，x 表示起始，y 表示趋势

```
double      nextafter(double x, double y);
float       nextafterf(float x, float y);
long double nextafterl(long double x, long double y);
double      nexttoward(double x, long double y);
float       nexttowardf(float x, long double y);
long double nexttowardl(long double x, long double y);
```

符号拷贝，将参数 2 的符号拷贝给参数 1

```
double      copysign(double x, double y);
double      _copysign(double x, double y);
long double _copysignl(long double x, long double y);
```

浮点型识别函数

```
int a = fpclassify(x);    //宏，识别浮点型的状态类型

#define FP_INFINITE  _INFCODE    无穷的        1.2/0.0
#define FP_NAN       _NANCODE    nan           sqrt(-1)
#define FP_NORMAL    _FINITE     普通的        1.3
#define FP_SUBNORMAL _DENORM     非规格化数    4.9406564584124654e-324
#define FP_ZERO      0           正负0         1/(1.2e300*1.2e300)

int b = isfinite(2.3);    //宏，是否有穷的，是返回1，否则返回0
int c = isinf(1.2);       //是否是无穷
int d = isnan(3.1);       //是否是非数 nan
int e = isnormal(23.3);   //是否是普通数
int f = signbit(12.3);    //是否是负数，不是返回0
int g = isgreater(x, y);  // x > y?
int h = isgreaterequal(x, y); // x >= y ?
int i = isless(x, y);     // x < y ?
int j = islessequal(x, y); // x <= y ?
int k = islessgreater(x, y); // x > y || x < y ?
int l = isunordered(x, y); //判断两个数是否有非正常数
```

24、<complex.h> 复数操作库

复数: $z = a + bi$

a 是实部, b 是虚部, a,b 是实数, 即浮点型, 计算时就是用 a,b 运算

C 语言实现复数形式, 就是定义一个 2 元素的浮点型数组, 来存储 a,b, 如下

```
typedef struct _C_double_complex
{
    double _Val[2];
} _C_double_complex;    //虚实都是 double 类型的

typedef struct _C_float_complex
{
    float _Val[2];
} _C_float_complex;     //虚实都是 float 类型的

typedef struct _C_ldouble_complex
{
    long double _Val[2];
} _C_ldouble_complex;   //虚实都是 long double 类型的

typedef _C_double_complex _Dcomplex;
typedef _C_float_complex  _Fcomplex;
typedef _C_ldouble_complex _Lcomplex; //三个重命名,

_Dcomplex a = { 12, 4 }; //定义一个复数变量, 实部是 12, 虚部是 4
```

复数操作函数如下:

计算复数的大小

```
double      cabs( _Dcomplex _Z);  
float       cabsf( _Fcomplex _Z);  
long double cabsl( _Lcomplex _Z);
```

计算复数余弦

```
_Dcomplex  ccos( _Dcomplex _Z);  
_Fcomplex  ccosf( _Fcomplex _Z);  
_Lcomplex  ccosl( _Lcomplex _Z);
```

计算复数反余弦

```
_Dcomplex  cacos( _Dcomplex _Z);  
_Fcomplex  cacosf( _Fcomplex _Z);  
_Lcomplex  cacosl( _Lcomplex _Z);
```

计算复双曲余弦

```
_Dcomplex  ccosh( _Dcomplex _Z);  
_Fcomplex  ccoshf( _Fcomplex _Z);  
_Lcomplex  ccoshl( _Lcomplex _Z);
```

计算复数反双曲余弦

```
_Dcomplex  cacosh( _Dcomplex _Z);  
_Fcomplex  cacoshf( _Fcomplex _Z);  
_Lcomplex  cacoshl( _Lcomplex _Z);
```

计算复数正弦

```
_Dcomplex  csin( _Dcomplex _Z);  
_Fcomplex  csinf( _Fcomplex _Z);  
_Lcomplex  csinl( _Lcomplex _Z);
```

计算复数反正弦

```
_Dcomplex  casin( _Dcomplex _Z);  
_Fcomplex  casinf( _Fcomplex _Z);  
_Lcomplex  casinl( _Lcomplex _Z);
```

计算复数双曲正弦

```
_Dcomplex  csinh( _Dcomplex _Z);  
_Fcomplex  csinhf( _Fcomplex _Z);  
_Lcomplex  csinhl( _Lcomplex _Z);
```

计算复数反双曲正弦

```
_Dcomplex  casinh( _Dcomplex _Z);  
_Fcomplex  casinhf( _Fcomplex _Z);  
_Lcomplex  casinhl( _Lcomplex _Z);
```

计算复数正切

```
_Dcomplex ctan( _Dcomplex _Z);  
_Fcomplex ctanf( _Fcomplex _Z);  
_Lcomplex ctanl( _Lcomplex _Z);
```

计算复数反正切

```
_Dcomplex catan( _Dcomplex _Z);  
_Fcomplex catanf( _Fcomplex _Z);  
_Lcomplex catanl( _Lcomplex _Z);
```

计算复数双曲正切

```
_Dcomplex ctanh( _Dcomplex _Z);  
_Fcomplex ctanhf( _Fcomplex _Z);  
_Lcomplex ctanhl( _Lcomplex _Z);
```

计算复数反双曲正切

```
_Dcomplex catanh( _Dcomplex _Z);  
_Fcomplex catanhf( _Fcomplex _Z);  
_Lcomplex catanhl( _Lcomplex _Z);
```

计算复数的以 e 为底的指数

```
_Dcomplex cexp( _Dcomplex _Z);  
_Fcomplex cexpf( _Fcomplex _Z);  
_Lcomplex cexpl( _Lcomplex _Z);
```

计算一个复数的虚部

```
double      cimag( _Dcomplex _Z);  
float       cimagf( _Fcomplex _Z);  
long double cimagl( _Lcomplex _Z);
```

计算复数自然对数, e 为底

```
_Dcomplex clog( _Dcomplex _Z);  
_Fcomplex clogf( _Fcomplex _Z);  
_Lcomplex clogl( _Lcomplex _Z);
```

计算复数对数, 10 为底

```
_Dcomplex clog10( _Dcomplex _Z);  
_Fcomplex clog10f( _Fcomplex _Z);  
_Lcomplex clog10l( _Lcomplex _Z);
```

计算复数的相位角

```
double      carg( _Dcomplex _Z);  
float       cargf( _Fcomplex _Z);  
long double cargl( _Lcomplex _Z);
```

计算共轭复数

```
_Dcomplex    conj( _Dcomplex _Z);  
_Fcomplex    conjf( _Fcomplex _Z);  
_Lcomplex    conjl( _Lcomplex _Z);
```

检索一个数字的指定幂的值，其中基数和指数是复数，此函数具有沿负实轴的指数的分支切割

```
_Dcomplex    cpow( _Dcomplex _X, _Dcomplex _Y);  
_Fcomplex    cpowf( _Fcomplex _X, _Fcomplex _Y);  
_Lcomplex    cpowl( _Lcomplex _X, _Lcomplex _Y);
```

计算黎曼球面上的投影

```
_Dcomplex    cproj( _Dcomplex _Z);  
_Fcomplex    cprojf( _Fcomplex _Z);  
_Lcomplex    cprojl( _Lcomplex _Z);
```

计算复数的实部

```
double        creal( _Dcomplex _Z);  
float         crealf( _Fcomplex _Z);  
long double   creall( _Lcomplex _Z);
```

计算复数平方根

```
_Dcomplex    csqrt( _Dcomplex _Z);  
_Fcomplex    csqrtf( _Fcomplex _Z);  
_Lcomplex    csqrtl( _Lcomplex _Z);
```

检索复数的平方幅度

```
double        norm( _Dcomplex _Z);  
float         normf( _Fcomplex _Z);  
long double   norml( _Lcomplex _Z);
```

从实部和虚部构造复数

```
_Dcomplex    Cbuild( double _Re, double _Im);  
_Fcomplex    FCbuild( float _Re, float _Im);  
_Lcomplex    LCbuild( long double _Re, long double _Im);
```

将两个复数相乘

```
_Dcomplex    Cmulcc( _Dcomplex _X, _Dcomplex _Y);  
_Fcomplex    FCmulcc( _Fcomplex _X, _Fcomplex _Y);  
_Lcomplex    LCmulcc( _Lcomplex _X, _Lcomplex _Y);
```

将复数乘以浮点数

```
_Dcomplex    Cmulcr( _Dcomplex _X, double _Y);  
_Fcomplex    FCmulcr( _Fcomplex _X, float _Y);  
_Lcomplex    LCmulcr( _Lcomplex _X, long double _Y);
```

25、<tgmath.h> 泛型数学，仅包含了

```
#include <math.h>
#include <complex.h>
```

为啥叫泛型？因为这个头文件可算一切数了，实数，复数，应用广泛

26、<setjmp.h> 非局部跳转

goto 是在局部范围内跳来跳去，这头文件里的宝贝可以跨区域跳

goto 结构就是跳到标签行继续执行，如下：

```
int main(void)
{
    printf("第 1 行\n");
    goto step1;
    printf("第 2 行\n");
    printf("第 3 行\n");
step1: //标签，跟 switch 位域的标签意义一样
    printf("第 4 行\n");
}
```

输出：第 1 行

goto 直接跳到 step1，中间两个 printf 跳过了

输出：第 4 行

往前跳

```
int main(void)
{
    printf("第 1 行\n");
step1:
    printf("第 2 行\n");
    printf("第 3 行\n");
    goto step1;
    printf("第 4 行\n");
}
```

输出：第 1 行

输出：第 2 行

输出：第 3 行

goto 跳到 step1

输出：第 2 行

输出：第 3 行

goto 跳到 step1

输出：第 2 行

输出：第 3 行

goto 跳到 step1

输出：第 2 行

输出：第 3 行

...死循环了。

这个跳跃是局部的跳跃，不能跨函数，如下：


```

void fun(void)
{
    printf("第 1 行\n");
step1:
    printf("第 2 行\n");
}

int main(void)
{
    printf("第 3 行\n");
    goto step1; //报错，未定义
    printf("第 4 行\n");
}

```

标签仅仅是个执行起始点，本身没有效果。

<setjmp.h>头文件里：

```

#define _JBLEN 16
#define _JBTYPE int
typedef _JBTYPE jmp_buf[_JBLEN];

```

所以 jmp_buf 是 16 元素 int 类型的数组

```

int setjmp(jmp_buf _Buf); //设置跳点
void longjmp(jmp_buf _Buf, int _Value); //跳

```

举例

1、一定要先执行 setjmp 设置好跳点

2、相应逻辑调用 longjmp 跳到跳点，参数 2 的值会通过 setjmp 的返回值返回

```

jmp_buf buf;
int i = 0;
void fun(void)
{
    i++;
    longjmp(buf, i);
    printf("fun\n");
}

int main(void)
{
    printf("第 1 行\n");
    int a = setjmp(buf); //设置跳点，
    if (3 > a) //当 longjmp 的传来的值为 5 时，不执行 fun()
    {
        printf("while %d\n", a);
        fun(); //函数内跳跃到跳点
    }
    printf("第 2 行\n");
}

```

27、<signal.h> 信号处理头文件

信号就是信息，程序运行出现某种特殊状况，及时报告给系统。然后根据需要进行处理。

设置信号处理函数

```
_crt_signal_t signal( int _Signal, _crt_signal_t _Function);
```

参数 1：信号标识符，下面有列表。信号有很多种，vs 这个头文件就支持这个几个，不同的编译环境，支持的信号有不同。另外其基本含义可能也有差别，基于语言环境的实现。

参数 2：信号处理函数。即当该信号出现时，采用这个函数处理。函数类型：

```
typedef void (* _crt_signal_t)(int); 函数内容自定义，函数头必须是这个类型
```

C 语言提供的默认处理函数，没什么意思，没什么处理

```
#define SIG_DFL ((_crt_signal_t)0)    // 默认处理
#define SIG_IGN ((_crt_signal_t)1)    // 无视信号
#define SIG_GET ((_crt_signal_t)2)    // 返回信号
#define SIG_SGE ((_crt_signal_t)3)    // 错误信号
#define SIG_ACK ((_crt_signal_t)4)    // 阅
```

主动触发指定信号，以下信号在其条件达成时被动产生，C 语言也提供了主动产生信号的函数

```
int raise(int _Signal);
```

产生信号后，可以自动调用 signal 的参数 2，信号处理函数

我们可以自定义信号处理逻辑与方法

```
#define SIGINT      2    // 控制台中断 ctrl+c
#define SIGILL      4    // 无效的指令，比如 A、B 进程共用一个文件，A 进程在使用的过程中，被 B 进程更改并覆盖，这时 A 进程可能会出现这个错误信号
#define SIGFPE      8    // 错误的算数运算，溢出、被零除或无效操作。
                        // vs 下直接强制中断，走不到处理函数那一步
#define SIGSEGV     11   // 非法内存操作，比如野指针的写入
#define SIGTERM     15   // 向程序发送终止请求，发了各种终止，也没看到效果
#define SIGBREAK    21   // ctrl+break 键盘上 pause break 按键
#define SIGABRT     22   // abort 调用触发异常终止
#define SIGABRT_COMPAT 6 // 同 SIGABRT，以兼容其他平台
```

28、<threads.h> (C11) 线程库

这个头文件 vs2022 的 C 语言不支持，C++ 支持 <thread> 头文件，内部包含线程的各种操作，C++ 全面学习里，咱们会详细介绍这个。

那 windows 里 C 语言怎么创建线程啊？

有专属的线程操作的一整套的 win32API，windows.h 高级应用课里讲

```
int thrd_create(thrd_t * thr, thrd_start_t func, void* arg);
```

创建线程，win32API: CreateThread

返回值：成功返回 thrd_success，失败返回 thrd_nomem

参数 1：返回线程 ID

参数 2：线程函数

参数 3：给线程函数传递的自定义数据

```
int thrd_equal(thrd_t lhs, thrd_t rhs);
```

判断两个线程 ID 是否是一样的，即出自同一个线程

```
thrd_t thrd_current(void);
```

获取当前线程 ID, win21API : GetCurrentThread

```
int thrd_sleep(const struct timespec* duration, struct timespec* remaining);
```

线程挂起，阻塞线程执行，直到时间结束，

win32API: SuspendThread(暂停) ResumeThread(恢复) Sleep

```
void thrd_yield(void);
```

让出时间片给同优先级的线程

```
void thrd_exit(int res);
```

退出线程，参数是退出码，比如 exit(1)

win32API: ExitThread TerminateThread

```
int thrd_detach(thrd_t thr);
```

独立线程，当期结束时，其资源都释放

```
int thrd_join(thrd_t thr, int* res);
```

阻塞当前线程，直到 thr 线程执行完毕

29、<stdatomic.h>(c11) 原子操作库

is not yet supported when compiling as C, but this is planned for a future release

C++里有 atomic 的库，所以到 C++ 的教程里，咱们会详细的讲解测试。

暂时就依据 cpp 网站上的内容，给大家翻译一下：

什么是原子操作？

原子，就是很小的粒子，意为不可分割的最小的整体，在编程里就是该代码(原子操作)是不可分割的，必须执行完毕不能被中断，或者一点儿都不执行。

比如多线程中最同一个变量 a 进行自加 1 操作

A 线程对 a 自加的过程中，线程 B 也可能正在执行 a 自加，两个或者多个线程同时对同一个变量自加，导致，每个线程里得到的 a 自加后的值都是不确定的，造成了程序逻辑的不可预知性，所以我们希望，当 A 线程对 a 自加时，其他线程不要动，等我自加完了，你们再依次自加。这时就可以把 a 声明成原子变量，使用原子自加函数。这样，每次自加都是不可中断的，必须完整的一个自加过程，别的线程只能等待。当然可以对变量 a 加锁，实现相同的逻辑。

加锁与原子的区别？

原子操作是硬件支持的操作，加锁是软件层面的逻辑控制，所以一般来讲，原子实现的操作速度要快一些。

原子类型

<code>atomic_bool(C11)</code>	<code>_Atomic _Bool</code>
<code>atomic_char(C11)</code>	<code>_Atomic char</code>
<code>atomic_schar(C11)</code>	<code>_Atomic signed char</code>
<code>atomic_uchar(C11)</code>	<code>_Atomic unsigned char</code>
<code>atomic_short(C11)</code>	<code>_Atomic short</code>
<code>atomic_ushort(C11)</code>	<code>_Atomic unsigned short</code>
<code>atomic_int(C11)</code>	<code>_Atomic int</code>
<code>atomic_uint(C11)</code>	<code>_Atomic unsigned int</code>
<code>atomic_long(C11)</code>	<code>_Atomic long</code>
<code>atomic_ulong(C11)</code>	<code>_Atomic unsigned long</code>
<code>atomic_llong(C11)</code>	<code>_Atomic long long</code>
<code>atomic_ullong(C11)</code>	<code>_Atomic unsigned long long</code>
<code>atomic_char8_t(C23)</code>	<code>_Atomic char8_t</code>
<code>atomic_char16_t(C11)</code>	<code>_Atomic char16_t</code>
<code>atomic_char32_t(C11)</code>	<code>_Atomic char32_t</code>
<code>atomic_wchar_t(C11)</code>	<code>_Atomic wchar_t</code>
<code>atomic_int_least8_t(C11)</code>	<code>_Atomic int_least8_t</code>
<code>atomic_uint_least8_t(C11)</code>	<code>_Atomic uint_least8_t</code>
<code>atomic_int_least16_t(C11)</code>	<code>_Atomic int_least16_t</code>
<code>atomic_uint_least16_t(C11)</code>	<code>_Atomic uint_least16_t</code>
<code>atomic_int_least32_t(C11)</code>	<code>_Atomic int_least32_t</code>
<code>atomic_uint_least32_t(C11)</code>	<code>_Atomic uint_least32_t</code>
<code>atomic_int_least64_t(C11)</code>	<code>_Atomic int_least64_t</code>
<code>atomic_uint_least64_t(C11)</code>	<code>_Atomic uint_least64_t</code>
<code>atomic_int_fast8_t(C11)</code>	<code>_Atomic int_fast8_t</code>
<code>atomic_uint_fast8_t(C11)</code>	<code>_Atomic uint_fast8_t</code>
<code>atomic_int_fast16_t(C11)</code>	<code>_Atomic int_fast16_t</code>
<code>atomic_uint_fast16_t(C11)</code>	<code>_Atomic uint_fast16_t</code>
<code>atomic_int_fast32_t(C11)</code>	<code>_Atomic int_fast32_t</code>
<code>atomic_uint_fast32_t(C11)</code>	<code>_Atomic uint_fast32_t</code>
<code>atomic_int_fast64_t(C11)</code>	<code>_Atomic int_fast64_t</code>
<code>atomic_uint_fast64_t(C11)</code>	<code>_Atomic uint_fast64_t</code>
<code>atomic_intptr_t(C11)</code>	<code>_Atomic intptr_t</code>
<code>atomic_uintptr_t(C11)</code>	<code>_Atomic uintptr_t</code>
<code>atomic_size_t(C11)</code>	<code>_Atomic size_t</code>
<code>atomic_ptrdiff_t(C11)</code>	<code>_Atomic ptrdiff_t</code>
<code>atomic_intmax_t(C11)</code>	<code>_Atomic intmax_t</code>
<code>atomic_uintmax_t(C11)</code>	<code>_Atomic uintmax_t</code>

原子变量操作函数

`_Bool atomic_is_lock_free(const volatile A * obj);`

判断原子对象是否是无锁的

```
void atomic_store(volatile A * obj, C desired);  
void atomic_store_explicit(volatile A * obj, C desired, memory_order order);
```

原子对象赋值

```
C atomic_load(const volatile A * obj);  
C atomic_load_explicit(const volatile A * obj, memory_order order);
```

从原子对象中读取值

```
C atomic_exchange(volatile A * obj, C desired);  
C atomic_exchange_explicit(volatile A * obj, C desired, memory_order order);
```

用原子对象的值交换一个值

```
atomic_compare_exchange_strong  
atomic_compare_exchange_strong_explicit  
atomic_compare_exchange_weak  
atomic_compare_exchange_weak_explicit
```

如果旧值符合预期，则将值与原子对象交换，否则读取旧值

```
atomic_fetch_add  
atomic_fetch_add_explicit
```

原子加法

```
atomic_fetch_sub  
atomic_fetch_sub_explicit
```

原子减法

```
atomic_fetch_or  
atomic_fetch_or_explicit
```

原子按位或

```
atomic_fetch_xor  
atomic_fetch_xor_explicit
```

原子按位异或

```
atomic_fetch_and  
atomic_fetch_and_explicit
```

原子按位与

标准是建议要设计这些功能，实际编译器厂商会设计出更多的功能，对于标准给定的一些功能有代替的功能，拓展的功能，所以大家的平时应用，要根据自己的知识，学以致用，随机应变。

/标准简介 <https://en.cppreference.com/w/>

1967 年，剑桥大学的马丁·理查兹 (Martin Richards) 对 CPL (CPL 是一种非常接近硬件的语言。非常复杂，实现困难) 语言进行了简化，设计出了 BCPL(Basic Combined Programming Language, 基本组合编程语言)语言。

1970 年，肯·汤普森 (Ken Thompson) 以 BCPL 语言为基础，设计出了 B 语言(BCPL 首字母)。随即用 B 语言写出了 UNIX 操作系统。

1972 年，丹尼斯·里奇(Dennis M.Ritchie)在 B 语言基础上重新设计，创造出了 C 语言，取 BCPL 的第二个字母 C。丹尼斯·里奇被称为 C 语言之父。

1973 年，丹尼斯·里奇与肯·汤普森用 C 语言重写了 UNIX 操作系统。

1978 年，布莱恩·凯尼根 (Brian W.Kernighan) 与丹尼斯·里奇(Dennis M.Ritchie)出版了第一版《C Programming Language》。

1988 年出版了第二版《C Programming Language》，这本书被称为 C 语言圣经，此书中定义的 C 语言标准被称为 K&R C。K、R 是两人名字首字母。当然大家不用买这本书看，这是最早期的 C 语言了，现今的 C 语言标准变化了很多。

1989 年，美国国家标准协会 ANSI 发布了 C 语言的完整标准，后被称为 ANSI C 或 C89。

1990 年，国际标准化组织 ISO 采纳 ANSI C，并发表了第一个 C 语言国际标准，后被称为 C90。大家看一些资料中描述 C89、ANSI C、C90 其实就是同一个。

提示：ISO 网站中 C 语言标准编号为 ISO/IEC 9899，C++为 ISO/IEC 14882。

1999 年，ISO 发布了 ISO/IEC 9899:1999 标准，简称 C99。

2011 年，ISO 发布了 ISO/IEC 9899:2011 标准，简称 C11。

2018 年，ISO 发布了 ISO/IEC 9899:2018 标准，简称 C18/C17。

2023 年，ISO/IEC 9899:2023 标准将发布，简称 C23。

新标准就是在上一代标准上拓展了新的内容，优化了漏洞，而不是取代。

C99 标准介绍

1、移除隐式的 int

比如

```
main(void) { }
```

即函数没有写返回值类型，此时编译器默认识别为 int，以 int 类型返回值处理。旧式写法

C99 标准移除该写法，希望大家使用明确的返回值类型，如下：

```
int main(void) { return 0; }
```

但是现代变 C 语言编译器并没移除旧式写法，依然支持，目的是保持向下兼容性，因为依然存在着大量的旧式写法的代码，为了保证其可用性，所以依然支持，可能未来某个时间点，就彻底遗弃了。比如 C++，完全不支持改写法。

```
main() { return 0; }
```

```
//error C4430: 缺少类型说明符 - 假定为 int。注意：C++ 不支持默认 int
```

所以说，标准里的一些内容，属于建议性质的，是否采用，取决于厂商。其次，建议大家使用标准版的主函数，而不是执着于(我爱咋写就咋写)

2、gets 函数的移除

在 vs2022 中使用该函数输入字符串时，报错了，使用新函数 gets_s 取代。

老版本编译器还可以继续使用，比如 vs2005

```
char* gets(char* str);
```

```
char* gets_s(char* str, rsize_t n);
```

返回值：str 的首地址

参数 1：字符数组

参数 2：实际输入字符最多为 n-1，给\0 留个位置，不然异常中断。

3、标识符的名字

长度由任意长的数字，下划线，大小写字母组成，首字符不能是数字。

新增 \U 形式字符作变量名，如下：

```
int 包 = 12;           //中文变量名，
int \U00005305 = 12;  //如此表示一个字符
int \u5305 = 12;
```

\U 表示 unicode 字符集，其后跟 8 位 16 进制数

\u 表示 unicode 字符集，其后跟 4 位 16 进制数

16 进制数 5303 就是汉字包对应的编号，必须凑够 8 位，所以前面补 0000

当然这种写法，可能没人用

unicode 字符是国际通用的字符集，内装国际通用的大多数字符，大家可以搜索“unicode 字符集在线转换”，就能得到每个汉字对应的 unicode 编码了，当然，不是所有汉字都支持

4、注释

C 语言的老注释：/* */

C99 标准增加了单行注释：//

C 借鉴了 C++，因为 C++ 设计初衷之一就是要兼容 C 语言语法，所以在基本语法层面，二者尽量都会保持一致，互相借鉴。

5、新增类型

_Bool 布尔类型，逻辑真假，即 0 与 1

```
_Bool a = 12; //输出 a 的值是 1
<stdbool.h>为 _Bool 定义新宏名 bool 并定义 1 为 true，0 为 false
```

long long 与 unsigned long long 即 64 位的有无符号的整形。vc++6.0 不支持这个类型。

```
long long          long long int          signed long long int
unsigned long long unsigned long long int
```

__int8	有符号 8 位整型	unsigned __int8	无符号 8 位整型	%hhd	%hhu
__int16	有符号 16 位整型	unsigned __int16	无符号 16 位整型	%hd	%hu
__int32	有符号 32 位整型	unsigned __int32	无符号 32 位整型	%d	%u
__int64	有符号 64 位整型	unsigned __int64	无符号 64 位整型	%ld	%lu
__int128	有符号 128 位整型	unsigned __int128	无符号 128 位整型	%lld	%llu

128 的这个类型取决于编译环境，有的实现了，有的没实现，vs2022 还没支持

size_t

```
#ifdef _WIN64
    typedef unsigned __int64 size_t;      %zu %zd %g
    typedef __int64          ptrdiff_t;
    typedef __int64          intptr_t;
#else
    typedef unsigned int      size_t;
    typedef int               ptrdiff_t;
    typedef int               intptr_t;
#endif
```

复数类型:

均在<complex.h>内介绍了, 大家可以看看标准库部分的课程讲解

虚数类型: 就是专门研究复数的虚数部分, 网站上的案例是使用 complex 头文件, 但是 vs2022 还不支持这个类型

6、柔性数组

//定义数组时, 如果初始化状态下, 那么元素个数可以不写

//也就是数组的大小根据初始化数据个数自动确定

```
int a[] = { 1, 2, 3, 4, 5, 6 };
```

```
int b[] = { 1, 2, 3 };
```

C99 在结构体里的实现了类似的功能, 叫柔性数组, 如下:

```
struct Node
{
    int a;
    short b[5];    //只能是普通数组
    double c;
    float d[];    //柔性数组, 用来拓展结构体的空间, 必须在最后
};
```

特点:

- 1、必须放在结构体最后一个成员, 不能写元素个数, 或者写 0
- 2、不占结构体空间, 大小还是 16, d 不计算在内, 多大都不算
- 3、一般思维使用:

```
struct Node n = { 12, {3, 4}, 2.5, {1, 2, 3, 4, 5, 6} };
```

```
printf("%d", n.d[5]);
```

那么 d 就是 6 个元素, 总 24 个字节。

- 4、上面太麻烦了, 还得初始化, 柔性数组设计初衷的使用方式如下:

```
struct Node* pt = (struct Node*)malloc(sizeof(struct Node) + 10 * sizeof(float));
```

```
free(pt);
```

//sizeof(struct Node) 是结构体的大小, 即前三个成员的大小,

//10 * sizeof(float) 是数组 d 的大小

//用完直接 free 即可

在链表操作中

```
struct Node
{
    int a;
    short b[10]; //只能是普通数组
    double c;
    struct Node* next;
    float d[0];    //柔性数组, 用来拓展结构体的空间, 必须在最后
};

struct Node* pt1 = (struct Node*)malloc(sizeof(struct Node) + 3 * sizeof(float));
struct Node* pt2 = (struct Node*)malloc(sizeof(struct Node) + 7 * sizeof(float));
struct Node* pt3 = (struct Node*)malloc(sizeof(struct Node) + 12 * sizeof(float));
pt1->next = pt2;
```



```
pt2->next = pt3;
pt3->next = NULL;
```

这样就实现了链表节点的多样性

5、大家看到这，感觉指针成员也能实现相同的功能啊，没错也可以，也是来个链表，如下：

```
struct Node
{
    int a;
    short b[10]; //只能是普通数组
    double c;
    struct Node* next;
    float *d;    //柔性数组，用来拓展结构体的空间,必须在最后
};
```

使用：

```
struct Node* pt = (struct Node*)malloc(sizeof(struct Node));
pt->d = (float*)malloc(sizeof(float) * 10);
free(pt->d); //先放这个
free(pt);    //再放这个
```

- 1、看起来麻烦一些，因为两个 malloc 的空间必须分别释放，推荐写法一下释放
- 2、pt->d 指向的空间是游离的，跟结构体 pt 不是整体。而推荐写法是 1 个 malloc，整个空间是连续的，这更体现了结构体的整体性，使用效率更高，所以推荐写法更好。
- 3、推荐写法的柔性部分，只能最后一个成员，如果有个指针成员，那就只能单独 malloc 了。

6、VLA: variable-length arrays

变量表示长度的数组，即元素个数是变量，不是数组长度可变，数组从定义的那一刻起，整个声明周期内，其大小都是固定不可变的。形式如下：

```
int n = 12; //在用于元素个数时，必须有确定值
int a[n];    //元素个数用变量，vs2022 不支持，不能进行初始化，因为是运行时分配空间

scanf("%d", &n); //动态定义大小
int b[n];        //必须有确定值
```

int(*p)[n]; //不是数组，是数组指针，此时叫 VM, variably-modified types
要求，必须是自动存储类，即局部变量，并且声明时不可进行初始化

```
extern int n;
int a[n];    //错误
static int n = 12;
int a[n];    //错误
```

部分编译器的最新标准支持这个写法，vs2022 还不支持。

编译环境内查看宏：__STDC_NO_VLA__ 如果是 1，表示不支持 VLA

上边这种用法不是 VLA 的初衷，初衷是用在参数上，如下：

```
void fun(int m, int n, int a[m][n])
{
```

```
}
```

声明:

```
void fun(int m, int n, int a[m][n]);
```

```
void fun(int, int, int a[*][*]); //声明可以不写变量名, 这时用星号代替
```

7、初始化指定元素

数组

```
int a[10] = { [2] = 3, [5] = 6 }; //将下标为 2, 5 的元素分别初始化为 3, 6, 其余为 0
```

```
int b[3][4] = {[2][1] = 3, [0][2] = 6}; //将下标为 21, 02 的元素分别初始化为 3, 6, 其余为 0
```

```
int a[3] = {}; //c23 将支持, 相当于 int a[3] = {0};
```

结构体

```
struct Node
```

```
{
```

```
    int a;
```

```
    double b;
```

```
    float c;
```

```
    short d[5];
```

```
};
```

```
struct Node a = { .b = 12.3, .d = {1, 2, 4} };
```

```
struct Node b = {}; //c23 将支持, 相当于 struct Node b = {0};
```

联合

```
union Node
```

```
{
```

```
    int a;
```

```
    double b;
```

```
    float c;
```

```
    short d[5];
```

```
};
```

```
union Node a = { .b = 12.3 }; //只能初始化 1 个成员
```

8、复合文字: 运算符

数组:

```
int a[5] = { 1, 2, 3, 4, 5 };
```

```
(int[5]) { 1, 2, 3, 4, 5 }; //定义了一个无名的 5 元素数组, 返回地址
```

```
int *p = (int[5]) { 1, 2, 3, 4, 5 }; //定义指针接着
```

```
int a[5] = (int[5]) { 1, 2, 3, 4, 5 }; //不可以, 数组不能互相赋值
```

结构体:

```
struct Node
```

```
{
```

```
    int a;
```

```

    double b;
    float c;
};
struct Node a = { 12, 12.3, 3.4f };
a = (struct Node){ 14, 14.3, 4.4f }; //可以直接赋值, 因为结构体可以, 无名的机构体变量
struct Node *p = &(struct Node){ 14, 14.3, 4.4f };

```

9、restrict 限定符, 或者叫类型修饰符

const volatile restrict(c99) _Atomic(c11 可选支持)

restrict 用来修饰指针, 表示该指针是所指向空间的唯一且初始的指针, 所以对于该指针的操作, 编译器可以进行整合优化。

```

int* restrict p = (int*)malloc(12);
*p = 1;
*p += 2;
*p += 3;

```

此时, 编译器可以整合优化为, 当然编译器会不会这样做, 那得看它有没有实现这个优化。

```

*p = 6;

```

多个同一限定符, 只有一个有效

```

int* restrict restrict restrict p = (int*)malloc(12);

```

_Atomic(c11 可选支持): 原子限定符, 修饰的变量是原子变量。用于多线程开发中。
可选支持的意思就是, 编译器厂商可以设计也可以不设计, C 语言没有, C++有。

10、尾随逗号: 加不加都行

```

enum AA { a, b, c, d, e, };
int a[4] = { 1, 2, 3, 4, };

```

11、浮点型的 16 进制表示法, 简称 p 记法

在浮点型部分讲过这个

%a : 格式说明符, 浮点型的 16 进制形式

举例 : 0xa.23p5

p5 就是 2 的 5 次方, 即 32

```

a      10 * 32      320
0.2    2 * (1/16) * 32    4
0.03   3 * (1/256) * 32    0.375
...
320 + 4 + 0.375 == 324.375

```

12、浮点型环境: fenv.h, 浮点型的舍入方向, 浮点型的计算状态

13、主函数：C99 标准主函数写法

```
int main(void)
{
    return 0;
}

int main(int argc, char* argv[])
{
    return 0;
}
```

其中 return 0; 可以不写，编译器默认给加上。

14、变量可以随时定义了

C99 之前，变量的声明与定义必须放在所在作用域的最前面，如下：

```
int a = 12;
a = 34;
int b = 13; //报错了，必须放最前面
```

如下：

```
int a = 12;
int b = 13; //可以
a = 34;
```

C99 起，变量定义就没有顺序可言了，随用随定义即可

15、for 循环

```
for (int i = 0; i < 10; i++)
    printf("%d ", i);

//C99 起允许循环控制变量定义在小括号内，i 是 for 循环结构的局部变量，

int i;
for (i = 0; i < 10; i++)
    printf("%d ", i);

//建议写成如此，比较通用
```

16、inline：内联函数

形式如下：

```
inline void fun(void)
{
}

}
```

内联函数具有内部链接作用域，即 static 的同功能，只在所在文件有效。

正常的函数调用是：调用处跳转到代码块内，执行完再跳回调用处，这个过程经过两次跳跃，花费一定的时间。

内联函数，编译期，在调用处将代码直接黏贴过去，相当于没有调用，也就没了跳过的过程，这在代码的执行上加快的速度。代码的使用角度跟普通函数没有区别，区别在于编译器的处理角度，优化角度。

优点是执行快，缺点是，如果函数代码很多，函数调用次数也很多，那么该段代码会在源代码中复制黏贴很多分，这大大增加了代码的量，从而增加生成的软件的体积。所以，一般内联函数的代码块很小，几行代码，并且调用的频率很高。

但是实际上，编译器会自行根据条件判断某个函数是否以内联处理，这是编译器的优化功能。即使某个函数不是内联函数，编译器有可能以内联处理。某个函数声明为内联函数，编译器有可能不把它做内联处理。因为人并不是机器，主观上无法判断是否真的合适。

17、函数

函数没有参数时，加 `void`，表示不接受任何参数，即没有参数，什么都不加表示参数不确定

```
void fun1() {} //参数不确定
void fun2(void) {} //无参数
fun1(12, 3); //一切正常
fun2(12, 3); //有警告
```

函数形式

```
void fun1(a, b, c)
int a, b; //老版 K&R C 的写法，尽量不要用了，c23 就删除了
double c; //形参写变量名，然后下面说明变量类型
{
    printf("%d, %d, %lf", a, b, c);
}
void fun2(int a, int b, double c) //新版本了
{
    printf("%d, %d, %lf", a, b, c);
}
```

预定义标识符： `__func__`

用来获取所在函数的函数名

```
void helloworld(void)
{
    puts( __func__ ); //输出函数名 helloworld
}
```

数组函数参数时，修饰符可以放紧挨着变量名的方括号里，如下：

```
void fun1(const int a[]) {}
void fun2(int a[const]) {} //vs2022 还不支持这种
```

18、可变参数宏： `__VA_ARGS__`

可变参数宏

```
#define PRINT(...) printf(__VA_ARGS__)
PRINT("%d,%lf", 12, 34.5);
```

...表示参数可变，`__VA_ARGS__` 用来获取可变参数，即传递的所有东西，直接替换该宏处

```
#define PRINT(...) printf("%d,%lf\n", __VA_ARGS__)
PRINT(12, 34.5);
```

预定义宏

`__STDC_HOSTED__` (C99) 如果是 1，表示本地系统支持标准 C 库

`__STDC_ISO_10646__` (C99) 支持最新版本的 iso10646 标准，即最新的 unicode 字符集的版本

`__STDC_IEC_559__` (C99) 如果支持浮点数 IEC60559 标准时定义为 1，否则数值是未定义 ieee754

`__STDC_IEC_559_COMPLEX__` (C99) 如果复数支持 IEC60559 标准时定义为 1

`__STDC_MB_MIGHT_NEQ_WC__` (C99) 如果 'x' != 'L'x' 则为 1

`__STDC_UTF_16__` (C11) 如果支持 `char16_t`，则为 1 `uchar.h`

`__STDC_UTF_32__` (C11) 如果支持 `char32_t`，则为 1 `uchar.h`

`__STDC_ANALYZABLE__` (C11) 附录 L 被支持为 1，各种错误检查情况

`__STDC_LIB_EXT1__` (C11) 附录 K 被支持为 1，一系列边界检查_s函数的支持

`__STDC_NO_ATOMICS__` (C11) 如果不支持原子算数运算，设为 1

`__STDC_NO_COMPLEX__` (C11) 如果不支持复数算数运算，设为 1

`__STDC_NO_THREADS__` (C11) 如果不支持线程操作，设为 1

`__STDC_NO_VLA__` (C11) 如果支持 VLA，则为 1

`__STDC_IEC_60559_BFP__` (C23) 如果支持 2 进制浮点运算

`__STDC_IEC_60559_DFP__` (C23) 如果支持 10 进制浮点运算

19、_Pragma

前面学过 `#pragma`，作用是为编译器设置编译指示

`_Pragma` 作用一样，形式对比如下：

```
_Pragma("once")

#pragma once
```

二者功能上没有区别，仅仅是形式上有点儿区别，那么新的形式有啥好处么？

写上完整的预处理指令，很麻烦，我们梦想通过宏来实现：如下：

```
#define ONCE #pragma once
```

这样，每次设置仅需要：`PACK(4)` 即可设置对齐数，非常方便，但是却不能实现，因为 `define` 后的 `#`，会被作为字符串说明符，而不是预处理符号，所以 `#pragma`，语法报错了

有了新形式，就可以了，因为没有 `#`了，形式如下：

```
#define ONCE _Pragma("once")

ONCE //使用
```

20、新的标准库

`<complex.h>` `<fenv.h>` `<inttypes.h>` `<stdbool.h>` `<stdint.h>` `<tgmath.h>`

C11

1、原子类型以及原子操作库: vs2022 的 C 目前不支持, C++里支持

`__STDC_NO_ATOMICS__` (C11) 如果不支持原子算数运算, 设为 1, 确实不支持

定义形式:

```
_Atomic const int* p1;
const atomic_int* p2;
const _Atomic(int)*p3;
```

2、存储类说明符: `_Thread_local`

学过的: `auto register static extern`

`_Thread_local` 修饰的变量是线程作用域的变量, 比如

`_Thread_local int a = 0;`

多个线程使用这个变量, 相当于多个线程每个线程一份, 互不影响。

他可以跟 `extern static` 合用。

3、获取类型最大字节对齐数: `_Alignof` 运算符

`_Alignas` 设置最大字节对齐数

`<stdalign.h>` 这个头文件部分讲过这个知识点

4、字符前缀

```
char c1 = 'a';
char c2 = u8'a'; //c23, VS2022 暂不支持
wchar_t c3 = L'a'; //<wchar.h>
char16_t c4 = u'a'; //<uchar.h> c11
char32_t c5 = U'a'; //<uchar.h> c11

char* s1 = "abcde";
char* s2 = u8"abcde"; //c99
wchar_t* s3 = L"abcde"; //<wchar.h>
char16_t* s4 = u"abcde"; //<uchar.h> c11
char32_t* s5 = U"abcde"; //<uchar.h> c11
```

5、泛型编程: `_Generic`

它的功能类似 C++ 的函数重载, 其执行逻辑跟 `switch` 特别像, 几乎一样, 形式如下:

```
_Generic( x, //参数, 其类型 决定下面执行哪个分支
    long double: printf("<x>是 long double 类型"), //逗号结尾
    double:      printf("<x>是 double 类型"),
    float :      printf("<x>是 float 类型"),
    default:     printf("<x>不是浮点类型") //最后一个分支不用逗号了
);
```

执行逻辑: `x` 是什么类型的, 就执行哪个分支的代码, 分支内的代码随意, 是 C 语言合法语句即可
简单应用逻辑

```
int sum_int(int a, int b) { return a + b; }
double sum_double(double a, double b) { return a + b; }
```

```
float sum_float(float a, float b) { return a + b; }
```

```
#define Sum(x,y) _Generic( (x+y), \    //通过两个参数的计算结果决定调用哪个
    int: sum_int(x,y), \
    double: sum_double(x,y), \
    float: sum_float(x,y), \
    default: printf("无此类型")\
)
```

调用:

```
printf ("%d\n", Sum(12, 13)); //12+13 是 int, 所以调用 int 分支
printf ("%lf\n", Sum(12.6, 13.7)); //12.6+13.7 是 double, 所以调用 double 分支
printf ("%lf\n", Sum(12.6f, 13.7)); //12.6f+13.7 是 double, 所以调用 double 分支
printf ("%lf\n", Sum(12, 13.7f)); //12+13.7f 是 float, 所以调用 float 分支
```

6、stdnoreturn.h 头文件: **_Noreturn**

vs2022 还不能支持这个, 但是有了相同的功能替代它, c23 也弃用了它

vs2022 中为: `__declspec(noreturn)`

不需要任何头文件, 用来修饰无返回值的函数

7、匿名结构体与联合

匿名就是没有名字 并且 没有定义变量 的意思, 用在结构体嵌套里, 如下:

```
struct nei
{
    int a;
    struct    //匿名结构体, 不能有名字。有就炸了
    {
        double b;
        float f;
    };
    long c;
    struct
    {
        double b;
        float f;
    } w;    //有了变量名, 非匿名结构体
};
```

```
struct nei n = {12, 3.4, 5.6f, 21L};
printf("%d,%lf,%f,%ld", n.a, n.b, n.f, n.c);
//匿名结构体可以使用 n 直接访问, 相当于在结构体内进行小分组
```

n.w.b = 12.3; //非匿名对象必须通过其变量访问

n.w.f = 3.4f;

8、静态断言：_Static_assert static_assert

编译时检测

举例如下：

```
static_assert(2+2==4, "表达式为假");
```

参数 1：整型常量表达式

参数 2：表达式为假时，错误提示信息

应用：

```
static_assert(__STDC_NO_VLA__ == 0, "env not support VLA");
```

9、标准库

<stdalign.h> <uchar.h> <stdatomic.h> <stdnoreturn.h> <threads.h>

10、文件操作模式：u x

x 附加到 w w+ 上，如果文件存在，则文件打开失败，防止对原文件内容覆盖，否则，原文件被无差别覆盖。如下：

u 是可重复打开文件，正常 w/a 打开文件，是不允许被二次打开的，把 u 附加到 w/a 上，那么就可以二次打开了，如下：

11、边界检测函数组

在标准库里有大量的_s 版本的函数，前面讲解标准库已经遇见所有_s 的函数了，这块就不再重复说了，非常多。

C17

same as C11，被用来代替 C11，没有增加新特性，在 C11 基础上进行了补充与修正，即优化 ATOMIC_VAR_INIT 被弃用

realloc() 可以申请 0 字节空间，C23 又弃用

__STDC_VERSION__ 版本值为 201710L

C23：目前还没正式发布，内容还不完善，介绍部分

1、10 进制浮点型：_Decimal32, _Decimal64, _Decimal128 目前 vs2022 还不支持

32	非规格化：	$\pm 1 \cdot 10^{-101}$	DF
	近 0 值：	$\pm 1 \cdot 10^{-95}$	
	无穷：	$\pm 9.999'999 \cdot 10^{96}$	
64	非规格化：	$\pm 1 \cdot 10^{-398}$	DB
	近 0 值：	$\pm 1 \cdot 10^{-383}$	
	无穷：	$\pm 9.999'999'999'999'999 \cdot 10^{384}$	
128	非规格化：	$\pm 1 \cdot 10^{-6176}$	DL
	近 0 值：	$\pm 1 \cdot 10^{-6143}$	
	无穷：	$\pm 9.999'999'999'999'999'999'999'999'999 \cdot 10^{6144}$	

2 进制浮点型: float, double, long double

区别:

比如: 1.6

2 进制浮点型存储的是 1.6 的二进制, 是不精确的, 1.599999999 或者 1.60000000001

10 进制浮点型直接存储 1.6, 是精确的, 可以用 == 比较

等以后支持了这个, 给大家测试讲解 10 进制浮点型的存储标准。

2、2 进制整数表示法: 0b10111001

整数分隔符: ' 三个一组, 在语言环境部分讲解过这个东西

```
99'999'999'999
```

3、设置代码属性, C 还不支持, C++ 支持

[[deprecated("i dont like")]] //deprecated 弃用的意思, 字符串是弃用信息, fun 不能被使用

```
void fun(void)
```

```
{
    printf("hello world");
}
```

fun(); //编译报错, 错误原因就是那个字符串, 只作用于函数

这种东西的作用: 用来注释源代码, 一代一代库源代码更新, 旧的不需要的内容, 可以用此标注, 这样别人用这个代码的时候, 使用的旧函数, 就能得到错误提示。

[[fallthrough]] : 用于 case 标签, 消除两个标签间无 break 的警告

```
switch (1)
{
    case 1:
        printf("hello case 1\n");
        [[fallthrough]];
    case 2: //不报警告
        printf("hello case 2\n");
    case 3: //报警告, 缺少 break
        printf("hello case 3\n");
}
```

[[nodiscard("i dont like")]] 用来提示函数返回值是否被使用

```
int fun(void)
```

```
{
    printf("hello world");
    return 1;
}
```

```
int a = fun();
```

```
fun(); //有警告
```

[[maybe_unused]] 消除未使用的实体警告

int b; //如果 b 变量没有被使用，编译器会有警告，未使用的变量 b

[[maybe_unused]] int b; //警告没有了

[[noreturn]] 表明函数没有返回值

```
void fun(void)
```

```
{
```

```
    printf("hello world");
```

```
}
```

4、函数参数形式

```
void fun(int , int) //允许这种写法
```

```
{
```

```
    printf("hello world");
```

```
}
```

```
fun(1,2);
```

```
void fun1(const int *a , int) //修饰*a
```

```
{
```

```
    printf("hello world");
```

```
}
```

```
void fun2(const int a[10], int) //修饰*a
```

```
{
```

```
    printf("hello world");
```

```
}
```

```
void fun3( int* const a, int) //修饰 a
```

```
{
```

```
    printf("hello world");
```

```
}
```

```
void fun3(int a[const 10], int) //const 就要放进方括号里
```

```
{
```

```
    printf("hello world");
```

```
}
```

5、静态断言

```
_Static_assert (expression, message) (C11)
```

```
_Static_assert (expression) (C23)
```

6、goto 的标签后，加个空语句 ；

7、新增两个条件编译： #elifdef #elifndef